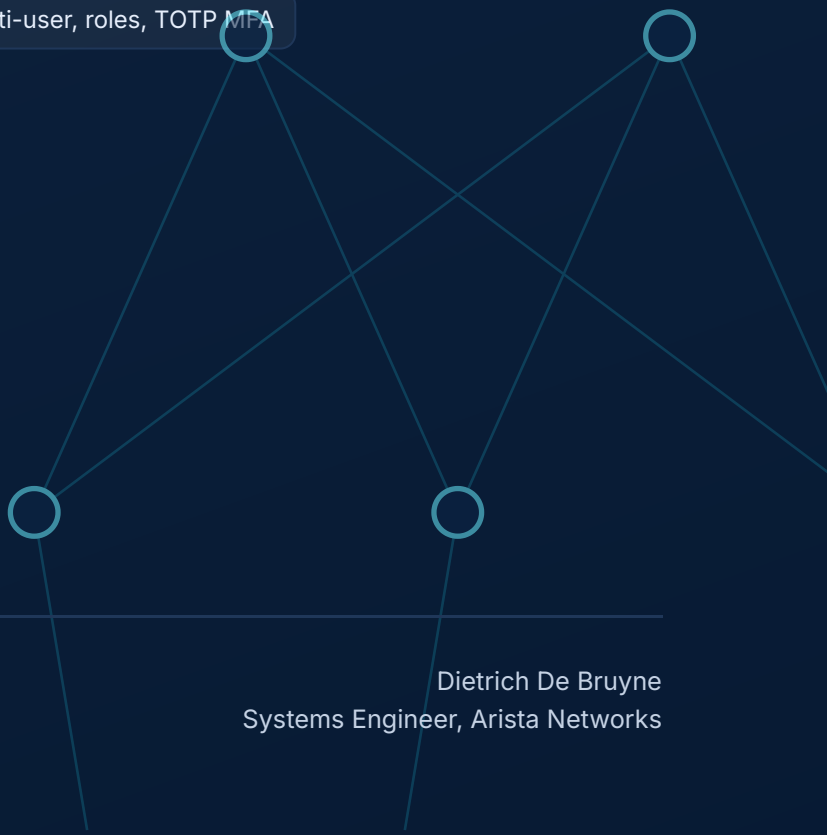
avdlab

AVD · ANTA · CONTAINERLAB

Documentation release 1.1

The web-administered Arista AVD lab orchestrator: templates → knobs → build → deploy → validate → CloudVision, with the architecture, schematics and a page-by-page guide.

- AVD 6.4 examples as templates
- containerlab + cEOS on Linux, Mac or Windows hosts
- ANTA 1.10 validation and PSIRT
- CloudVision, NetBox, ACT
- Bring-your-own-model AI designer
- Multi-user, roles, TOTP MFA



v1.1.0

22 October 2026

Dietrich De Bruyne
Systems Engineer, Arista Networks

Contents

Ch. 0 **Overview** 3

Elevator pitch 3
The documents 3
How to read 3
Quick start (LAN, five minutes plus image download) 4
Feature map 4
What avdlab 1.1 does not do 5
Diagrams 5
Versions pinned by avdlab 1.1 5
Where this documentation comes from 6
Getting help 6

Ch. 1 **Concepts** 7

1.1 One rule: the AVD inputs are the source of truth 7
1.2 The pipeline 8
1.3 Templates and knobs 9
1.4 The objects 9
1.5 Jobs 11
1.6 Where things run 11
1.7 Beginner and advanced mode 11
1.8 Users and roles 11
1.9 Vocabulary 11

Ch. 2 **Architecture** 13

(a) Components 13
(b) Deployment topologies 16
(c) Lab life cycle: create → build → deploy → validate 18
(d) Job engine 20
(e) Data model 21
(f) Security model 23
(g) Request path through Caddy 27

Ch. 3 **Installation** 29

3.1 Mac mini or any Docker host on a LAN 29
3.2 Azure VM with public HTTPS 30
3.3 Upgrading 31
3.4 Backups 34
3.5 Settings reference 34

Ch. 4 **Lab hosts** 39

4.1 Requirements 40
4.2 Adding a Linux host 40
4.3 Bootstrap recipe clab-host 40
4.4 Verify 42
4.5 Mac and Windows lab hosts (beta) 42
4.6 Probe output explained 43
4.7 Host monitor 43
4.8 Images and architecture 44

Ch. 5 **User guide** 45

5.0 The typical workflow 45
5.1 Shell: navigation, top bar, footer 45
5.2 Sign-in, account requests 46
5.3 Dashboard 47

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.4 Getting started 48
5.5 Hosts 49
5.6 Images 51
5.7 Labs 52
5.8 New lab 53
5.9 A lab: header and tabs 55
5.10 CloudVision page (targets) 66
5.11 NetBox page (targets) 66
5.12 Devcontainer (advanced) 66
5.13 Learn 67
5.14 Account 68
5.15 Users (admin) 69
5.16 Admin settings (admin) 70
5.16a Audit log (admin, 1.1.0) 71
5.17 Feedback 71
5.18 Changelog and About 71
5.19 Jobs (advanced) 71

Ch. 6 AI designer 73
6.1 How a design call works 73
6.2 The tools the model may call 75
6.3 Providers 76
6.4 Configuration 76
6.5 Privacy 76
6.6 Limits and behaviour to expect 77

Ch. 7 API reference 78
7.1 Conventions 78
7.2 WebSockets 78
7.3 Routes per router 79
7.5 Example: build, deploy and validate from a script 85

Ch. 8 Operations 86
8.1 Jobs and logs 86
8.2 Troubleshooting matrix 87
8.3 Monitoring 91
8.4 Updating 92
8.5 Versioning and releases 92
8.6 Rotating the secret-store key 93

Ch. 9 Development 94
9.1 Repository layout 94
9.2 Running it locally 94
9.3 Tests 95
9.4 How templates and knobs are defined 96
9.5 How to add a job 96
9.6 How to add a page or a lab tab 97
9.7 Conventions that keep the codebase coherent 97
9.8 Design contracts 98
9.9 Releasing 98
9.10 CI, code scanning and dependency updates (1.1.0) 98

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 0

Overview

What avdlab is, how to read this book, and a five-minute quick start.

This is the reference documentation for **avdlab 1.1.0**: the web-administered Arista AVD lab orchestrator built in this repository (FastAPI backend + React SPA). Everything below is derived from the code at version `1.1.0` and from `CHANGELOG.md`, which is the feature history. 1.1.0 is the **security hardening release**: it changes how sessions, secrets, the audit trail, the container, the self-update and the lab-host bootstrap work; the behaviour of labs themselves is unchanged from 1.0.x. The folder was renamed from `docs/1.0/` to `docs/1.1/` with this release.

Elevator pitch

1. avdlab turns "I want an Arista EVPN/VXLAN lab" into a few clicks in a browser.
2. You pick one of the official **AVD 6.4 examples** as a template and adjust its **knobs** (spines, leaf pairs, pools, tenants, CloudVision).
3. The knobs write real **AVD inputs** (`inventory.yml` + `group_vars/`), validated by **pyavd** and committed to a git repo per lab.
4. **Build** runs pyavd in-process: structured configs, EOS CLI, documentation, ANTA catalogs and a **containerlab** topology.
5. **Deploy** copies the lab over SSH to a Linux **lab host** and starts the cEOS-lab switches with containerlab.
6. **Validate** runs **ANTA 1.10** from an agent container next to the switches and keeps every report.
7. Optionally the lab is onboarded into **CloudVision** (CVaaS or on-prem), synced into **NetBox**, or targeted at **Arista Cloud Test**.
8. Test hosts, traffic generators, web terminals, config diffs, incremental config pushes and an ANTA PSIRT scan make it a working lab, not a slide.
9. An optional **AI designer** (bring your own model) turns a sentence into validated knob changes — it never writes YAML.
10. Multi-user (admin / operator / viewer, TOTP MFA, server-side sessions, audit trail), runs as an unprivileged, read-only container on a Mac mini on a LAN or on an Azure VM behind Caddy with Let's Encrypt.

The documents

#	FILE	WHAT IT ANSWERS	READ IT WHEN
—	README.md	this index	first
1	01-concepts.md	the mental model: inputs as source of truth, template → knobs → build → deploy → validate → CloudVision, the objects (labs, hosts, images, jobs), beginner/advanced mode, roles	before you click anything
2	02-architecture.md	components, deployment topologies, the lab life-cycle sequence, the job engine (concurrency, redaction), the data model, the 1.1.0 security model, the request path	you operate, extend or audit the app
3	03-installation.md	install on a Mac mini / LAN Docker host, on an Azure VM with public HTTPS; the hardened compose stack and the `update` profile; upgrading 1.0.x → 1.1.0; backups; every `AVDLAB_*` setting	you install or upgrade
4	04-lab-hosts.md	lab host requirements, the `clab-host` bootstrap recipe (signed Docker repo, pinned containerlab), lab tree permissions, Mac (OrbStack VM) and Windows (WSL2) beta hosts, jump hosts, probe output, sudo, images and CPU architecture	you add a lab host
5	05-user-guide.md	page by page and lab tab by lab tab: what every control does, the typical workflow, the `single-dc-l3ls` knobs, Account sessions, the Audit log page	you use the app
6	06-ai-designer.md	how *Design with AI* and *Start from a sentence* work: tools, dry run, repair round, apply, providers, private base URLs, privacy, limits	you want to use or approve the AI designer
7	07-api.md	REST and WebSocket reference: one table per router with method, path, role and purpose; all job types and their parameters; the 1.1.0 status codes	you script avdlab or debug the SPA
8	08-operations.md	jobs and logs, a troubleshooting matrix built from every field fix in the changelog and the 1.1.0 refusals, monitoring and audit export, key rotation, updating, the release rules	something is wrong, or you run releases
9	09-development.md	repo layout, running backend/frontend/mock mode, lint and tests, the lockfile, how templates and knobs are defined, how to add a job, a page or a template, secure-coding conventions, CI/CodeQL/Dependabot, release	you change the code
—	`../security/`	the security documentation set: security architecture, policies, incident response, compliance matrix	you review or certify a deployment

How to read

- **New user, network background:** 01 → 05 (follow the "typical workflow" at the top) → 04 when you add your first host.
- **Installer / operator:** 03 → 04 → 08, with 02 §(b) and §(f) for the network and security picture. Upgrading from 1.0.x: read 03 §3.3 *Upgrading from 1.0.x to 1.1.0* first.
- **Developer:** 02 → 09 → 07; the design contracts in `docs/CONTRACT-*.md` are the original per-phase specifications.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

- **Security review:** start with `docs/security/` (security architecture, policies, incident response, compliance matrix), then 02 §(f) security model (what the code does), 03 §3.5 (every hardening setting and its default), 06 §6.5 (privacy of the AI designer), 07 (roles and status codes per route), 08 §8.3 (audit export) and 09 §9.7/§9.10 (secure-coding conventions, CI gates).

PDF edition. `python3 scripts/docs-pdf/build.py` renders this whole set (cover, contents with page numbers, re-themed diagrams, screenshots of the UI in mock mode) to `dist/avdlab-<version>-documentation.pdf`; `scripts/release.py patch --tar DIR --pdf` does it as part of a release. See `scripts/docs-pdf/README.md`.

Conventions used throughout:

- `monospace` = a literal value: a file path, a setting, an API path, a knob key, a job type.
- "Lab host" = the Linux machine (or Linux VM) that runs containerlab and the cEOS containers. "Orchestrator" = the machine that runs the avdlab container. They can be the same machine but usually are not.
- Every long-running action is a **job** (`lab.build`, `lab.deploy`, …). The UI shows its live log in the job drawer.
- BETA marks features that are implemented but not yet field-tested (macOS / Windows lab hosts, jump hosts). PREVIEW marks upstream features that are themselves previews (ANTA `psirt`).

Quick start (LAN, five minutes plus image download)

```
git clone <repo> ~/avdlab && cd ~/avdlab/deploy
cp .env.example .env            # set AVDLAB_ADMIN_PASSWORD; AVDLAB_SESSION_SECRET may stay empty (generated once)
docker compose up -d --build    # → http://<docker-host>:8080, sign in as admin, set up the authenticator (admins need MFA)
```

Then, in the browser: **Hosts** → *Generate SSH key* → put the public key on a Linux box → *Add host* → *Probe* → *Bootstrap* → **Images** → save your arista.com token → *Download* `4.34.2F` → **Labs** → *New lab* → `single-dc-l3ls` → *Create lab* → *Build* → *Deploy* → *Validate*. Chapter 3 covers the public HTTPS install, chapter 5 §5.0 the workflow in detail.

Feature map

FEATURE	SINCE	CHAPTER
Hosts over SSH: probe, `clab-host` bootstrap recipe, verify, TOFU host keys, sudo password	0.1.0 / 0.1.1	4
Templates from all AVD 6.4 examples + `custom` import; pyavd build; git per lab; file editor	0.1.0, 0.7.0	1, 5
containerlab deploy / destroy / status, ANTA validation with stored reports	0.2.0	2, 5
CloudVision targets, onboarding (TerminAttr), `cv_deploy`, device tags, pre-flight	0.3.0–0.8.0	5
Linux clients, ping/iperf traffic, running configs, config diff, snapshots, web terminals	0.4.0	5
Per-lab git remote, archive import/export, host monitoring, cEOSarm, VS Code links	0.4.0	4, 5
ZTP provisioning, NetBox sync, MSS foundation, Arista Cloud Test target, kill job/container	0.5.0	5
Editable topology, changelog, feedback, About	0.6.0	5
Update check and self-update, in-app AVD learning, interactive test hosts	0.7.0	3, 5, 8
arista.com token management, image picker	0.7.1	5
Accounts, roles, password policy, TOTP MFA, audit log; public HTTPS with Caddy	0.8.0	2, 3, 5
CloudVision-inspired UI, beginner/advanced mode, account requests, admin settings, e-mail	0.9.0	1, 5
AI lab designer (bring your own model)	0.10.0	6
New lab from a sentence, Getting started, incremental config push, ANTA PSIRT scan, host health	0.10.1	5, 6
Lab hosts on macOS (OrbStack VM) and Windows (WSL2), beta; arm64 default for arm hosts	1.0.0	4
SSH jump host per lab host BETA	1.0.1	4
Explicit platform choice when adding a host (Linux / Mac / Windows) with setup steps; stale jobs marked failed at startup	1.0.2	4, 8
PDF edition of this documentation (`scripts/docs-pdf/build.py`)	1.0.3	README
Server-side sessions (list, revoke, sign out everywhere), absolute lifetime, generated session secret, login rate limit, MFA required per role, TOTP replay guard	1.1.0	2, 5, 7
Secret ownership and purpose, reserved names, Fernet key rotation (`python -m avdlab.cli rotate-secrets`)	1.1.0	2, 5, 8
Audit trail with actor, new events, retention, `/admin/audit` page with filters and NDJSON export	1.1.0	5, 7, 8
CSP / Permissions-Policy headers, Origin guard, trusted proxies, `/meta` stripped, `/meta/build`	1.1.0	2, 3, 7
Hardened container (uid 10001, read-only root, no capabilities, limits), docker-socket-proxy `update` profile, digest-pinned images, no default secrets	1.1.0	3
Self-update needs a digest-pinned release image, optional keyless cosign verification	1.1.0	3, 8
Job engine: concurrency limit, central log redaction, log cap, job directory retention	1.1.0	2, 7
Upload size cap and `replace` flag, ACT password as an encrypted knob, git remote / branch / SMTP validation	1.1.0	5, 7, 8

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

FEATURE	SINCE	CHAPTER
Bootstrap from the signed Docker repository with a pinned, checksum-verified containerlab; private lab trees (0700) shared with the agent gid	1.1.0	4
AI base URLs on private addresses refused unless an admin allows them	1.1.0	6
Secure SDLC: ruff, `requirements.lock`, SBOM, CI with SCA / secrets / Trivy, CodeQL, Dependabot, signed releases with provenance	1.1.0	9

What avdlab 1.1 does not do

- It does not run cEOS on macOS or Windows directly — always in a Linux VM (chapter 4).
- It does not resume jobs after an API restart; the job engine is in-process and single-node (chapter 2 §(d)).
- It does not import NetBox back into AVD, onboard ACT labs into CloudVision, or provide a full MSS design (the MSS tab is a foundation).
- It has no `local` host kind (containerlab through the orchestrator's own Docker socket without SSH) yet.
- It does not resume a job queue across restarts, and the login rate limit and AI rate limit are per process (not shared between replicas).

Diagrams

Every diagram is kept as Mermaid source (`diagrams/<name>.mmd` , also inline in the documents so GitHub renders it) and as a pre-rendered SVG (`diagrams/<name>.svg`) for viewers without Mermaid support.

DIAGRAM	USED IN	TYPE
`concepts-pipeline`	01	flowchart: template → knobs → inputs → build → deploy/validate/CloudVision/NetBox
`lab-states`	01	state diagram of a lab (`draft` … `destroyed`)
`components`	02	component diagram
`deploy-lan` , `deploy-azure` , `deploy-jump`	02	deployment diagrams
`seq-lifecycle`	02	sequence: create lab → build → deploy → validate
`job-states`	02, 08	job engine state machine
`data-model`	02	ER diagram of all tables
`security`	02	where credentials live and travel
`request-path`	02	a request through Caddy and the middlewares
`host-setup`	04	add host → probe → bootstrap → verify
`ai-designer`	06	sequence of a design call
`self-update`	03	sequence of the in-app update

To re-render after editing a `.mmd` file:

```
cd docs/1.1/diagrams
for f in *.mmd; do npx -y @mermaid-js/mermaid-cli@11 -i "$f" -o "${f%.mmd}.svg" -b transparent; done
```

(add `-p <puppeteer.json>` with `{"args": ["--no-sandbox"]}` when rendering as root in a container).

Versions pinned by avdlab 1.1

COMPONENT	VERSION	WHERE IT IS SET
pyavd / AVD collection	6.4.0	`backend/pyproject.toml` , `backend/requirements.lock` , `AVDLAB_AVD_VERSION` (native toolchain + display)
ANTA	1.10.0	`AVDLAB_ANTA_VERSION` , `AVDLAB_ANTA_IMAGE=ghcr.io/aristanetworks/anta:1.10.0`
Agent image	`ghcr.io/aristanetworks/avd/universal:python3.11-avd-v6.4.0` (digest-pinned in `agent/Dockerfile`)	`AVDLAB_AGENT_IMAGE`
containerlab (native install by bootstrap)	0.79.0, checksum-verified GitHub release	`AVDLAB_CLAB_VERSION`
containerlab (container mode)	`ghcr.io/srl-labs/clab:0.79.0` (was `:latest` in 1.0.x)	`AVDLAB_CLAB_IMAGE`
Client containers	`ghcr.io/srl-labs/network-multitool:latest`	`AVDLAB_HOST_IMAGE` , knob `client_image`
Default cEOS-lab	4.34.2F	knob `ceos_version` in every template
Python (API)	3.11 (`python:3.11-slim` , digest-pinned)	`deploy/Dockerfile.backend`
Node (SPA build)	22 (`node:22-bookworm-slim` , digest-pinned; `npm ci` from `package-lock.json`)	`deploy/Dockerfile.backend`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

COMPONENT	VERSION	WHERE IT IS SET
React / react-router	React 18, `react-router-dom` 7.18.4	`frontend/package.json`
Update helper	`docker:27-cli@sha256:851f91d2…`	`AVDLAB_UPDATE_HELPER_IMAGE`
docker-socket-proxy (profile `update`)	`tecnativa/docker-socket-proxy:0.3.0` (digest-pinned)	`deploy/docker-compose.yml`
Caddy (profile `public`)	`caddy:2` (digest-pinned)	`deploy/docker-compose.yml`
Postgres (profiles `postgres`, `netbox`)	`postgres:16-alpine` (digest-pinned)	`deploy/docker-compose.yml`
NetBox (optional profile)	v4.2 image (tag, operator-configurable)	`NETBOX_VERSION`

Where this documentation comes from

FACT	SOURCE OF TRUTH IN THE REPO
Feature history, field fixes	`CHANGELOG.md`
Routes and roles	`backend/avdlab/main.py`, every `**/api.py`, `core/security.py` (`role_denial`)
Settings	`backend/avdlab/core/config.py`, `core/passwords.py`, `core/app_settings.py`, `deploy/.env.example`, `deploy/docker-compose.yml`
Security controls	`core/security.py` (sessions, RoleGuard, OriginGuard, secret store), `core/audit.py`, `core/ratelimit.py`, `jobs/engine.py` (redaction), `meta/update.py`; the policies in `docs/security/`
Templates and knobs	`templates/*/template.yml`, `backend/avdlab/labs/templates.py`
Navigation and pages	`frontend/src/components/Layout.tsx`, `frontend/src/pages/**`
Host platforms	`frontend/src/pages/Hosts.tsx` (`PLATFORMS`), `backend/avdlab/hosts/probe.py`, `docs/multi-os-hosts.md`
Release procedure	`scripts/release.py`, `scripts/sbom.py`, `.github/workflows/{ci,codeql,release}.yml`

Getting help

- In the app: the **Feedback** button on every page (with optional diagnostics) or its **E-mail** button.
- Start of every troubleshooting session: the job log of the failed job (job drawer, or `/jobs/<id>`), then the matrix in 08-operations.md §8.2.
- For field reports on the beta features (Mac/Windows lab hosts, jump hosts), include the probe output of the host.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 1

Concepts

The mental model: inputs as source of truth, the pipeline, the objects, modes and roles.

This chapter explains how avdlab thinks about a lab. If you know Arista AVD from the Ansible collection, most of it will feel familiar: avdlab is AVD plus containerlab plus ANTA, driven from a browser, with every action recorded.

1.1 One rule: the AVD inputs are the source of truth

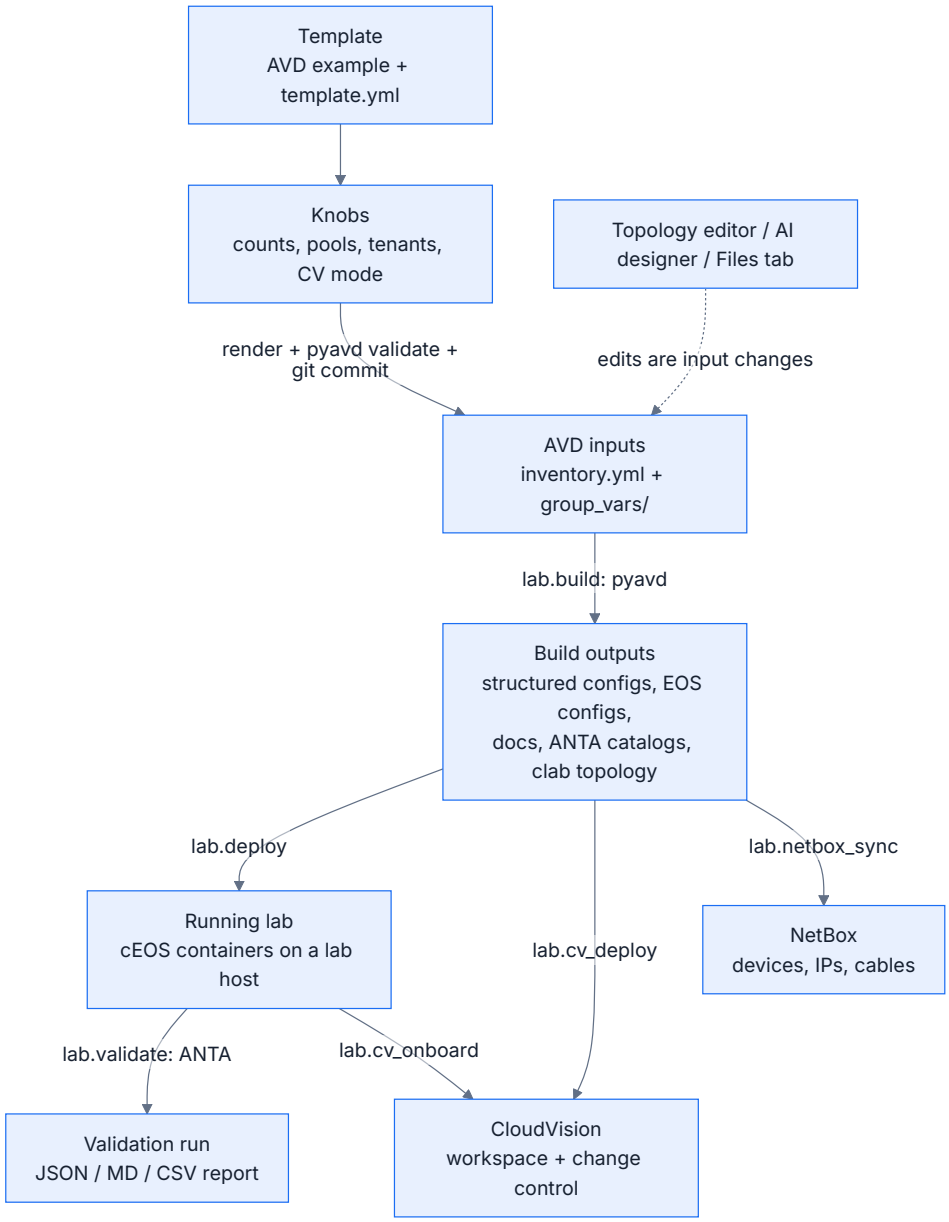
A lab in avdlab is an **AVD inventory**: `inventory.yml` plus `group_vars/` in the standard AVD layout, inside a git repository on the orchestrator (`/data/labs/<slug>/`). Nothing else is authoritative.

- avdlab never invents its own network model. Configs, the containerlab topology, the ANTA inventory and catalogs, NetBox objects and CloudVision tags are all **derived** from AVD's outputs (structured configs, fabric topology CSV, metadata).
- Every way of changing a lab — the knob form, the file editor, the topology editor, the AI designer, a hot-added test host — ends up as a change of those inputs, validated with `pyavd.validate_inputs`, and as a **git commit** of the lab repository. You can see and diff every revision on the Build tab.
- Because it is a normal AVD directory, the same lab can be opened in VS Code (Git & VS Code tab), exported as a zip, pushed to a git remote or cloned back in, and built with `ansible-playbook` outside avdlab.

The AI designer follows the same rule strictly: the model can only propose knob values and topology operations; it never writes YAML (chapter 6).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

1.2 The pipeline



Pipeline from template to CloudVision

The stations, in the order the Getting started page presents them:

STATION	WHAT HAPPENS	WHERE IN THE UI	JOB
1. Host	A Linux machine with Docker reached over SSH is probed and bootstrapped (Docker, containerlab, sysctls, agent image).	Hosts	`host.probe`, `host.bootstrap`, `host.verify`
2. Image	A cEOS-lab version is downloaded from arista.com (your token) or uploaded, and pushed to hosts.	Images	`image.download`, `image.push`
3. Lab	A template plus knob values becomes AVD inputs in a git repo. Nothing runs yet.	Labs → New lab	— (inline API call)
4. Build	pyavd produces structured configs, EOS CLI configs, device and fabric documentation, ANTA catalogs, and avdlab turns the fabric topology CSV into `clab/topology.clab.yml`.	Lab header *Build*, Build tab	`lab.build`
5. Deploy	The lab directory is copied to the host, containerlab starts the cEOS nodes with their intended config as startup-config, avdlab waits for eAPI.	Lab header *Deploy*, Runtime tab	`lab.deploy`
6. Validate	ANTA runs the catalog AVD generated for this fabric (BGP, MLAG, VXLAN, interfaces, …) from the agent container on the lab's management network.	Lab header *Validate*, Validation tab	`lab.validate`
7. CloudVision	The switches are onboarded (TerminAttr + onboarding token) and the configs and tags are pushed as a CloudVision workspace with `arista.avd.cv_deploy`.	CloudVision page, lab CloudVision tab	`lab.cv_onboard`, `lab.cv_deploy`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Optional side branches: NetBox sync (`lab.netbox_sync`), Arista Cloud Test as a second execution target, clients and traffic, MSS segmentation (foundation), ZTP provisioning.

1.3 Templates and knobs

A **template** is an AVD 6.4 example (inventory, group_vars, playbooks) plus a descriptor `template.yml` that lists the **knobs** the UI exposes. Shipped templates:

TEMPLATE	GENERATOR	WHAT IT IS
`single-dc-l3ls`	`l3ls`	eBGP underlay + EVPN/VXLAN overlay, MLAG leaf pairs, L2 leafs; **fully generated** from counts, pools, ASNs, tenants and servers. The default first lab.
`dual-dc-l3ls`	`static`	two L3LS fabrics joined by EVPN gateways
`single-dc-multipod-l3ls`	`static`	five-stage Clos with super-spines and two pods
`l2ls-fabric`	`static`	layer-2 leaf-spine, MLAG spines as gateways
`campus-fabric`	`static`	campus L2LS with IDFs, `network_ports`, 802.1X port profile
`cv-pathfinder`	`static`	CV Pathfinder WAN (control plane only on cEOS)
`isis-ldp-ipvpn`	`static`	MPLS core: IS-IS + LDP, VPN-IPv4, PE/P/RR
`single-dc-l3ls-ipv6`	`static`	IPv6 underlay, optional dual-stack management (`mgmt_ipv6_subnet`)
`custom`	`none`	your own AVD inventory, imported from an archive or git

The **generator** decides what knobs can do:

- `l3ls` — the whole inventory and group_vars for the node types are generated from the knobs (counts, pools, ASNs, serial scheme, tenants, servers). Changing `leaf_pairs` really adds leaf pairs.
- `static` — the example's group_vars are loaded as they are; knobs apply **targeted edits** (management subnet and gateway re-based, device password, serial numbers, platform, DNS, CloudVision, …). For anything else you edit group_vars in the Files tab.
- `none` (`custom`) — the files are the source of truth; knobs only shape the containerlab topology (image, architecture, client image, device tags, provisioning, segmentation, ACT).

Knob types are `int`, `string`, `cidr`, `ipv4`, `pool` (AVD pool syntax, validated), `secret` (stored encrypted, returned masked), `object`, `yaml` and `bool`. Every knob has help text and a `learn` block (summary, AVD keys it renders, pitfalls, doc link) shown as (i) popovers in learning mode.

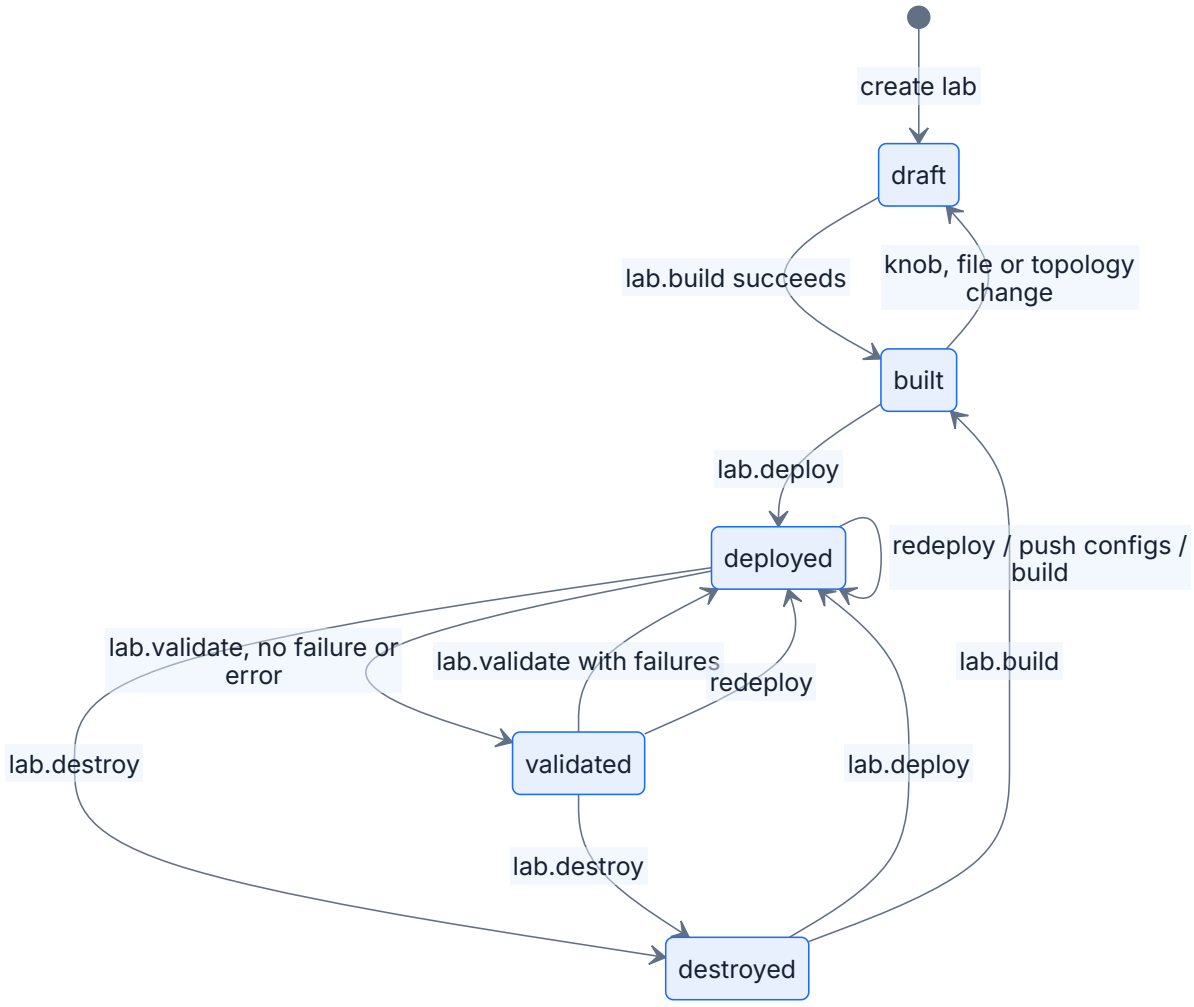
Saving knobs re-renders the inputs. For generated and static templates, a knob save rewrites the knob-managed files (`inventory.yml`, `group_vars/`). Hand edits in those files mark the related knobs as *custom*, but the next knob save replaces them (the topology editor warns about this). Topology edits on generated labs are stored as `topology_overrides` in the lab settings and re-applied after every knob render, so they survive.

1.4 The objects

OBJECT	WHAT IT IS	STORED IN
Host	A lab host: SSH address, user, port, optional jump host, platform (linux / mac / windows), credentials (references to secrets), kind `linux` (containerlab host) or `act` (ACT tools server), and the capabilities document of the last probe.	`hosts` table; host keys in `/data/known_hosts`
Image	A cEOS-lab image: format (`cEOS`, `cEOS64`, `cEOSarm`), version, docker tag (`arista/ceos:<v>` or `arista/ceos:<v>-arm64`), architecture, local tarball, which hosts have it.	`images` table, `/data/images/`
Lab	One AVD fabric: template, knob values, state, target (`clab` or `act`), host, cEOS version, optional CloudVision and NetBox target bindings.	`labs` table + git repo `/data/labs/<slug>/`
Job	One run of an action with parameters, status, result, artefacts and a log file (secrets masked, capped at 20 MiB, directory kept 90 days).	`jobs` table + `/data/jobs/<id>/log.txt`
Secret	An encrypted value: SSH key, password, arista.com token, CloudVision token, NetBox token, generic (GitHub token, SMTP password, AI keys). Since 1.1.0 every secret has an **owner** (who created it) and a **purpose** (`ssh`, `host`, `git`, `arista`, `cloudvision`, `netbox`, `ai`, `generic`): operators see and bind only their own secrets, admins see all. Values are never returned by the API.	`secrets` table (Fernet ciphertext, rotatable keys)
CloudVision target	A CVaaS tenant or on-prem cluster with a service-account token, optional onboarding token and ZTP script.	`cv_targets`
NetBox target	A NetBox URL with a write-only API token.	`netbox_targets`
Validation run	One ANTA run of a lab: totals and the reports.	`validation_runs` + `anta/runs/<id>/` in the lab repo
Session	One sign-in of a user from one browser (1.1.0): id, IP, user agent, created / last seen / expires. The cookie only carries the id; revoking the row ends the session.	`sessions` table
Audit event	Who did what to whom, when and from where: sign-ins, account, secret, host and lab changes (1.1.0 adds the actor).	`audit_log` table, pruned after `AVDLAB_AUDIT_RETENTION_DAYS`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Lab states



Lab state machine

STATE	MEANING	HEADER BUTTONS ENABLED
`draft`	inputs exist; intended configs missing or stale	Build
`built`	build outputs match the inputs	Build, Deploy
`deployed`	containers were started (or the last validation had failures/errors)	Build, Validate, Destroy (Deploy becomes *Redeploy* on the Runtime tab)
`validated`	the last ANTA run had no failure and no error	Build, Validate, Destroy
`destroyed`	containerlab destroyed the topology	Build, Deploy

Notes: building a running lab does not change its state — the Runtime and Build tabs then offer **Push config changes** for the switches whose intended config changed. Changing the lab target (`clab` ↔ `act`) is refused while the lab runs and resets a built lab to `draft`. A lab can be deleted only when it is not running.

Targets

TARGET	EXECUTION	NOTES
`clab` (default)	containerlab on a Linux lab host over SSH	everything in this documentation unless stated otherwise
`act`	Arista Cloud Test: the build runs pyavd in digital-twin mode and writes an ACT topology YAML (with a tools server and client nodes) that you upload in act.arista.com; deploy pushes configs from the ACT tools server (a host of kind `act`) over eAPI	no containerlab, no incremental push, no traffic/snapshot jobs

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

1.5 Jobs

Everything that takes longer than a page load is a **job**: the API answers `202 {job_id}` immediately and the job runs as an asyncio task in the API process. Its log is written to `/data/jobs/<id>/log.txt` and streamed to the browser over a WebSocket; the **job drawer** opens automatically when you start one. Jobs can be killed (the task is cancelled and its SSH sessions are closed, so remote commands die too). Most lab actions refuse to start while a conflicting job of the same lab is running (HTTP 409 naming the running job). Since 1.1.0 at most `AVDLAB_JOB_MAX_CONCURRENT` (4) jobs run at once; further jobs wait as `queued` (first in, first out, still cancellable) and say so in their log. Every secret a job reveals is registered for masking, so it appears as `***` in the log.

Short read-only actions run **inline** with a time budget instead (lab status 20 s, running config 25 s, CloudVision and NetBox connection tests 15 s, ZTP status 15 s).

1.6 Where things run

RUNS ON THE ORCHESTRATOR (API CONTAINER)	RUNS ON THE LAB HOST
knob rendering, pyavd validation, **build**	containerlab, the cEOS containers
the git repositories of all labs	a copy of the lab directory in `/opt/avdlab/labs/<slug>/` (`AVDLAB_REMOTE_ROOT`)
the database, secret store, job logs	the **agent container** (`AVDLAB_AGENT_IMAGE`): ansible, pyavd, ANTA — used for validate, cv_deploy, config pushes
NetBox sync (pynetbox), AI designer calls, e-mail	the ANTA 1.10 image for PSIRT scans, the linux client containers, the `ztp` node

The switches' management network exists only on the lab host. avdlab never talks eAPI to a switch from the orchestrator: eAPI checks run as `curl` on the host, ANTA and Ansible run in containers attached to the containerlab management network. That is why a lab host in the cloud, on the LAN or in a VM on a laptop all behave the same.

1.7 Beginner and advanced mode

The top bar has a **Beginner / Advanced** toggle (remembered per browser, default *Beginner*).

	BEGINNER	ADVANCED
Navigation	hides NetBox, Devcontainer, Jobs	everything
Lab tabs	hides MSS, NetBox, ACT, Git & VS Code (a *Show advanced* link at the end of the tab row switches mode)	all tabs (ACT only for target `act`)
Hints	one-line hints on Hosts, Images, Labs, New lab and the Build / Topology / Validation tabs	hidden
Dashboard	welcome banner, intro video, tiles, next steps	adds hosts, labs and recent-jobs tables

Hidden pages stay reachable by URL; beginner mode only declutters. Separately, **learning mode** (switch at the bottom of the sidebar) adds knob help popovers, click-to-explain on config viewers and glossary hover cards.

1.8 Users and roles

avdlab has individual accounts with three roles, enforced by the backend on every `/api/*` request:

ROLE	MAY
`viewer`	read everything (all GET), follow live job logs; change their own password and MFA. No web terminals, no changes.
`operator`	everything except user administration (`/api/users`), admin settings (`/api/admin`), the audit trail (`/api/audit`), the self-update and deleting hosts; sees and binds only the secrets they created
`admin`	everything

A refused call answers `403 {"detail": "role viewer may not POST /api/labs"}`. Passwords follow a policy (≥ 12 characters, 3 of 4 character classes, no username, not a common password), are stored as argon2id hashes, and accounts lock after 5 failed sign-ins for 15 minutes (the sign-in form does not reveal whether the account exists or is locked). TOTP MFA with recovery codes is **required for admins** by default (`mfa_required_roles`) and optional for other roles: an admin without an authenticator can only use the Account page until MFA is set up. A sign-in creates a server-side **session** that expires after 12 hours regardless of activity and can be revoked from the Account page. Public account requests can be enabled; an admin approves them with a role. Details: chapter 2 §(f) and chapter 5 (Account, Users, Audit log).

1.9 Vocabulary

TERM	MEANING IN AVDLAB
AVD	Arista Validated Designs; here the `pyavd` 6.4 Python package (eos_designs + eos_cli_config_gen)
eos_designs inputs	the fabric data model in `group_vars/` (node types, pools, tenants, connected endpoints)
structured config	the per-device data AVD derives from the inputs (`intended/structured_configs/<host>.yml`)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

TERM	MEANING IN AVDLAB
intended config	the EOS CLI rendered from the structured config (`intended/configs/<host>.cfg`)
cEOS-lab	containerised EOS; one container per switch
containerlab (clab)	the tool that wires cEOS containers with veth links from `clab/topology.clab.yml`
ANTA	Arista Network Test Automation; the test framework that validates the running fabric
agent image	the container image avdlab runs on the lab host for Ansible/ANTA work (default: AVD universal image)
TerminAttr	the EOS streaming agent that connects a switch to CloudVision
cv_deploy	the `arista.avd.cv_deploy` role that pushes configs and tags into a CloudVision workspace
knob	a template parameter exposed in the UI; writes AVD inputs
connected endpoint	a server/firewall/... attached to leaf ports in AVD; becomes a linux client container in the lab
test host	a linux client hot-added to a running lab on a free switch port

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 2

Architecture

Components, deployment topologies, the lab life cycle, job engine, data model, security and request path.

avdlab is one container (the **api**: FastAPI + the built React SPA) plus whatever it reaches over the network: lab hosts over SSH, and optional CloudVision, NetBox, an AI provider, an SMTP server and GitHub. This chapter shows the pieces, how they are deployed, how a lab moves through them, and how credentials are protected.

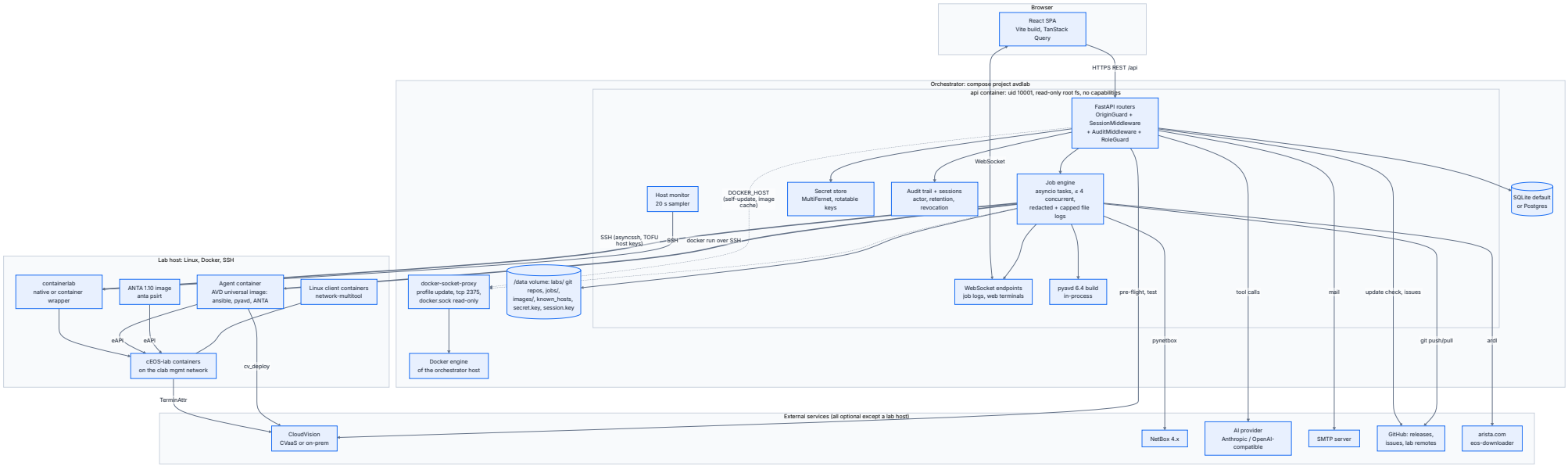
(a) Components

Browser. The SPA (React 18, TypeScript, Tailwind, TanStack Query, React Flow for the topology, xterm.js for terminals, CodeMirror for YAML) is built into `frontend/dist` and served by the same API process. Any path that is not `/api/...` and not a real file returns `index.html`, so deep links and page refreshes work.

API. One uvicorn process runs the FastAPI app (`backend/avdlab/main.py`) as the unprivileged user `avdlab` (uid/gid 10001) in a container with a read-only root filesystem (chapter 3). Routers are imported from a list of feature modules (`FEATURE_MODULES`), each registering its own routes and job types; the auth, jobs, secrets and audit routers are always present. Middleware, from outside in: `ProxyHeadersMiddleware` (only when `AVDLAB_TRUST_PROXY_HEADERS=true`, trusting `AVDLAB_TRUSTED_PROXIES` only), a security-headers middleware (`nosniff`, `DENY`, `Referrer-Policy`, `Content-Security-Policy`, `Permissions-Policy`), CORS, `OriginGuard` (the `Origin` of a mutating `/api` call or WebSocket upgrade must be ours), `SessionMiddleware` (signed cookie carrying the session id), `AuditMiddleware` (declarative audit events for lab and feedback routes) and `RoleGuard` (session row, MFA policy and role permissions). Startup (`lifespan`) creates or migrates the database, bootstraps the first admin, prunes old audit rows and dead sessions, migrates pre-1.1.0 ACT passwords into encrypted knobs, marks jobs left `running` by a previous process as failed (1.0.2), starts the host monitor, the job housekeeping, the audit pruner and the update checker, and closes a self-update that the previous process started. A malformed `AVDLAB_SECRET_KEY(S)` or a weak `AVDLAB_SESSION_SECRET` stops the process before it listens.

Job engine (`jobs/engine.py`). Single-node and in-process: every job is an asyncio task; at most `AVDLAB_JOB_MAX_CONCURRENT` (4) run at once, the rest wait as `queued`. Status lives in the `jobs` table, the log in `/data/jobs/<id>/log.txt` (every revealed secret masked, capped at `AVDLAB_JOB_LOG_MAX_BYTES`), and lines fan out to WebSocket subscribers. CPU-bound work (pyavd build, pynetbox, zipping) runs in worker threads via `asyncio.to_thread`. See (d).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Component diagram

Build. pyavd runs inside the API container: `validate_inputs` → facts → structured configs → EOS configs → device docs → fabric docs → ANTA catalogs, written in the standard AVD directories of the lab repo, committed, then `topology/clab.py` generates the containerlab topology (deduplicated links from the fabric topology CSV, per-node `EosIntfMapping.json`, a `ceos-config` file per node with a unique serial number, linux nodes for connected endpoints).

Storage. SQLite at `/data/avdlab.db` by default (Postgres via `AVDLAB_DATABASE_URL`). Schema changes are applied by `create_all` plus a small list of `ALTER TABLE ... ADD COLUMN` "light migrations" (`core/db.py::_ADDED_COLUMNS`; 1.1.0 adds `secrets.created_by`, `secrets.purpose`, `users.totp_last_step`, `audit_log.actor` and the `sessions` table). Labs are directories with a git repository under `/data/labs/`; job logs under `/data/jobs/` (directories older than `AVDLAB_JOB_LOG_RETENTION_DAYS` are removed, rows stay); downloaded images under `/data/images/`; SSH host keys in `/data/known_hosts`; the Fernet key in `/data/secret.key` unless `AVDLAB_SECRET_KEY` / `AVDLAB_SECRET_KEYS` is set; the cookie-signing key in `/data/session.key` unless `AVDLAB_SESSION_SECRET` is set. `/data` is the only writable volume — scratch files go to the `tmpfs` on `/tmp` and `/home/avdlab`.

Lab host. Only SSH and Docker are required; bootstrap installs Docker from the signed `download.docker.com` repository and containerlab (natively from the pinned, checksum-verified GitHub release `AVDLAB_CLAB_VERSION`, or as a wrapper script around `ghcr.io/srl-labs/clab:0.79.0`). avdlab copies the lab to `<AVDLAB_REMOTE_ROOT>/labs/<slug>/` (mode 0700, shared with the agent image's gid — chapter 4) and runs everything there: `sudo containerlab deploy`, `curl` against each switch's eAPI, and `docker run` of the **agent container** on the containerlab management network for ANTA (`arista.avd.anta_runner`), `arista.avd.cv_deploy` and `arista.eos.eos_config` pushes; the dedicated **ANTA image** for `anta psirt`; **client containers** for traffic tests; a `ztp` node (agent image running dnsmasq + an HTTP server) in ZTP mode.

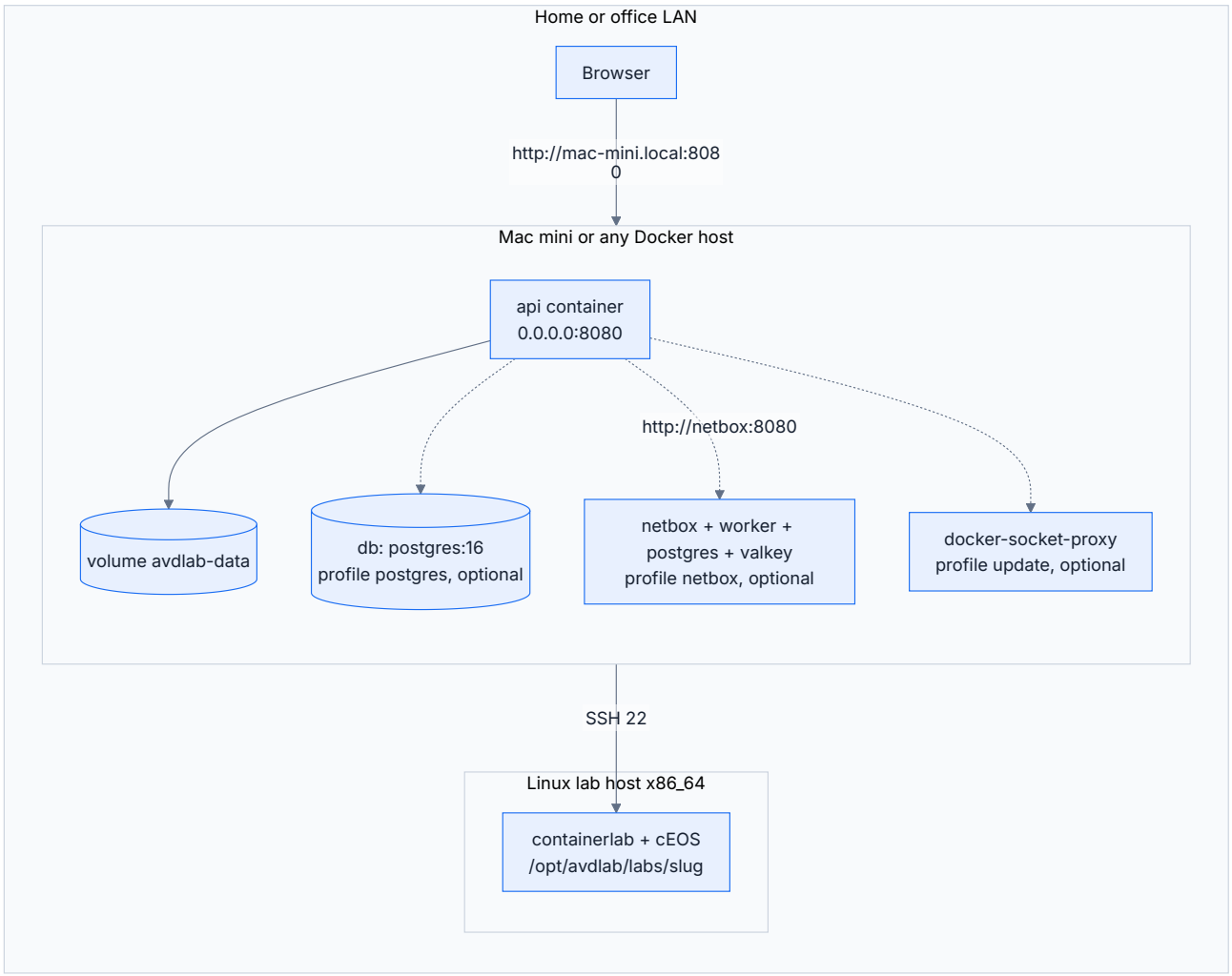
External services. CloudVision receives TerminAttr streams from the switches (through the lab host's NAT) and `cv_deploy` REST calls from the agent container; the orchestrator itself only does a 15 s pre-flight / connection test. NetBox is written from the orchestrator with pynetbox. The AI provider is called from the orchestrator with plain HTTP (httpx); a base URL that resolves to a private, loopback or metadata address is refused unless an admin allows it (chapter 6). SMTP is used for account-request and feedback mail (AUTH only over TLS unless `AVDLAB_SMTP_ALLOW_INSECURE`). GitHub is used for the update check (Releases API, `https://` only), optional feedback issues and lab git remotes (`https`, `ssh`, scp-like; local paths only with `AVDLAB_GIT_ALLOW_LOCAL`). arista.com is reached by `eos-downloader` (`ardl`) — from the orchestrator (download to the image store) or on the lab host (remote download, token on stdin).

Docker on the orchestrator. The api container has no Docker socket. When the About page's *Update now* or the local image cache is wanted, the compose profile `update` starts a **docker-socket-proxy** (`tecnativa/docker-socket-proxy`, socket mounted read-only, no published port, only the container/image/network/volume endpoints allowed) and `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` points the docker CLI inside the api at it (chapter 3).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

(b) Deployment topologies

Mac mini (or any Docker host) on a LAN

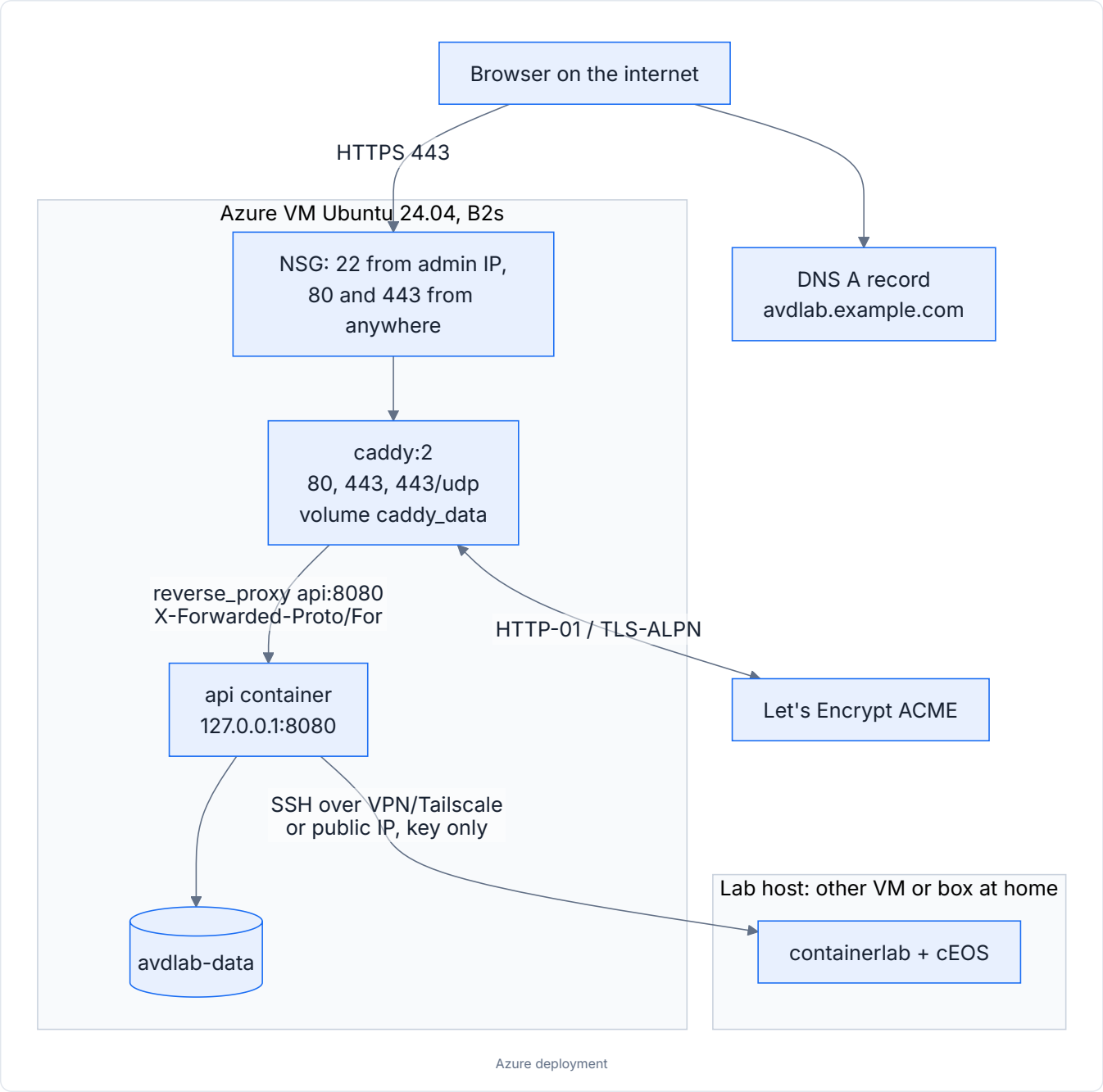


LAN deployment

`docker compose up -d --build` in `deploy/` starts the `api` service only; it publishes port 8080 on all addresses (`AVDLAB_BIND_ADDRESS=0.0.0.0`). Postgres, NetBox and the docker-socket-proxy are separate compose **profiles** (`--profile postgres` , `--profile netbox` , `--profile update`). The Mac mini runs only the orchestrator; the labs run on a separate Linux lab host (cEOS-lab for x86_64 is a Linux container and cannot run on macOS itself). The `update` profile is optional: through the proxy the API can use the local Docker engine to cache/import cEOS images and run the in-app self-update, without ever seeing `/var/run/docker.sock` itself.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

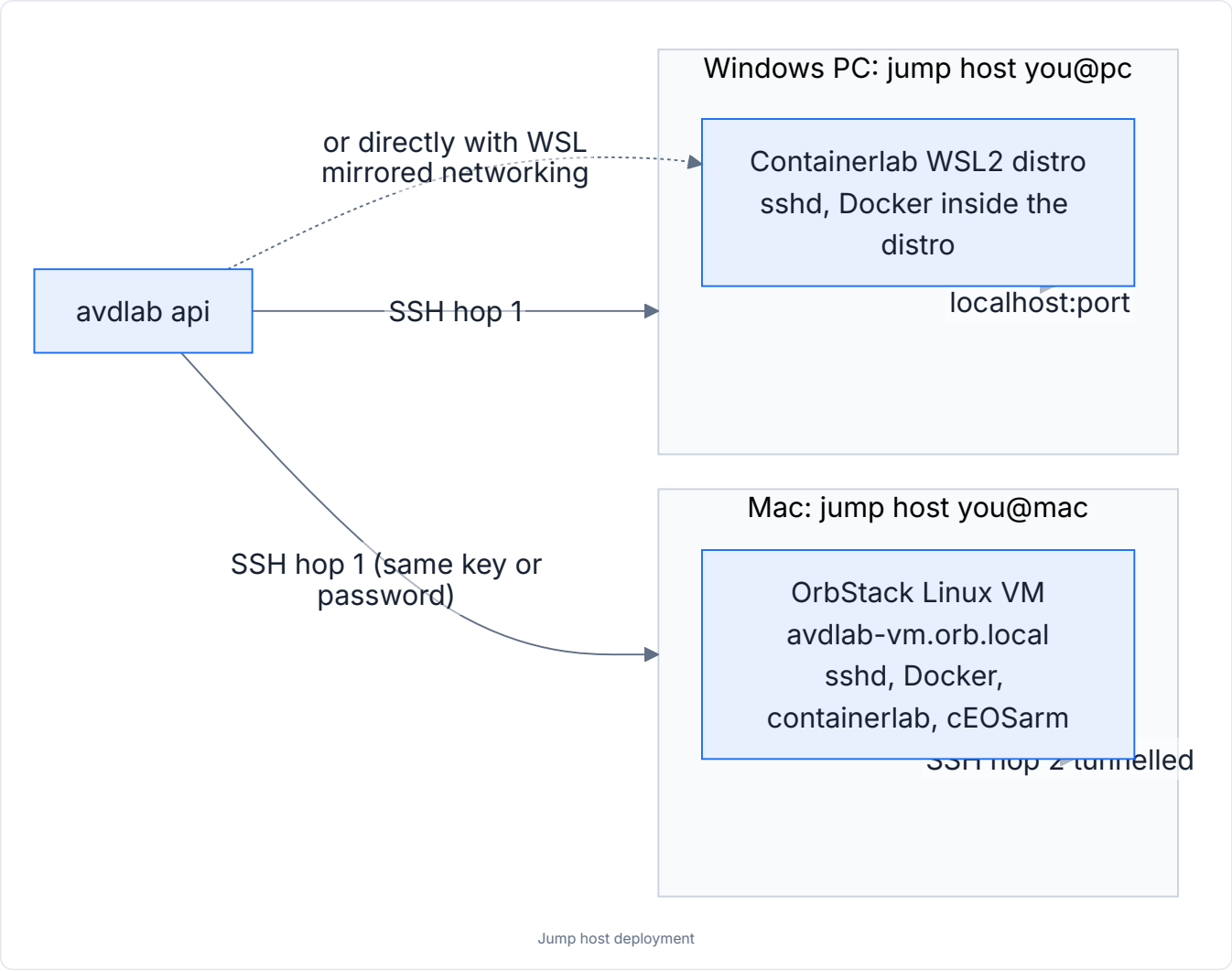
Azure VM with public HTTPS



The compose profile `public` adds Caddy 2. Caddy obtains and renews a Let's Encrypt certificate for `AVDLAB_DOMAIN` (certificates in the `caddy_data` volume), redirects port 80 to https, adds HSTS and security headers, caps request bodies (4 GB on the image upload, 16 MB elsewhere) and proxies everything — WebSockets included — to `api:8080`. The API port is bound to `127.0.0.1` only, so the app is reachable only through Caddy (or an `ssh -L 8080:127.0.0.1:8080` tunnel for debugging). The orchestrator VM is small (B2s, 2 vCPU / 4 GB); the lab host is the big machine and only needs to be reachable over SSH from the VM — through a VPN / Tailscale, or over its public IP restricted to the VM's address. `deploy/azure/setup-azure-vm.sh` automates the VM side and starts the `public` and `update` profiles together (chapter 3).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Jump host for Mac and Windows lab hosts BETA



Jump host deployment

containerlab needs a Linux kernel, so on a Mac or a Windows PC the lab host is a **Linux VM on that machine**. Those VMs are NATed behind their host, so a lab host can have a **jump host** (`[user@]host[:port]`): avdlab first connects to the Mac or PC with the same key/password, then opens the SSH connection to the VM through that tunnel. Both hops' host keys are pinned in `known_hosts` ; commands, file sync and terminals all use the one tunnelled connection. Chapter 4 has the setup steps.

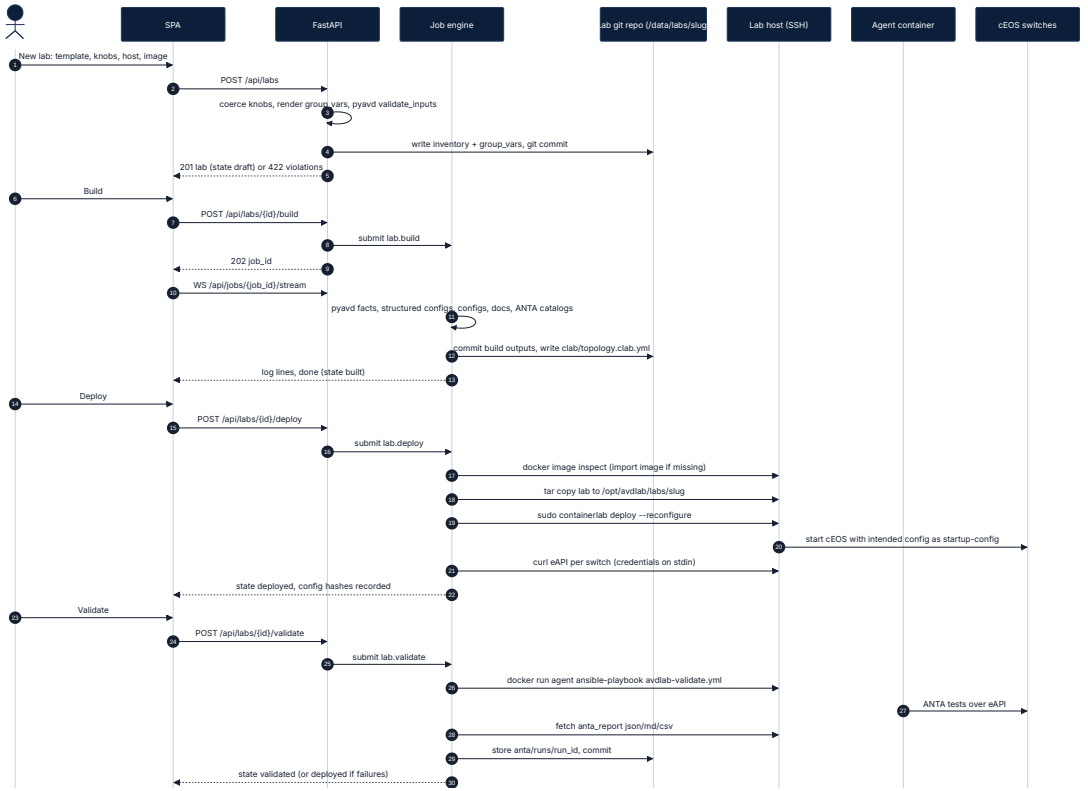
(c) Lab life cycle: create → build → deploy → validate

1–5 **Create.** `POST /api/labs` is inline: knob values are merged over the template defaults and coerced (types, ranges, pool syntax); the generator renders inventory and group_vars; pyavd validates the resolved host variables. A violation answers `422 {"detail": {"message", "violations": [{host?, path?, message}]}}` and nothing is written. Otherwise the files are committed to a fresh git repository and the lab is `draft` .

6–13 **Build.** `lab.build` runs the pyavd pipeline in a thread, streams progress into the job log, commits all outputs and generates `clab/` (and, for target `act` , the ACT topology). The job result lists the hosts, warnings, changed files and ANTA test counts shown on the Build tab.

14–21 **Deploy.** `lab.deploy` refuses a lab that was never built or a host that was never probed / lacks Docker or containerlab, and an image whose architecture does not match the host. It checks the image on the host (importing the tarball from the image store when missing), copies the lab directory (excluding `.git` and reports), removes a stale empty Docker network that still holds the lab's management subnet (left by a host crash), runs `containerlab deploy -t clab/topology.clab.yml --reconfigure` with sudo, reads `containerlab inspect` , and polls eAPI on every cEOS node until all answer or `wait_timeout` (600 s default) passes. The switches boot from their intended config (containerlab `startup-config`); in ZTP mode they boot without one and fetch it from the `ztp` node instead. The SHA-256 of every intended config is recorded so a later **Push config changes** knows what changed.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

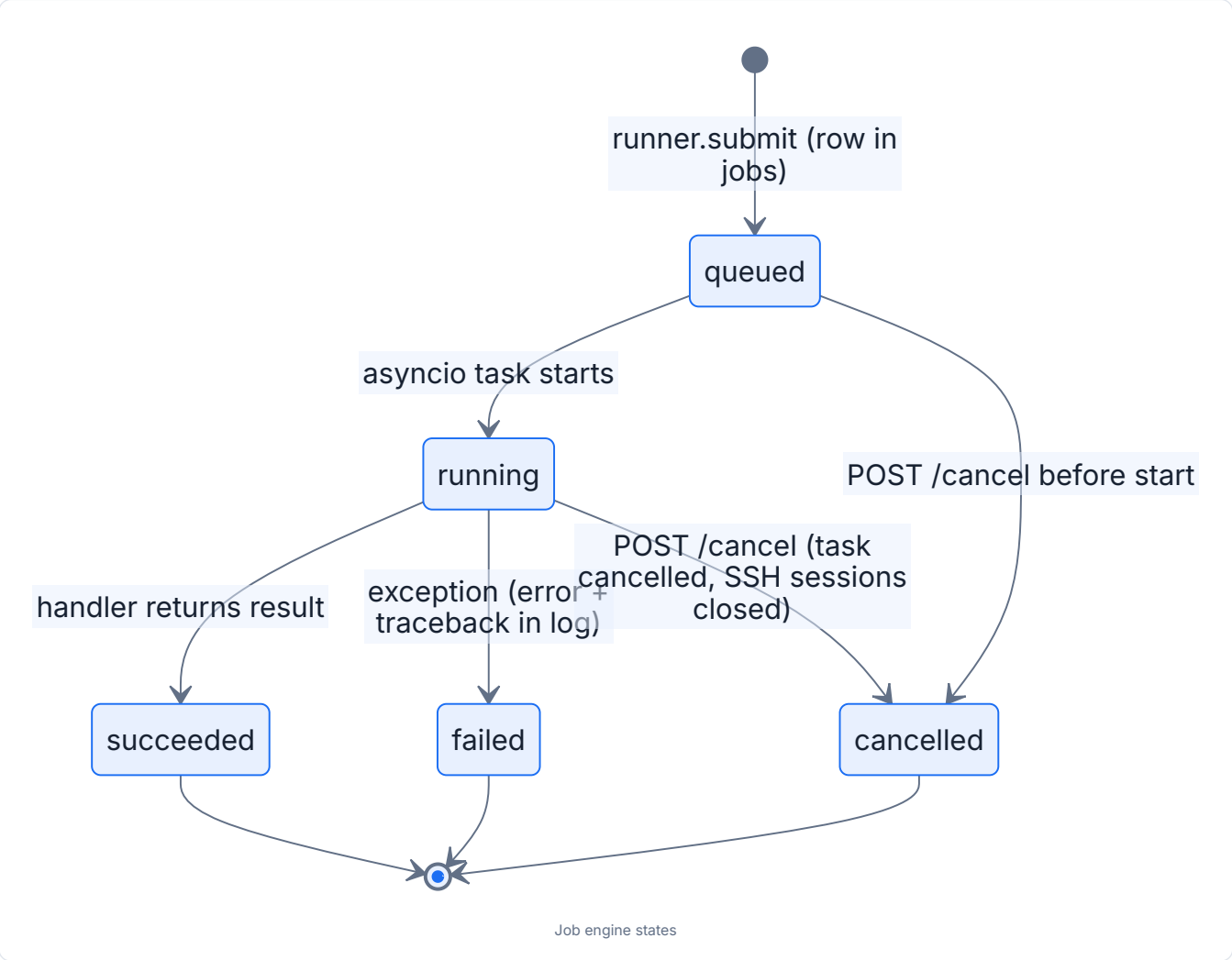


Lab life-cycle sequence

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

22–28 **Validate.** `lab.validate` writes `playbooks/avdlab-validate.yml` (AVD `anta_runner`, optional extra fabric validation, skipped tests, tags, user catalogs), checks that the agent image ships ANTA ≥ the required minimum, syncs the lab, runs the playbook in the agent container on the management network, pulls back `anta_report.{json,md,csv}`, stores them under `anta/runs/<run_id>/`, commits, and sets the lab to `validated` (no failure and no error) or keeps it `deployed`.

(d) Job engine



Job engine states

A job type is a coroutine registered with `@job("<type>")`. `runner.submit(type, params=, lab_id=, host_id=)` inserts the row (`queued`), creates a `JobContext` (log file, cancel event, redactor, list of SSH connections it holds) and starts the task. When `AVDLAB_JOB_MAX_CONCURRENT` (4; 0 = unlimited) jobs are already running the task waits first-in-first-out and writes `job queued: n of 4 job slots busy, m waiting` to its log; `runner.running` / `runner.queued` expose the counters. The task then sets `running`, logs `job <id> (<type>) started`, awaits the handler and stores its return value as the `result` (`succeeded`), or the exception text as `error` plus the traceback in the log (`failed`).

Cancel (`POST /api/jobs/{id}/cancel`, the *Kill* button) sets the cancel event, cancels the asyncio task and closes every SSH connection the job registered — remote commands die with their session. An exception raised because its SSH session was closed under it is recorded as `cancelled`, not `failed`.

Streaming. `WS /api/jobs/{id}/stream` first sends the existing log (`{"line": ...}` frames), then live lines, then `{"done": true}`. Lines are timestamped `[HH:MM:SS]`.

Redaction (1.1.0). Every `JobContext` carries a `Redactor`. Job code calls `ctx.redact(*values)` at the point where it reveals a secret (SSH or sudo password, private key, device password, ACT password, CloudVision / NetBox / arista.com tokens, git token or key); every later log line replaces those values with `***`, and a PEM block spread over several lines is collapsed. Built-in patterns are always applied: `Bearer <token>`, `token|password|passwd|pwd|secret|api_key` followed by `=` / `:`, `scheme://user:pass@host`, AWS `AKIA…` ids and private-key blocks. Values shorter than 4 characters are not registered (they would mask ordinary text). The log is capped at `AVDLAB_JOB_LOG_MAX_BYTES` (20 MiB): the file ends with `[log truncated]` and the job continues.

Retention. `<data>/jobs/<id>` directories older than `AVDLAB_JOB_LOG_RETENTION_DAYS` (90; 0 = keep) are removed at startup and daily by `runner.start_housekeeping()`; the `jobs` rows, results and lab-repo artefacts stay.

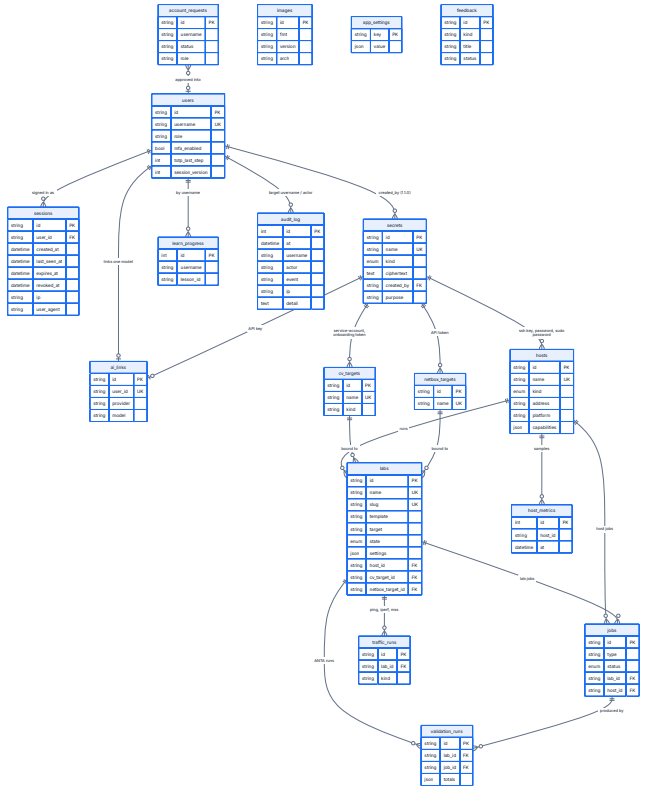
© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Single process. The engine lives in the API process. A job that was `queued` or `running` when the API stopped is not resumed; since 1.0.2 the next start marks such rows `failed` with the error *interrupted: the avdlab server restarted while this job was running*, so the lab is not blocked (see the troubleshooting matrix in chapter 8).

(e) Data model

All models are in one module, `backend/avdlab/core/db.py`. Ids are 32-character hex UUIDs (integers for the append-only tables `audit_log`, `learn_progress`, `host_metrics`). Only the columns that matter are drawn.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Data model

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

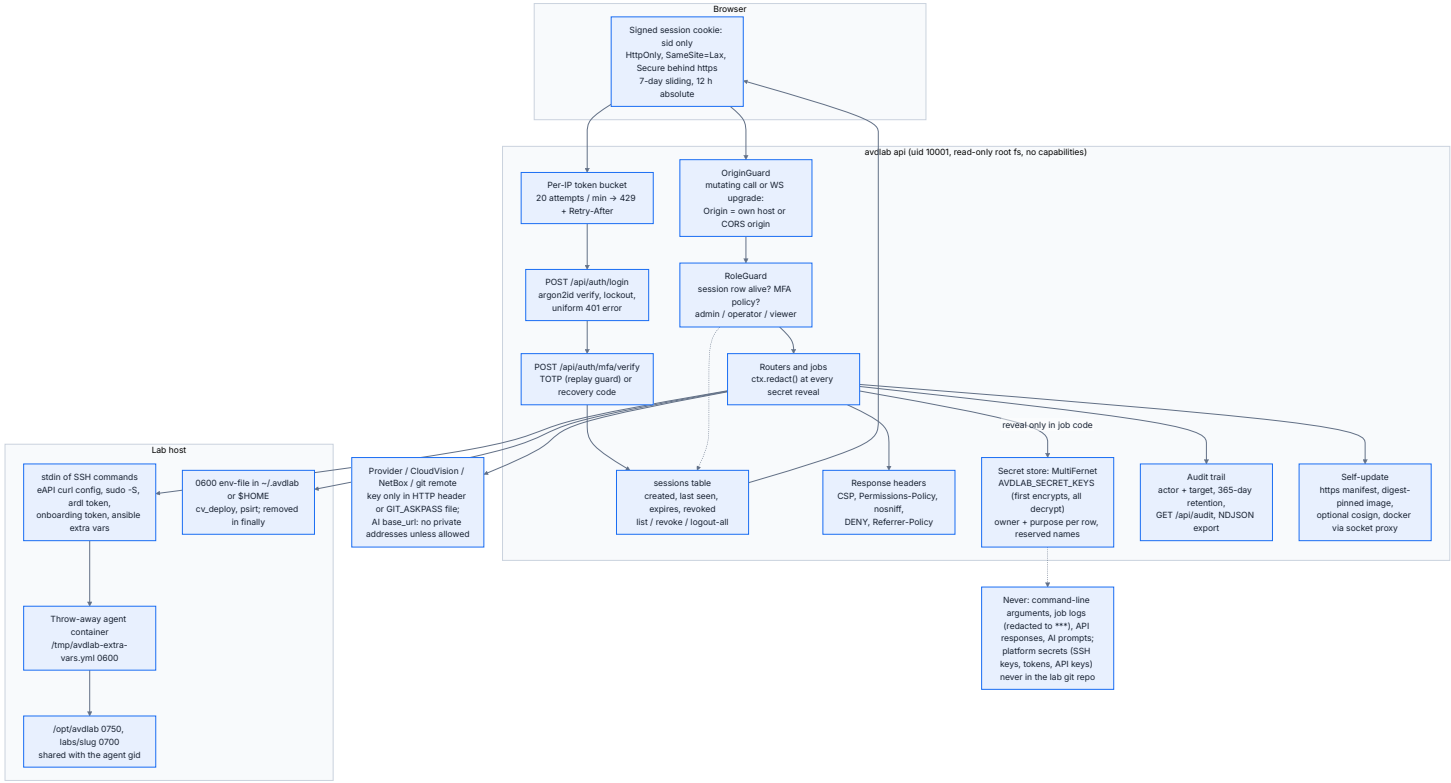
TABLE	HOLDS	NOTES
`users`	accounts	argon2id hash, role, `disabled`, `must_change_password`, TOTP secret (Fernet ciphertext), `totp_last_step` (1.1.0 replay guard: the last accepted 30 s step), hashed recovery codes, `failed_logins` / `locked_until`, `session_version`
`sessions`	server-side sessions (1.1.0)	one row per sign-in: `user_id`, `created_at`, `last_seen_at`, `expires_at` (absolute lifetime), `ip`, `user_agent`, `revoked_at`; the cookie carries only the row id; revoked and expired rows are pruned daily
`secrets`	every credential	`kind` = `ssh_key`, `password`, `arista_token`, `cv_token`, `netbox_token`, `generic`; `ciphertext` is Fernet (any configured key decrypts, the primary one encrypts); `meta` holds non-secret facts (public key); 1.1.0: `created_by` (owner, NULL = legacy/system row, admins only) and `purpose` (`ssh`, `host`, `git`, `arista`, `cloudvision`, `netbox`, `ai`, `generic`; NULL = unrestricted legacy row)
`hosts`	lab hosts	`kind` `linux` / `act`; FK to up to three secrets (key, password, sudo password); `capabilities` = last probe document
`images`	cEOS image store	unique `(fmt, version)`; `present_on_hosts` is a JSON list of host ids
`labs`	labs	`settings` JSON carries knob values, encrypted `secret_knobs` (device password and, since 1.1.0, `act.password`), `topology_overrides`, `pushed_configs`, last inspect/status/deploy/validation/CV/NetBox/ZTP results, git remote config
`jobs`	every job	`params`, `result`, `artefacts`, `error`, timestamps (the log file on disk is subject to the 90-day directory retention)
`validation_runs`	ANTA runs	totals + path of the JSON report in the lab repo
`traffic_runs`	ping / iperf / MSS verification runs	`kind` = `ping`, `iperf` or `mss`
`cv_targets`	CloudVision targets	service-account and onboarding token secrets, `verify_certs`
`netbox_targets`	NetBox targets	token secret, `verify_certs`
`account_requests`	public sign-up requests	`pending` / `approved` / `rejected`, granted role, decision metadata
`app_settings`	admin-editable settings	key → JSON value; an unset key falls back to its `AVDLAB_*` env default
`ai_links`	one linked AI model per user	API key in `secrets` (write-only)
`audit_log`	the audit trail	`username` = the target of the event, `actor` = who did it (1.1.0; NULL for anonymous events such as a failed login), `event` (`login.ok`, `login.failed`, `login.locked`, `logout.all`, `session.revoked`, `mfa.enabled`, `registration.duplicate`, `secret.created/deleted/bound`, `host.created/updated/deleted`, `lab.created/deleted`, `lab.file_written`, `lab.git_configured/removed`, `feedback.deleted`, `job.cancelled`, `ai.apply`, …), IP, user agent, `detail`; rows older than `AVDLAB_AUDIT_RETENTION_DAYS` (365) are pruned
`feedback`	in-app feedback	also written as `/data/feedback/<id>.md`; optional GitHub issue URL
`learn_progress`	passed lesson checkpoints	unique per (username, lesson, checkpoint)
`host_metrics`	monitor samples	pruned to 7 days

`ai_links.user_id`, `sessions.user_id`, `secrets.created_by`, `host_metrics.host_id`, `learn_progress.username` and `audit_log.username` / `audit_log.actor` are logical links, not database foreign keys.

(f) Security model

This section describes what the code does. The policies around it (threat model, hardening checklist, incident response, compliance mapping) are the separate set `docs/security/`.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Security model

Authentication. `POST /api/auth/login` verifies the argon2id hash. Unknown user, wrong password **and a locked account** all get the same `401 {"detail": "invalid username or password"}` (1.1.0 — there is no more `423`, so the sign-in form cannot be used to enumerate accounts; lockouts are still recorded and visible to admins on the Users page and in the audit log). After `AVDLAB_LOGIN_MAX_FAILURES` (5) failures the account is locked for `AVDLAB_LOGIN_LOCKOUT_MINUTES` (15). Both `/auth/login` and `/auth/mfa/verify` sit behind a **per-IP token bucket** (`AVDLAB_AUTH_RATE_LIMIT`, 20 attempts per minute, burst 20; `0` disables): over the limit → `429 {"detail": "too many attempts: try again later"}` with a `Retry-After` header. With MFA enabled the session first holds only `mfa_pending_user_id` (nothing is granted) and `POST /api/auth/mfa/verify` accepts a 6-digit TOTP code (30 s) or a single-use recovery code; 5 wrong codes restart the sign-in. A TOTP code is accepted **once**: `users.totp_last_step` remembers the last accepted time step and the same or an older step is refused (replay guard).

Sessions (server-side since 1.1.0). A successful sign-in creates a row in `sessions` (`user_id`, `created_at`, `last_seen_at`, `expires_at`, `ip`, `user_agent`). The cookie (Starlette `SessionMiddleware`, signed with the session secret, `AVDLAB_SESSION_MAX_AGE` = 7 days sliding) carries only the session id `sid`, the user id and the `session_version`; it is always HttpOnly and `SameSite=Lax`, and `Secure` when `AVDLAB_PUBLIC_URL` starts with `https://` (or `AVDLAB_SECURE_COOKIES=true`). Every request loads the row: a missing, revoked or expired row — the **absolute lifetime** `AVDLAB_SESSION_ABSOLUTE_MAX_AGE` is 43200 s (12 h) regardless of activity — makes the request anonymous and closes WebSockets with `4401`. Sign-out revokes the row, so a replayed cookie is useless. The Account page lists the user's sessions (`GET /api/auth/sessions`: id, current, IP, user agent, timestamps), revokes one (`POST /api/auth/sessions/{sid}/revoke`) or signs out everywhere (`POST /api/auth/logout-all`). Disabling a user, resetting a password or MFA (admin) revokes all of that user's sessions; a password change revokes the others (`session_version` bump + row revocation). Cookies issued before 1.1.0 carry no `sid` and are invalid.

Session secret. `AVDLAB_SESSION_SECRET` empty (the 1.1.0 default) → a 64-hex secret is generated once into `/data/session.key` (0600) and reused across restarts. An explicit value must be at least 32 characters; the old default `change-me-session-secret` or a shorter value is a `RuntimeError` at startup — the API refuses to run with a guessable cookie key. Several api replicas must share one explicit value.

MFA policy. `mfa_required_roles` (admin setting; env default `AVDLAB_MFA_REQUIRED_ROLES=["admin"]`) lists the roles that must have an authenticator. A user whose role is listed but has no MFA gets a **restricted session**: only `/api/auth/*` and `/api/meta*` work, everything else answers `403 {"message": "mfa_required", "detail": "mfa_required"}` (WebSocket close `4403`). `/auth/me`, the login responses and `GET /users` rows carry `mfa_required: true` and the SPA redirects to the Account page with a banner until the authenticator is enrolled.

Authorisation. `RoleGuard` (an ASGI middleware inside the session middleware) loads the user and session row, applies the MFA policy, then `role_denial(role, method, path)` to every `/api/*` request and WebSocket upgrade:

RULE	EFFECT
admin	allowed everywhere
`/api/users*`, `/api/admin*`, `/api/audit*`	admin only (routers also check `require_role("admin")`); `POST /api/meta/update/check|apply` are admin-only in the router
`/api/register*`	public (account requests)
operator	everything else, except `DELETE /api/hosts/...`
viewer	`GET` / `HEAD` / `OPTIONS`, WebSocket streams, `POST /api/auth/*` (own password, MFA, sessions); never a `.../terminal` WebSocket

Denied HTTP calls get `403 {"detail": "role viewer may not POST /api/labs"}`; denied WebSockets are closed with 4403. Anonymous requests pass RoleGuard and are answered 401 by the routers, except the public endpoints (`/api/auth/login`, `/api/auth/mfa/verify`, `/api/auth/logout`, `/api/auth/policy`, `/api/register`, `/api/register/status/{id}`, `/api/meta`, `/api/meta/changelog`, `/api/meta/notices`). `GET /api/meta` no longer exposes build facts; `git_sha`, `build_date`, `avd_version`, `anta_version`, `python` and `platform` moved to the authenticated `GET /api/meta/build`.

Origin guard. A mutating `/api` request (anything but `GET` / `HEAD` / `OPTIONS`) or a WebSocket upgrade that carries an `Origin` header must match the request host or one of the configured CORS origins (`AVDLAB_CORS_ORIGINS` + `AVDLAB_PUBLIC_URL`); otherwise `403 {"detail": "origin not allowed"}` / close `4403`. Together with `SameSite=Lax` this closes cross-site request forgery against the cookie session. Requests without an `Origin` (curl, scripts) are unaffected.

Response headers. Every response carries `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, a `Content-Security-Policy` (`AVDLAB_CSP`; the default allows `'self'` scripts, inline styles for Tailwind, `data:` / `blob:` images, `ws:` / `wss:` connections for terminals and job logs, `https:` media for a hosted intro video, `frame-ancestors 'none'`, `object-src 'none'`) and a `Permissions-Policy` (`AVDLAB_PERMISSIONS_POLICY`: camera, microphone, geolocation, payment, USB and FLoC off). Setting either to `""` disables that header (for example to let Caddy own it). Caddy adds HSTS (`max-age=31536000`), removes the `Server` header and caps request bodies; it deliberately does **not** set a CSP of its own.

Trusted proxies. With `AVDLAB_TRUST_PROXY_HEADERS=true` uvicorn's `ProxyHeadersMiddleware` honours `X-Forwarded-Proto/For` only from `AVDLAB_TRUSTED_PROXIES` (default loopback + RFC 1918), never from `*`. A proxy on a public address must be added to the list, otherwise the audit log and the session list show the proxy's IP.

Secret store. Credentials are rows in `secrets`, encrypted with Fernet. The keys come from `AVDLAB_SECRET_KEYS` (comma-separated: the **first encrypts, all decrypt** — a `MultiFernet`), with `AVDLAB_SECRET_KEY` appended as a decrypt-only key when both are set, or a key generated on first start into `/data/secret.key` (0600); malformed keys stop the process at startup. **Key rotation**: put the new key first, keep the old one after it, restart, run `python -m avdlab.cli rotate-secrets --dry-run` then without `--dry-run` (re-encrypts every secret and TOTP seed with the primary key), check with `python -m avdlab.cli check-keys` (which key opens how many rows; exit status 2 if some row opens with no key), then drop the old key (chapter 8 §8.6, full playbook in `docs/security/`). TOTP secrets of users are encrypted with the same keys — **lose every key and every secret and every MFA enrolment is unrecoverable**.

Every secret has an **owner** (`created_by`) and a **purpose** (`ssh`, `host`, `git`, `arista`, `cloudvision`, `netbox`, `ai`, `generic`; defaulted from the kind when not given). Operators and viewers list, delete and bind only their own rows (`GET /api/secrets` returns `own: true`; another user's row is a `404`);

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

admins see everything with `created_by` and `created_by_name`. The names `smtp_password`, `ai_default_api_key`, `github_token` and `system:*` are **reserved** for admins (`403`). Binding a secret to a host (`POST` / `PATCH /api/hosts`) or to git auth (`PUT /api/labs/{id}/git`, `POST /api/labs/import/git`) requires ownership (or admin) **and** a compatible purpose, else `403 {message, violations}`: a host takes `ssh` for the key and `host` for the passwords, git takes `git`. Keys generated on the Hosts page belong to the caller with purpose `ssh`. Rows created before 1.1.0 have `created_by` / `purpose` NULL: they are visible to admins only and usable in any binding by an admin; existing host bindings are not re-checked retroactively. List endpoints return names and kinds only; tokens on targets, the arista.com token, the SMTP password and AI keys are **write-only** (`has_token`-style booleans come back). Only job and service code calls `reveal()`, and every reveal is registered with `ctx.redact()` so the value cannot appear in a job log. Audit events `secret.created`, `secret.deleted`, `secret.bound` record who did what.

Audit trail. Every row of `audit_log` has a `username` (the target: the account created, the user who signed in) and an `actor` (who did it; NULL for anonymous events such as a failed login or a duplicate registration). Events cover sign-ins and lockouts, logout-all and session revocation, MFA, registration (`registration.duplicate`), secrets, hosts (`host.created/updated/deleted`), labs (`lab.created/deleted`, `lab.file_written`, `lab.git_configured/removed` — some recorded declaratively by `AuditMiddleware` on a 2xx answer, detail = `METHOD path` plus path parameters, never the body), `feedback.deleted`, `job.cancelled` and `ai.apply`. Admins query it with `GET /api/audit` (filters `since`, `until`, `actor`, `event` exact or `host.*` prefix, `username`, `limit` ≤ 1000, `offset`) and export it as NDJSON with `GET /api/audit/export` (newest first, ≤ 100 000 rows); the SPA page is *Admin › Audit log*. Rows older than `AVDLAB_AUDIT_RETENTION_DAYS` (365; `0` = forever) are pruned at startup and daily, together with dead sessions — export before they age out if you need them longer.

Where credentials travel (what reaches a lab host or an external service, and how):

CREDENTIAL	PATH	NEVER
SSH private key / password	in memory to asyncssh; registered with `ctx.redact()`	written to disk on the orchestrator outside the secret store; argv
sudo password	`sudo -S` on stdin	on a command line
device password (eAPI)	`curl -K -` with the `user = "u:p"` line on stdin; the management IP is validated with `ipaddress` and the URL `shlex.quote`d before any shell	on a command line or in the log
ACT node password	encrypted knob `secret_knobs["act.password"]` (1.1.0), masked as `********` in every lab response, decrypted only in the ACT executor / build; newline, CR, NUL refused (`422`)	plain text in `settings.knobs` (pre-1.1.0 labs are migrated at startup)
ansible credentials (config push, ACT deploy, test hosts)	stdin of `docker run -i`, copied to `/tmp/avdlab-extra-vars.yml` (0600) inside the throw-away container	`-e @/dev/stdin` (broken in Ansible, fixed in 0.7.2), host filesystem
CloudVision service-account token (cv_deploy)	0600 env-file under `~/.avdlab/` on the lab host (`executors.common.env_file_content` refuses control characters), passed with `--env-file`, removed in `finally`	the lab repo, a command line
ANTA credentials (PSIRT scan)	0600 env-file in `$HOME`, removed in `finally`	same
CloudVision onboarding token	stdin of `docker exec -i <node>` writing the TerminAttr token file (plus a copy on flash)	`docker cp` (hidden by the cEOS tmpfs), the lab repo
ZTP bootstrap script / onboarding token	written by the deploy job into the remote `clab/ztp/www/` with mode 0600 via stdin	the lab repo
arista.com token	env of the local `ardl` process, or stdin of a remote `ardl`	logs
git remote token / SSH key	throw-away `GIT_ASKPASS` script reading a 0600 temp file / `GIT_SSH_COMMAND -i <0600 file>` with `StrictHostKeyChecking=accept-new` and `/data/git_known_hosts`, credential helper disabled, temp dir removed; `token` auth refused for `http://` and `git://` remotes	`.git/config`, the stored URL, a command line
NetBox token, CloudVision REST token	`Authorization` header (pynetbox / httpx)	logs (redacted)
AI API key	request header only; every provider error passes through `mask()`; the base URL may not resolve to a private address unless `ai_allow_private_urls`	logs, HTTP responses, the prompt
SMTP password	aiosmtplib login, only over STARTTLS or implicit TLS (465) unless `AVDLAB_SMTP_ALLOW_INSECURE=true`	logs and error messages; a plaintext connection
lab host client SSH (ACT `apply-from-tools-server.sh`)	`StrictHostKeyChecking=accept-new` with a per-lab `act/clients/known_hosts`	`=no` with `/dev/null`

The lab's **device password** knob is a lab credential, not a platform secret: by design it is rendered into the AVD inputs (sha512 hash of the local user, type-7 BGP peer-group passwords, and `ansible_password` where the example has it), so it is part of the lab repository and of any export or git push of that lab. The AI designer never sees it.

Container and supply chain. The api image is built from digest-pinned base images (`node:22-bookworm-slim`, `python:3.11-slim`, `docker:27-cli`), installs the frontend with `npm ci` from `package-lock.json` and the backend from the hash-pinned `backend/requirements.lock` (`pip install --require-hashes --no-deps`), and runs as `avdlab` (uid/gid 10001). Compose adds `read_only: true`, `cap_drop: [ALL]`, `no-new-privileges`, `tmpfs` on `/tmp` and `/home/avdlab`, and memory / CPU / pid limits. There is no Docker socket in the api container: the `update` profile's **docker-socket-proxy** exposes a filtered Docker API (containers, images, networks, volumes, info; `BUILD` only with `AVDLAB_PROXY_ALLOW_BUILD=1`) on the compose network, and `AVDLAB_DOCKER_HOST` points the CLI at it. The **self-update** accepts `https://` manifests only, requires a digest-pinned release image (`<image>:<tag>@sha256:<64 hex>`, `AVDLAB_UPDATE_REQUIRE_DIGEST`) read from the manifest or the GitHub release notes, optionally verifies the image with keyless **cosign** (`AVDLAB_UPDATE_COSIGN_IDENTITY` regexp + `AVDLAB_UPDATE_COSIGN_ISSUER`; identity set but no cosign CLI → refused), and is admin-

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

only. Third-party compose images (Caddy, Postgres, Valkey, the proxy) are digest-pinned; the Postgres and NetBox profiles have **no default secrets** and refuse to start with an empty one.

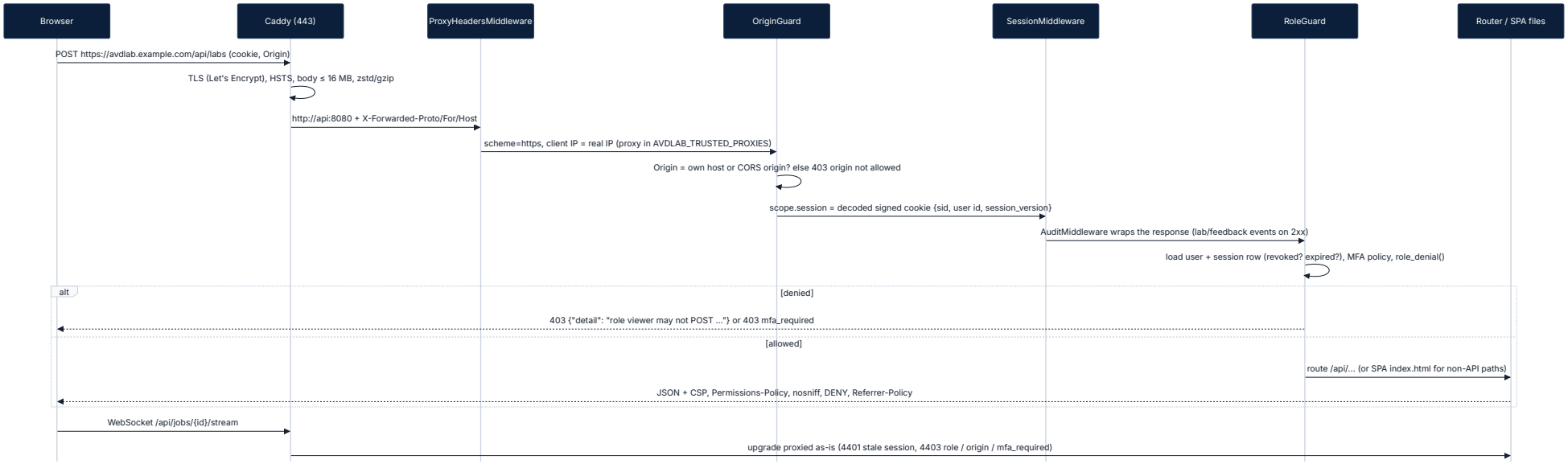
Rate limits and validation. Public account requests are limited to 5 per hour per IP; AI design calls to 30 per user per hour; sign-in attempts to 20 per minute per IP. `POST /api/register` always answers `202 {status: "pending", message: "request received"}`, also for a taken or pending username (audited as `registration.duplicate`). `jump_host` must be `[user@]host[:port]` without option-like or shell characters (`422`); git branch names must match `^[A-Za-z0-9._/-]{1,200}$` without `..`, `@{`, `//`, a leading `-` / `/` or a trailing `/`, `.`, `.lock`; image uploads are capped (`AVDLAB_UPLOAD_MAX_BYTES`, 4 GiB) and stored under a server-derived name.

SSH host keys. Trust on first use: an unknown host key (and the jump host's) is fetched once and appended to `/data/known_hosts`; a later mismatch fails the connection with a hint to remove the line if the host was reinstalled. The remote lab tree is private to the SSH user (`/opt/avdlab` 0750, `labs/<slug>` 0700) and shared with the agent image's gid (chapter 4 §4.3).

(g) Request path through Caddy

Caddy terminates TLS, caps the request body (4 GB on `/api/images/upload`, 16 MB elsewhere) and forwards to `api:8080` on the compose network with `X-Forwarded-Proto/For/Host`. With `AVDLAB_TRUST_PROXY_HEADERS=true` the app mounts uvicorn's `ProxyHeadersMiddleware` as the outermost layer, trusting only the addresses in `AVDLAB_TRUSTED_PROXIES` (loopback and RFC 1918 by default, which covers the compose network). Every inner layer then sees `https` and the real client IP (used in the audit log, the session list and the login / registration rate limits). The image's own `uvicorn --proxy-headers` default trusts only `127.0.0.1` and is not enough behind Caddy. `OriginGuard` rejects cross-site mutations, the session middleware decodes the cookie, `AuditMiddleware` records declarative events, RoleGuard checks the session row, the MFA policy and the role, and the router answers; non-API paths are served from the SPA build. WebSockets for job logs and terminals pass through Caddy unchanged. Without the `public` profile the same chain runs without Caddy and without the proxy-headers layer.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Request path through Caddy

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 3

Installation

LAN and Azure installs, upgrades, backups and every setting.

avdlab ships as one Docker image built from this repository (`deploy/Dockerfile.backend` : the SPA is built with Node 22 from `frontend/package-lock.json` with `npm ci` , the API runs on Python 3.11 with pyavd 6.4, ANTA 1.10, eos-downloader, git, openssh-client and the Docker CLI, installed from the hash-pinned `backend/requirements.lock` ; every base image is pinned by digest). The container runs as the unprivileged user `avdlab` (uid/gid 10001) with a **read-only root filesystem** — all state is in the `/data` volume. It is started with Docker Compose from the `deploy/` directory. Two supported installs:

	LAN INSTALL	PUBLIC INSTALL
Typical machine	Mac mini, any Docker host	Azure VM (Ubuntu 24.04, B2s is enough)
Compose command	`docker compose up -d --build` (add `--profile update` for the in-app update)	`docker compose --profile public --profile update up -d --build`
URL	`http://<host>:8080`	`https://<your domain>`
TLS	none (LAN only)	Caddy 2 + Let's Encrypt, automatic renewal
API port binding	`0.0.0.0:8080`	`127.0.0.1:8080` (only Caddy reaches it)

In both cases the **labs run on a separate Linux lab host** reached over SSH (chapter 4). The orchestrator itself is small; the lab host needs about 2 GB of RAM per cEOS node (the probe's rule of thumb) and enough disk for images.

3.1 Mac mini or any Docker host on a LAN

1. **Install a Docker engine.** Docker Desktop or OrbStack on macOS (turn on "start at login"), Docker Engine with the compose plugin on Linux. Keep a Mac from sleeping so jobs keep running: `sudo pmset -a sleep 0 disksleep 0` (or System Settings → Energy → *Prevent automatic sleeping*).
2. **Get the code** (git clone, or a release tarball — see 3.3) into e.g. `~/avdlab` .
3. **Configure:** `bash cd ~/avdlab/deploy cp .env.example .env # edit .env: # AVDLAB_ADMIN_PASSWORD the first admin's password (≥ 12 chars, 3 of 4 classes) # AVDLAB_SESSION_SECRET leave empty → generated once into /data/session.key; or openssl rand -hex 32 (≥ 32 chars) # AVDLAB_SECRET_KEY python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())" # (or leave empty: a key is generated into the data volume — back it up either way)`
4. **Start:** `docker compose up -d --build` , then open `http://<mac-mini>.local:8080` and sign in as the admin (`AVDLAB_ADMIN_USER` , default `admin`). If the password is the default `change-me` or fails the policy, the app still starts but asks for a new password at the first sign-in. Admins must enrol an authenticator (default `mfa_required_roles = ["admin"]`): until then the account only reaches the Account page.
5. **Optional Postgres:** set `AVDLAB_DATABASE_URL=postgresql+asyncpg://avdlab:<pw>@db:5432/avdlab` and `POSTGRES_PASSWORD=<pw>` (**required** — the `db` service has no default password since 1.1.0 and refuses to start without one), then `docker compose --profile postgres up -d` .
6. **Optional NetBox next to it:** set `NETBOX_SECRET_KEY` (`python3 -c "import secrets; print(secrets.token_urlsafe(50))"`), `NETBOX_API_TOKEN` (40 hex characters: `python3 -c "import secrets; print(secrets.token_hex(20))"`), `NETBOX_SUPERUSER_PASSWORD` , `NETBOX_DB_PASSWORD` and `NETBOX_REDIS_PASSWORD` — all **required**, the containers exit with a message when one is empty — then `docker compose --profile netbox up -d` . The first start runs migrations (a few minutes); NetBox is published on `127.0.0.1:8000` only (`NETBOX_BIND_ADDRESS` , `NETBOX_PORT` ; set `NETBOX_BIND_ADDRESS=0.0.0.0` to expose it on the LAN). In avdlab add a NetBox target with URL `http://netbox:8080` (same compose project) and that token.
7. **Optional Docker access (`update` profile):** `docker compose --profile update up -d` starts the **docker-socket-proxy** (`tecnativa/docker-socket-proxy` , socket mounted read-only, no published port) and `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` in `.env` points the API's docker CLI at it. It lets the API import/cache cEOS images locally (`docker save` for pushes) and is required for the in-app update (3.3). Source-mode updates (`docker compose up -d --build`) additionally need `AVDLAB_PROXY_ALLOW_BUILD=1` . The api container never mounts `/var/run/docker.sock` itself; the commented-out mount in `docker-compose.yml` is a legacy alternative that gives the container root-equivalent access and is not recommended.
8. **Add a lab host** (chapter 4) and an image (chapter 5, Images).

What the compose file hardens (1.1.0). The `api` service runs with `user: 10001:10001` , `read_only: true` , a `tmpfs` on `/tmp` (`AVDLAB_TMP_SIZE` , 1g) and `/home/avdlab` , `security_opt: no-new-privileges:true` , `cap_drop: [ALL]` , `mem_limit` / `memswap_limit` `AVDLAB_MEM_LIMIT` (2g), `cpus` `AVDLAB_CPUS` (2.0) and `pids_limit` `AVDLAB_PIDS_LIMIT` (512). Everything the API writes goes to `/data` (volume) or the tmpfs; large uploads stream straight to `/data/images` , so `AVDLAB_TMP_SIZE` only bounds scratch files. Caddy, Postgres, Valkey and the proxy images are pinned by digest (`netboxcommunity/netbox:${NETBOX_VERSION}` stays a tag because the version is configurable). Check a configuration with `docker compose -f deploy/docker-compose.yml --profile public --profile update --profile postgres --profile netbox config` .

Operate it with `docker compose logs -f api` (logs), `docker compose ps` (health: the image has a healthcheck on `/api/auth/me` expecting 200 or 401), `docker compose restart api` .

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

3.2 Azure VM with public HTTPS

Before you start

1. **DNS first.** Create the A record `<domain>` → the VM's public IP **before** the first start. Caddy requests the certificate immediately; repeated failures run into Let's Encrypt rate limits. For a dry run set `AVDLAB_ACME_STAGING=true` (staging CA, browsers warn); to switch to a real certificate afterwards, remove the setting, `docker compose --profile public down` and `docker volume rm avdlab_caddy_data`.
2. **Network security group.** Inbound **22** from your own IP only, **80** and **443** from anywhere (80 is needed for the ACME HTTP-01 challenge and the redirect). Do **not** open 8080.
3. **VM size.** B2s (2 vCPU / 4 GB) with Ubuntu 24.04 runs the orchestrator. The lab host is a different, bigger machine (or the same VM if it is big enough — then set `AVDLAB_REMOTE_ROOT` to e.g. `/opt/avdlab-labs` so labs do not land in the app checkout).

Setup script

```
curl -fsSLO https://raw.githubusercontent.com/<owner>/<repo>/main/deploy/azure/setup-azure-vm.sh   # or scp it
sudo bash setup-azure-vm.sh <git-url | release-tarball-url | tarball-path> avdlab.example.com you@example.com [admin-password]
```

`deploy/azure/setup-azure-vm.sh` is idempotent (safe to re-run):

STEP	WHAT IT DOES
1	installs Docker Engine + the compose plugin from the signed `download.docker.com` repository (only when missing)
2	puts the code in `/opt/avdlab` — `git clone` / `git pull`, or unpacks the tarball over it (`--strip-components=1`)
3	creates `deploy/.env` from `.env.example` (an existing `.env` is kept) and sets `AVDLAB_DOMAIN`, `AVDLAB_ACME_EMAIL`, `AVDLAB_PUBLIC_URL=https://<domain>`, `AVDLAB_TRUST_PROXY_HEADERS=true`, `AVDLAB_BIND_ADDRESS=127.0.0.1`, `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375`, `AVDLAB_PROXY_ALLOW_BUILD=1`; generates `AVDLAB_ADMIN_PASSWORD` (unless given or already non-default) and `AVDLAB_SESSION_SECRET` when they are defaults, and prints generated values once
4	`docker compose --profile public --profile update up -d --build`

The script opens no firewall ports (that is the NSG's job).

By hand

Set these keys in `deploy/.env`, then `cd /opt/avdlab/deploy && docker compose --profile public --profile update up -d --build`:

KEY	VALUE
`AVDLAB_DOMAIN`	`avdlab.example.com` (the A record)
`AVDLAB_ACME_EMAIL`	your address (Let's Encrypt account, expiry mails)
`AVDLAB_PUBLIC_URL`	`https://avdlab.example.com` — makes the session cookie `Secure` and is added to the CORS origins
`AVDLAB_TRUST_PROXY_HEADERS`	`true` — honour `X-Forwarded-Proto/For` from Caddy (real client IPs in the audit log and session list). Only proxies in `AVDLAB_TRUSTED_PROXIES` (default loopback + RFC 1918, which covers the compose network) are believed
`AVDLAB_BIND_ADDRESS`	`127.0.0.1`
`AVDLAB_ADMIN_PASSWORD`	a strong password. With `AVDLAB_PUBLIC_URL` set, a default or weak value makes avdlab generate a random 20-character bootstrap password, logged **once** in `docker compose logs api`, with a forced change at the first sign-in. Admins must enrol MFA
`AVDLAB_SESSION_SECRET`	empty (generated into `/data/session.key`) or `openssl rand -hex 32`; the old default `change-me-session-secret` is refused
`AVDLAB_SECURE_COOKIES`	optional; empty = follow `AVDLAB_PUBLIC_URL`, `true` / `false` forces it
`AVDLAB_DOCKER_HOST`	`tcp://docker-socket-proxy:2375` with `--profile update` (in-app update)

Check it

- `docker compose --profile public logs caddy` → look for `certificate obtained successfully`.
- `curl -I https://avdlab.example.com` → `HTTP/2 200` with `strict-transport-security`, `content-security-policy` and `permissions-policy` headers.
- Caddy rejects request bodies above 16 MB (`413`), except on `/api/images/upload` where the limit is 4 GB — the same as the API's `AVDLAB_UPLOAD_MAX_BYTES`; raise both together if you need more.
- Renewal is automatic (about 30 days before expiry); certificates and the ACME account live in `caddy_data`.
- With a `Secure` cookie, sign-in works only over https: through `ssh -L 8080:127.0.0.1:8080 <vm>` you see the login page but do not stay signed in.

Reaching the lab host from Azure

The orchestrator only needs **SSH** to the lab host (job logs and web terminals are proxied over that connection):

- a VPN or Tailscale between the VM and the lab host (best when the lab host is at home or on a LAN), or
- SSH to the lab host's public IP with key authentication only (Hosts → *Generate SSH key*), with the lab host's firewall/NSG allowing 22 only from the VM's IP.

The request path through Caddy and the middlewares is drawn in chapter 2 §(g).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

3.3 Upgrading

Three ways, all keep the `avdlab-data` volume (and therefore labs, users, secrets):

Release tarball

```
tar -xzf ~/Downloads/avdlab-vX.Y.Z.tar.gz -C ~        # writes into ~/avdlab
cd ~/avdlab/deploy && docker compose up -d --build
```

Extract in a terminal so existing files are overwritten. Double-clicking the archive in Finder unpacks it *next to* the existing folder and renames conflicting files to `name 2.ts`, `name 3.ts`, which breaks the TypeScript build; clean up with `find ~/avdlab -name "* [0-9].*" -not -path "*/node_modules/*" -delete`.

git

```
cd ~/avdlab && git pull && cd deploy && docker compose up -d --build
```

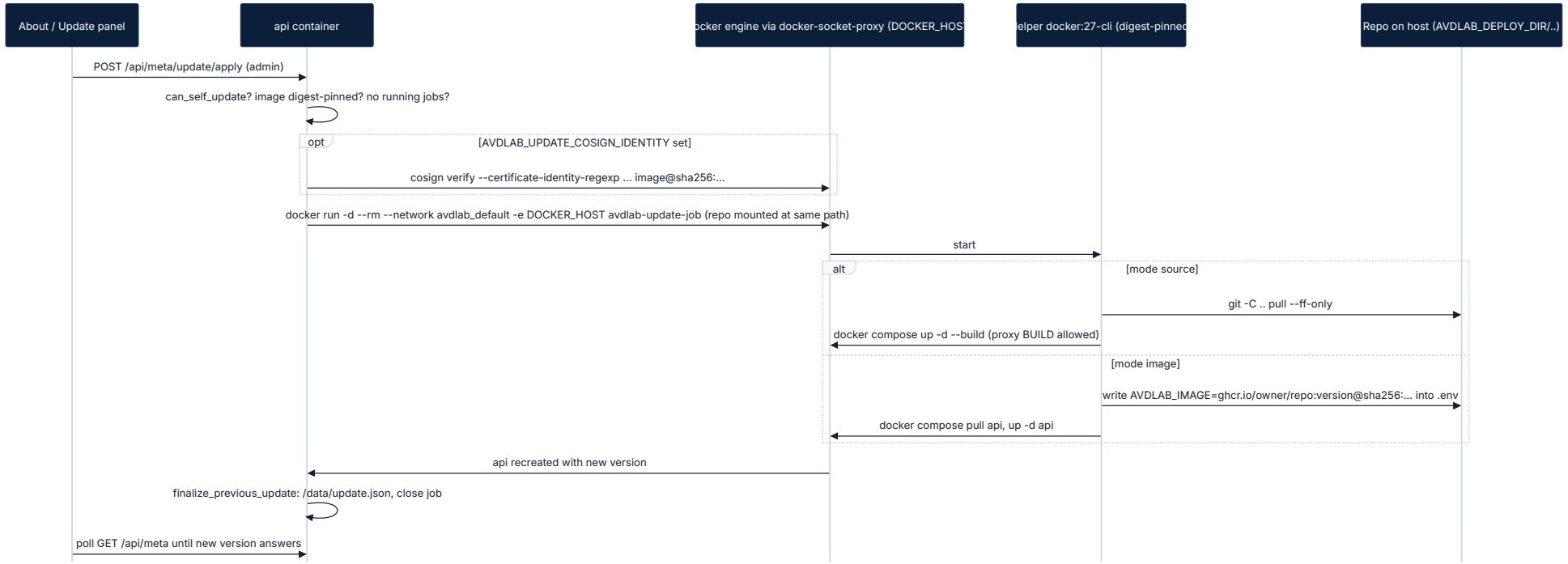
To bake the build info shown on About / `GET /api/meta/build` (authenticated since 1.1.0; `GET /api/meta` no longer exposes it): `AVDLAB_GIT_SHA=$(git rev-parse --short=12 HEAD) AVDLAB_BUILD_DATE=$(date -u +%FT%TZ) docker compose build`. The build needs `backend/requirements.lock` and `frontend/package-lock.json` in the tree (both are committed; chapter 9 §9.3 explains how to refresh the lockfile).

In-app update

The sidebar version badge shows an *update available* pill when a newer release exists; About and Changelog have an **Updates** panel with *Check now* and *Update now*.

REQUIREMENT	SETTING / ACTION
where to look	`AVDLAB_UPDATE_SOURCE=github:<owner>/<repo>` (latest GitHub release) or an `https://` manifest URL returning `{"latest", "url", "notes", "published_at", "image", "digest"?}` (`http://` is refused); empty = no checks
how often	`AVDLAB_UPDATE_CHECK_INTERVAL` hours (24); first check a few seconds after start, never blocking it; *Check now* forces one (admins)
how to update	`AVDLAB_UPDATE_MODE` : `source` (default; the checkout must be a git clone: `git pull --ff-only` + `docker compose up -d --build`), `image` (writes `AVDLAB_IMAGE=ghcr.io/<owner>/<repo>:<version>@sha256:…` into `.env` , then `docker compose pull api && docker compose up -d api`), `manual` (notify only)
Docker access	the `update` profile running and `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` (source mode also needs `AVDLAB_PROXY_ALLOW_BUILD=1` for `docker compose … --build`) — or, not recommended, the legacy `/var/run/docker.sock` mount
digest-pinned image (image mode)	`AVDLAB_UPDATE_REQUIRE_DIGEST=true` (default): the release image must be `<image>:<tag>@sha256:<64 hex>` ; the digest comes from the manifest (`image` with `@sha256:…` or a separate `digest` field) or from the GitHub release notes (`<image>@sha256:…` , else the first `sha256:…`). An unpinned image shows as `image_error` in the Update panel, `can_self_update` is false and *Update now* answers `409`
signature (optional)	`AVDLAB_UPDATE_COSIGN_IDENTITY` (regexp for `cosign verify --certificate-identity-regexp` , e.g. `^https://github.com/<owner>/<repo>/.github/workflows/release.yml@refs/tags/v`) + `AVDLAB_UPDATE_COSIGN_ISSUER` (GitHub Actions by default): the image is verified before `.env` is touched; identity set but no `cosign` CLI in the api image → refused (fail closed). Add the CLI with `COPY --from=ghcr.io/sigstore/cosign/cosign:v2.4.1 /ko-app/cosign /usr/local/bin/cosign` in `Dockerfile.backend`
who	`POST /api/meta/update/check` and `/apply` are **admin-only** (operators got `403`)
where the repo is	`AVDLAB_DEPLOY_DIR` = absolute **host** path of `deploy/` ; compose passes `${PWD}` , so run compose from `deploy/` (or set it explicitly)
no running jobs	the update is refused with 409 while any job runs or is queued

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Self-update sequence

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

The job `app.update` starts a **detached** helper container (`AVDLAB_UPDATE_HELPER_IMAGE`, `docker:27-cli` pinned by digest) with the repository mounted at the same path as on the host, so it survives the API being recreated. With `AVDLAB_DOCKER_HOST` set the helper joins the compose network `AVDLAB_UPDATE_HELPER_NETWORK` (`avdlab_default`) and talks to the same `DOCKER_HOST` instead of mounting the socket. The API follows its output into `/data/update.log`; the next API process reads `/data/update.json`, records `last_update: {from, to, at, ok}` and closes the job. The UI polls `/api/meta` until the new version answers; if it does not, check `/data/update.log`.

Database and settings on upgrade

New tables are created and new columns added automatically at startup (1.1.0: the `sessions` table; `secrets.created_by`, `secrets.purpose`, `users.totp_last_step`, `audit_log.actor`). Settings stored on **Admin settings** (database) win over `.env`, so an upgrade never overwrites them. When the `users` table is empty (first start, or an upgrade from a pre-0.8.0 single-admin install), the admin is created from `AVDLAB_ADMIN_USER` / `AVDLAB_ADMIN_PASSWORD`; afterwards those variables are no longer used.

Upgrading from 1.0.x to 1.1.0

Read this list before you pull. Back up the data volume first (3.4).

#	WHAT CHANGES	WHAT TO DO
1	**Everyone signs in again once.** Pre-1.1.0 cookies carry no session id and are invalid.	tell your users; nothing else
2	`AVDLAB_SESSION_SECRET` **default refused.** If `.env` still has `change-me-session-secret` (or a value shorter than 32 characters) the api does not start: `RuntimeError: AVDLAB_SESSION_SECRET is the old default …`.	set `openssl rand -hex 32`, or empty the line and let `/data/session.key` be generated (keep `/data` on a volume; with several api replicas set the value explicitly so they share one key)
3	**Admins must enrol MFA.** `mfa_required_roles` defaults to `["admin"]`; an admin without an authenticator sees only the Account page until MFA is set up.	enrol on the Account page after the upgrade, or set `AVDLAB_MFA_REQUIRED_ROLES=[]` (or `PUT /api/admin/settings {"mfa_required_roles": []}` *before* upgrading) to opt out
4	**Existing secrets are legacy rows** (`created_by` / `purpose` NULL): visible to admins only, usable in any binding by an admin. Operators see only secrets they create from now on.	nothing is re-checked retroactively; operators re-create (or an admin re-binds) the secrets they need
5	**Login errors are uniform** (`401` also for locked accounts); `POST /api/register` answers `202` for every accepted request.	update scripts that keyed on `423` / `409`; admins still see lockouts on the Users page
6	`/api/meta` **lost its build fields**; `GET /api/meta/build` (authenticated) has them.	update monitoring that read `git_sha` from `/api/meta`
7	**Behind a proxy:** `AVDLAB_TRUST_PROXY_HEADERS=true` now trusts `AVDLAB_TRUSTED_PROXIES` only (loopback + RFC 1918).	add a proxy on a public address to the list, otherwise audit and session IPs show the proxy
8	**CSP** is sent by the API (`AVDLAB_CSP`); the default covers the SPA (`style-src 'unsafe-inline'` for Tailwind, `media-src https:` for a hosted intro video, `connect-src ws: wss:` for terminals).	a deployment with its own header policy sets `AVDLAB_CSP` or empties it to let Caddy own the header
9	**AI base URLs** pointing at a LAN / loopback model server (Ollama, LM Studio, vLLM) are refused (`422`).	`PUT /api/admin/settings {"ai_allow_private_urls": true}` or `AVDLAB_AI_ALLOW_PRIVATE_URLS=true` (chapter 6)
10	**Audit retention**: rows older than a year are pruned from now on.	`AVDLAB_AUDIT_RETENTION_DAYS=0` to keep everything, or export with `GET /api/audit/export` first
11	**Key rotation is available** (`AVDLAB_SECRET_KEYS`).	optional; procedure in chapter 8 §8.6
12	`/data` **ownership**: the api runs as uid/gid 10001.	on an existing volume run once `docker compose run --rm --user 0 api chown -R 10001:10001 /data` (or `docker run --rm -v avdlab_avdlab-data:/data alpine chown -R 10001:10001 /data`); fresh volumes are pre-chowned by the image
13	**Docker access** moved to the `update` profile. If you used *Update now* or local cEOS image import:	`docker compose --profile update up -d` with `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` in `.env` (`AVDLAB_PROXY_ALLOW_BUILD=1` for source-mode updates). The old socket mount still works if you uncomment it, but it gives the api container root-equivalent access
14	**Image-mode updates need a digest-pinned image.** An unpinned release shows `image_error` in the Update panel and cannot be applied.	publish releases with `image: ghcr.io/<owner>/<repo>:<ver>@sha256:…` (or a `digest` field) in the manifest, or `<image>@sha256:…` in the GitHub release notes (`release.yml` does this); for signed releases set `AVDLAB_UPDATE_COSIGN_IDENTITY` and add the cosign CLI to the api image
15	**Secrets without defaults**: `POSTGRES_PASSWORD` (postgres profile) and `NETBOX_SECRET_KEY`, `NETBOX_API_TOKEN`, `NETBOX_SUPERUSER_PASSWORD`, `NETBOX_DB_PASSWORD`, `NETBOX_REDIS_PASSWORD` (netbox profile) must be in `.env`; NetBox listens on 127.0.0.1 only.	deployments that relied on the old defaults (`avdlab`, `netbox`, `admin`) copy exactly those values into `.env` to keep their databases readable; `NETBOX_BIND_ADDRESS=0.0.0.0` to expose NetBox
16	**ACT password migration**: the `act.password` knob of existing labs moves from plain text into the encrypted `secret_knobs` at the first start (`migrate_act_passwords`), and is returned masked (`********`) from then on.	nothing; sending the mask back keeps the stored value
17	**Lab hosts**: bootstrap installs the pinned containerlab `AVDLAB_CLAB_VERSION` (0.79.0; the check step accepts only that version) and makes `/opt/avdlab` 0750. Lab trees become 0700 at the next deploy/sync and are shared with the agent image's gid (`[perms] …` in the job log).	re-run **Bootstrap** on every lab host (chapter 4)
18	**Git remotes** on local paths (`/srv/…`, `file://`) stop working; branch names are validated on push/pull/clone; `token` auth is refused for `http://` / `git://`.	`AVDLAB_GIT_ALLOW_LOCAL=true` if you really need local remotes; use `https://`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

#	WHAT CHANGES	WHAT TO DO
19	**SMTP** AUTH over plain port 25/587 without STARTTLS fails with a clear message.	enable STARTTLS (or 465) or set `AVDLAB_SMTP_ALLOW_INSECURE=true`
20	**Caddy** caps bodies (4 GB upload, 16 MB elsewhere).	raise `request_body max_size` and `AVDLAB_UPLOAD_MAX_BYTES` together if needed
21	**Job logs**: directories older than 90 days are removed (rows stay); more than 4 concurrent jobs queue.	`AVDLAB_JOB_LOG_RETENTION_DAYS=0` / `AVDLAB_JOB_MAX_CONCURRENT` to change
22	**Image build** needs `backend/requirements.lock` and `frontend/package-lock.json`.	both are in the repository; a fork that edits `pyproject.toml` refreshes the lock (chapter 9 §9.3)

3.4 Backups

All app state is in the `avdlab-data` volume (mounted at `/data`):

PATH IN THE VOLUME	CONTENT	WHY IT MATTERS
`avdlab.db`	the SQLite database (unless Postgres)	users, hosts, labs, jobs, targets, settings, secrets (encrypted)
`secret.key`	the Fernet key (unless `AVDLAB_SECRET_KEY` / `AVDLAB_SECRET_KEYS` is set)	without it every stored secret **and every user's MFA** is lost
`session.key`	the cookie-signing secret (unless `AVDLAB_SESSION_SECRET` is set)	losing it only signs everyone out once
`known_hosts`, `git_known_hosts`	pinned SSH host keys of lab hosts, jump hosts and git remotes	losing it only means TOFU again; keeping it protects against a swapped host
`labs/<slug>/`	every lab as a git repository, including build outputs, ANTA runs, snapshots	the labs themselves
`jobs/<id>/log.txt`	job logs and artefacts (directories pruned after `AVDLAB_JOB_LOG_RETENTION_DAYS`)	history, troubleshooting
`images/`	downloaded cEOS tarballs	can be re-downloaded
`feedback/`, `update.json`, `update.log`, `update-check.json`	feedback copies, update bookkeeping	

```
cd ~/avdlab/deploy        # or /opt/avdlab/deploy
docker run --rm -v avdlab_avdlab-data:/data -v $PWD:/b alpine tar czf /b/avdlab-data-$(date +%F).tgz -C /data .
```

Keep `deploy/.env` with the archive (it holds `AVDLAB_SESSION_SECRET` and possibly `AVDLAB_SECRET_KEY` / `AVDLAB_SECRET_KEYS`; the audit trail can additionally be exported as NDJSON from *Admin › Audit log*). With Postgres, back up `avdlab-pg` as well (`pg_dump`). On Azure copy the archive off the VM (Blob Storage, or an Azure Backup policy on the disk). To restore, stop the stack, extract the archive into an empty `avdlab-data` volume, restore `.env`, start.

Lost the only admin password? Stop the app, empty the users table (`sqlite3 /data/avdlab.db "DELETE FROM users"` — this removes all accounts) and start again: the admin is re-created from the environment variables (behind a public URL with a weak `AVDLAB_ADMIN_PASSWORD` a random bootstrap password is logged once).

3.5 Settings reference

Every field of `backend/avdlab/core/config.py` (and the auth settings in `core/passwords.py`) can be set as `AVDLAB_<NAME>` in `deploy/.env`. Values marked **(admin setting)** are only defaults: the value saved on *Admin settings* (or with `PUT /api/admin/settings`) in the database wins. Rows marked **1.1.0** are new or changed in this release.

Storage, paths, database

VARIABLE	DEFAULT	MEANING
`AVDLAB_DATA_DIR`	`/data`	root for `labs/`, `images/`, `jobs/`, `known_hosts`, `git_known_hosts`, `secret.key`, `session.key` (compose forces `/data`; the only writable path besides the tmpfs)
`AVDLAB_TEMPLATES_DIR`	`<repo>/templates` (image: `/app/templates`, set in `.env.example`)	lab templates
`AVDLAB_DATABASE_URL`	`sqlite+aiosqlite:////data/avdlab.db`	or `postgresql+asyncpg://user:pw@db:5432/avdlab`
`AVDLAB_META_DIR`	repo root	directory holding `VERSION`, `CHANGELOG.md`, `LICENSE`, `THIRD_PARTY_NOTICES.md`

Authentication and sessions

VARIABLE	DEFAULT	MEANING
`AVDLAB_ADMIN_USER`	`admin`	first admin, created only while the users table is empty

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

VARIABLE	DEFAULT	MEANING
`AVDLAB_ADMIN_PASSWORD`	`change-me`	its password; default or weak → forced change at first sign-in; with `AVDLAB_PUBLIC_URL` set, a random 20-character bootstrap password is generated and logged once (**1.1.0**)
`AVDLAB_SESSION_SECRET`	empty (**1.1.0**, was `change-me-session-secret`)	cookie signing key. Empty → generated once into `<data_dir>/session.key` (0600) and reused. An explicit value must be ≥ 32 characters and not the old default (RuntimeError at startup otherwise)
`AVDLAB_SESSION_MAX_AGE`	`604800` (7 days)	sliding cookie lifetime in seconds
`AVDLAB_SESSION_ABSOLUTE_MAX_AGE`	`43200` (12 h)	**1.1.0** absolute lifetime of a server-side session in seconds; re-authentication afterwards regardless of activity
`AVDLAB_AUTH_RATE_LIMIT`	`20`	**1.1.0** login / MFA-verify attempts per minute per client IP (token bucket, burst = rate) → `429` + `Retry-After`; `0` disables
`AVDLAB_MFA_REQUIRED_ROLES`	`["admin"]`	**1.1.0** JSON list of roles that must have an authenticator; a user without one gets a restricted session (admin setting `mfa_required_roles`, API only)
`AVDLAB_SECRET_KEY`	empty	Fernet key of the secret store; empty = generated into `<data_dir>/secret.key`. With `AVDLAB_SECRET_KEYS` also set it is appended as a decrypt-only key
`AVDLAB_SECRET_KEYS`	empty	**1.1.0** comma-separated Fernet keys: the first encrypts, all decrypt (key rotation, chapter 8 §8.6); validated at startup
`AVDLAB_AUDIT_RETENTION_DAYS`	`365`	**1.1.0** audit rows older than this are pruned at startup and daily; `0` = keep forever
`AVDLAB_PASSWORD_MIN_LENGTH`	`12`	minimum password length (never below 8)
`AVDLAB_LOGIN_MAX_FAILURES`	`5`	consecutive wrong passwords before lockout
`AVDLAB_LOGIN_LOCKOUT_MINUTES`	`15`	lockout duration (the sign-in form does not reveal a lockout)
`AVDLAB_MFA_ISSUER`	`avdlab`	issuer label in authenticator apps

Public deployment (compose profile `public`)

VARIABLE	DEFAULT	MEANING
`AVDLAB_PORT`	`8080`	published host port of the API (compose only)
`AVDLAB_BIND_ADDRESS`	`0.0.0.0`	host address the API port binds to; `127.0.0.1` with the public profile (compose only)
`AVDLAB_DOMAIN`	—	DNS name for Caddy (compose/Caddy only)
`AVDLAB_ACME_EMAIL`	—	ACME account e-mail (compose/Caddy only)
`AVDLAB_ACME_STAGING`	`false`	`true` = Let's Encrypt staging CA (compose/Caddy only)
`AVDLAB_PUBLIC_URL`	empty	the URL users open; `https://…` → Secure cookie; appended to CORS origins; used in mails and as SMTP from-domain
`AVDLAB_SECURE_COOKIES`	empty	force the cookie's Secure flag (`true` / `false`); empty = derived from `AVDLAB_PUBLIC_URL`
`AVDLAB_TRUST_PROXY_HEADERS`	`false`	trust `X-Forwarded-Proto/For` from the proxies in `AVDLAB_TRUSTED_PROXIES` (only with the API not publicly reachable)
`AVDLAB_TRUSTED_PROXIES`	`127.0.0.1,::1,172.16.0.0/12,10.0.0.0/8,192.168.0.0/16`	**1.1.0** IPs/CIDRs whose `X-Forwarded-*` headers are honoured when `AVDLAB_TRUST_PROXY_HEADERS=true` (never `*`)
`AVDLAB_CORS_ORIGINS`	`["http://localhost:5173","http://localhost:8080"]`	JSON list; only needed for the Vite dev server; also the allowed `Origin` values of the Origin guard (plus the request host and `AVDLAB_PUBLIC_URL`)
`AVDLAB_CSP`	`default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; font-src 'self' data:; connect-src 'self' ws: wss:; media-src 'self' https:; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none'`	**1.1.0** `Content-Security-Policy` on every response; `""` disables the header
`AVDLAB_PERMISSIONS_POLICY`	`camera=(), microphone=(), geolocation=(), payment=(), usb=(), interest-cohort=()`	**1.1.0** `Permissions-Policy` on every response; `""` disables
`AVDLAB_MEM_LIMIT`, `AVDLAB_CPUS`, `AVDLAB_PIDS_LIMIT`, `AVDLAB_TMP_SIZE`	`2g`, `2.0`, `512`, `1g`	**1.1.0** resource limits and `/tmp` tmpfs size of the api container (compose only)

Lab hosts and images

VARIABLE	DEFAULT	MEANING
`AVDLAB_AGENT_IMAGE`	`ghcr.io/aristanetworks/avd/universal:python3.11-avd-v6.4.0`	container run on the lab host for ANTA, cv_deploy, config pushes, ZTP node; bootstrap pulls it

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

VARIABLE	DEFAULT	MEANING
`AVDLAB_CLAB_VERSION`	`0.79.0`	**1.1.0** containerlab release installed natively by the `clab-host` recipe (`x.y.z` ; the `.deb` / `.rpm` is checksum-verified against the GitHub release)
`AVDLAB_CLAB_IMAGE`	`ghcr.io/srl-labs/clab:0.79.0` (**1.1.0**, was `:latest`)	containerlab image for bootstrap's container mode; keep in step with `AVDLAB_CLAB_VERSION`
`AVDLAB_ANTA_IMAGE`	`ghcr.io/aristanetworks/anta:1.10.0`	image for the PSIRT scan (`anta psirt`), pulled on first use
`AVDLAB_HOST_IMAGE`	`ghcr.io/srl-labs/network-multitool:latest`	linux client containers (knob `client_image` wins per lab)
`AVDLAB_REMOTE_ROOT`	`/opt/avdlab`	root directory on the lab host (0750); labs go to `<root>/labs/<slug>` (0700, shared with the agent gid)
`AVDLAB_UPLOAD_MAX_BYTES`	`4294967296` (4 GiB)	**1.1.0** cap of `POST /api/images/upload` on `Content-Length` and the streamed bytes → `413` ; `0` disables (Caddy has its own 4 GB cap)
`AVDLAB_AVD_VERSION`	`6.4.0`	AVD version for the native toolchain install and display
`AVDLAB_ANTA_VERSION`	`1.10.0`	ANTA version for the native toolchain install and display
`AVDLAB_MONITOR_ENABLED`	`true`	host metrics sampler; `false` = no background SSH at all
`AVDLAB_MONITOR_INTERVAL`	`20.0`	sampling interval in seconds

Jobs

VARIABLE	DEFAULT	MEANING
`AVDLAB_JOB_MAX_CONCURRENT`	`4`	**1.1.0** jobs running at once; further jobs wait as `queued` (FIFO, cancellable); `0` = unlimited
`AVDLAB_JOB_LOG_MAX_BYTES`	`20971520` (20 MiB)	**1.1.0** per-job log cap; the log ends with `[log truncated]` , the job continues; `0` = unlimited
`AVDLAB_JOB_LOG_RETENTION_DAYS`	`90`	**1.1.0** `<data>/jobs/<id>` directories older than this are deleted at startup and daily (DB rows stay); `0` = keep
`AVDLAB_GIT_ALLOW_LOCAL`	`false`	**1.1.0** allow `file://` URLs and local paths as lab git remotes (the test suite sets it)

About, feedback, updates

VARIABLE	DEFAULT	MEANING
`AVDLAB_COPYRIGHT`	`© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks`	About page / meta
`AVDLAB_GIT_SHA` , `AVDLAB_BUILD_DATE`	empty	baked into the image at build time (build args); shown on About via the authenticated `GET /api/meta/build`
`AVDLAB_FEEDBACK_GITHUB_REPO`	empty	`owner/repo` ; with a secret named `github_token` , every feedback entry becomes a GitHub issue (label `feedback`)
`AVDLAB_UPDATE_SOURCE`	empty	`github:<owner>/<repo>` or an `https://` manifest URL (`http://` refused since 1.1.0); empty = no update checks
`AVDLAB_UPDATE_MODE`	`source`	`source` , `image` or `manual`
`AVDLAB_UPDATE_CHECK_INTERVAL`	`24.0`	hours; `0` = no background check
`AVDLAB_DEPLOY_DIR`	compose: `${PWD}`	host path of `deploy/` (self-update)
`AVDLAB_UPDATE_DOCKER_SOCKET`	`/var/run/docker.sock`	socket that must be mounted for self-update **when** `AVDLAB_DOCKER_HOST` **is empty** (legacy, not recommended)
`AVDLAB_DOCKER_HOST`	empty	**1.1.0** `DOCKER_HOST` for every docker CLI call of the api, e.g. `tcp://docker-socket-proxy:2375` (compose profile `update`); replaces the socket-mount check
`AVDLAB_UPDATE_HELPER_NETWORK`	`avdlab_default`	**1.1.0** compose network the detached update helper joins when `AVDLAB_DOCKER_HOST` is used
`AVDLAB_PROXY_ALLOW_BUILD`	`0`	**1.1.0** `1` lets the docker-socket-proxy pass `BUILD` / `SESSION` calls (needed for source-mode updates: `docker compose up -d --build`) (compose only)
`AVDLAB_UPDATE_REQUIRE_DIGEST`	`true`	**1.1.0** image-mode self-update needs `<image>:<tag>@sha256:<64 hex>` ; otherwise `image_error` and no update

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

VARIABLE	DEFAULT	MEANING
`AVDLAB_UPDATE_COSIGN_IDENTITY`	empty	**1.1.0** regexp for `cosign verify --certificate-identity-regexp`; empty = no signature check; set but no cosign CLI → update refused
`AVDLAB_UPDATE_COSIGN_ISSUER`	`https://token.actions.githubusercontent.com`	**1.1.0** `--certificate-oidc-issuer` for cosign
`AVDLAB_UPDATE_HELPER_IMAGE`	`docker:27-cli@sha256:851f91d241214e7c6db86513b270d58776379aacc5eb9c4a87e5b47115e3065c` (**1.1.0**, digest-pinned)	detached helper running git/compose
`AVDLAB_IMAGE`	`avdlab/api:latest`	image name of the api service (compose; rewritten by image-mode updates with `<image>:<version>@sha256:…`)

Admin settings defaults (UI wins)

VARIABLE	DEFAULT	MEANING
`AVDLAB_APP_TITLE`	`avdlab`	name in the UI and e-mails (admin setting)
`AVDLAB_GITHUB_URL`	empty	GitHub button in top bar, dashboard, register page (admin setting)
`AVDLAB_SUPPORT_EMAIL`	empty	contact on register and About pages (admin setting)
`AVDLAB_REGISTRATION_ENABLED`	`true`	public *Request an account* form (admin setting)
`AVDLAB_INTRO_VIDEO_URL`	`/intro/avdlab-intro.mp4`	dashboard intro video; empty hides it (admin setting)
`AVDLAB_FEEDBACK_MAIL_TO`	`dietrich@arista.com`	feedback and test-mail recipient(s), comma-separated (admin setting)
`AVDLAB_FEEDBACK_MAIL_ENABLED`	`true`	mail every new feedback entry (admin setting)
`AVDLAB_REGISTRATION_MAIL_TO`	empty (= feedback address)	account-request recipient(s) (admin setting)
`AVDLAB_SMTP_HOST`	empty	SMTP server; empty = no e-mail (admin setting)
`AVDLAB_SMTP_PORT`	`587`	`465` = implicit TLS (admin setting)
`AVDLAB_SMTP_USER`	empty	login; empty = no login (admin setting)
`AVDLAB_SMTP_PASSWORD`	empty	fallback; the encrypted secret `smtp_password` set in the UI wins
`AVDLAB_SMTP_STARTTLS`	`true`	STARTTLS (ignored on 465) (admin setting)
`AVDLAB_SMTP_ALLOW_INSECURE`	`false`	**1.1.0** allow SMTP AUTH over a plaintext connection (no STARTTLS and not port 465); refused with a clear error otherwise
`AVDLAB_SMTP_FROM`	empty (= `avdlab@<host of AVDLAB_PUBLIC_URL or localhost>`)	sender (admin setting)

AI designer defaults (UI wins)

VARIABLE	DEFAULT	MEANING
`AVDLAB_AI_DEFAULT_PROVIDER`	empty	`anthropic` or `openai` (OpenAI-compatible); empty = no shared model (admin setting)
`AVDLAB_AI_DEFAULT_BASE_URL`	empty	empty = the provider's public API (admin setting)
`AVDLAB_AI_DEFAULT_MODEL`	empty	model name (admin setting)
`AVDLAB_AI_DEFAULT_API_KEY`	empty	fallback; the encrypted secret `ai_default_api_key` set in the UI wins
`AVDLAB_AI_SHARED_ALLOWED`	`false`	users without their own link may use the shared model (admin setting)
`AVDLAB_AI_RATE_LIMIT`	`30`	design calls per user per hour
`AVDLAB_AI_TIMEOUT`	`180.0`	seconds per provider request
`AVDLAB_AI_ALLOW_PRIVATE_URLS`	`false`	**1.1.0** allow AI base URLs that resolve to loopback, link-local, RFC 1918, CGNAT or metadata addresses (Ollama on the LAN, …); admin setting `ai_allow_private_urls` (API only), the DB row wins

Other compose variables (not read by the app)

VARIABLE	DEFAULT	MEANING
`POSTGRES_PASSWORD`	— (**required**, 1.1.0; was `avdlab`)	password of the `db` service (profile `postgres`); the image refuses to start with an empty one
`NETBOX_VERSION`	`v4.2`	NetBox image tag (profile `netbox`; the only compose image not pinned by digest)
`NETBOX_PORT`	`8000`	published NetBox port
`NETBOX_BIND_ADDRESS`	`127.0.0.1`	**1.1.0** address the NetBox UI is published on; `0.0.0.0` to expose it
`NETBOX_SECRET_KEY`	— (required)	Django secret key of NetBox
`NETBOX_API_TOKEN`	— (required)	40-hex superuser API token

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

VARIABLE	DEFAULT	MEANING
`NETBOX_SUPERUSER_NAME` / `_EMAIL` / `_PASSWORD`	`admin` / `admin@example.com` / — (**required**, 1.1.0)	NetBox superuser
`NETBOX_DB_PASSWORD`, `NETBOX_REDIS_PASSWORD`	— (**required**, 1.1.0; were `netbox`)	NetBox Postgres and Valkey passwords; `netbox` / `netbox-worker` exit with a message and the Valkey services refuse an empty `--requirepass`

Compose volumes: `avdlab-data`, `avdlab-pg`, `caddy_data`, `caddy_config`, `netbox-media`, `netbox-pg`, `netbox-redis`. The project name is `avdlab`, so the data volume is `avdlab_avdlab-data` on the host.

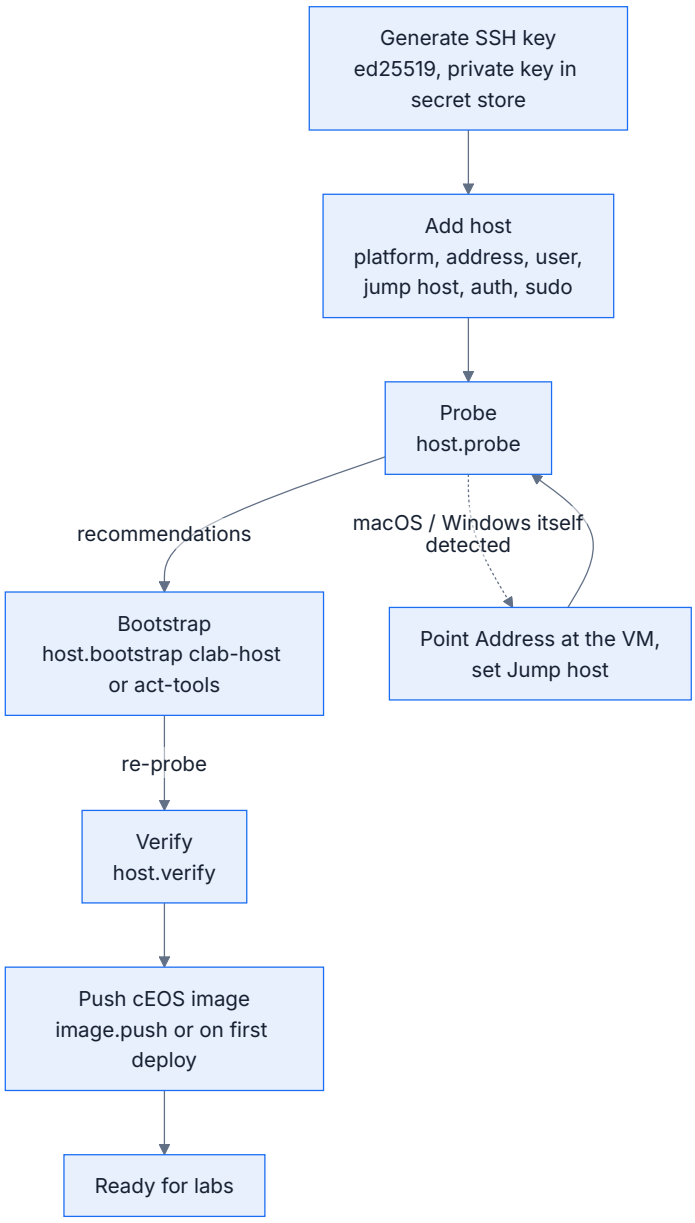
© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 4

Lab hosts

Requirements, bootstrap, Linux / Mac / Windows hosts, jump hosts, probe output, images and architecture.

A **lab host** is the machine that runs containerlab and the cEOS-lab containers. avdlab reaches it only over **SSH**: every command, file copy, container action, web terminal and monitoring sample goes through an SSH connection made with `asyncssh`. Nothing is installed on the lab host except what the bootstrap recipe installs (Docker, containerlab, sysctls, a working directory and container images).



Host setup flow

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

4.1 Requirements

ITEM	REQUIREMENT	CHECKED BY
OS	Debian/Ubuntu family or RHEL family (RHEL, Fedora, CentOS, Rocky, AlmaLinux, Oracle Linux) — detected from `/etc/os-release` `ID` / `ID_LIKE`	probe, bootstrap
CPU architecture	x86_64 for cEOS / cEOS64 images; arm64 (aarch64) for cEOSarm	probe; deploy refuses a mismatch
SSH	sshd reachable from the orchestrator (directly or via a jump host); key or password auth	every job
root	passwordless sudo (`NOPASSWD`) **or** a sudo password stored on the host	probe (`sudo -n true` , then a real `sudo` with the stored password)
RAM	rule of thumb 2 GB per cEOS node (probe prints "roughly N cEOS nodes") — an 8-switch fabric wants 16 GB	probe recommendation
Disk	≥ 20 GB free for Docker (`/var/lib/docker` or `/`); cEOS images are ~2 GB each	probe recommendation
Network	outbound HTTPS for `download.docker.com` , `github.com` (containerlab release), `ghcr.io` during bootstrap; to CloudVision (443) if labs stream to CVaaS	—

The kind of a host is `linux` (a containerlab host, the default) or `act` (the tools server of an Arista Cloud Test lab; bootstrapped with recipe `act-tools`).

4.2 Adding a Linux host

1. **Hosts → Generate SSH key.** Creates an ed25519 key pair; the private key goes into the secret store (kind `ssh_key`), the public key is shown once with *Copy public key* and *Copy install command* (`echo '<key>' >> ~/.ssh/authorized_keys`). Add it to the SSH user on the lab host.
2. **Give the user root.** Either passwordless sudo: `bash echo "$USER ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/avdlab` or enter a **sudo password** in the host form (stored encrypted, sent with `sudo -S` on stdin). With SSH password authentication the SSH password is reused for sudo unless a separate sudo password is set.
3. **Hosts → Add host.** Platform *Linux server or VM*, name, kind *containerlab host*, address, SSH user (default `ubuntu`) and port, authentication (*SSH key* → pick the key secret; or *Password* → existing secret or a new password), optional sudo password.
4. **Probe** (row action). See 4.6.
5. **Bootstrap** with recipe `clab-host` (4.3). The job re-probes at the end.
6. **Verify** (4.4).

The first connection pins the host key (trust on first use) in `/data/known_hosts` . If the host is reinstalled later, the connection fails with *host key mismatch* — remove its line from `known_hosts` (in the data volume) and connect again.

SSH connects with a 30 s timeout and retries transient TCP/handshake failures 3 times with a 3 s backoff; authentication failures are not retried (retrying a bad credential only locks accounts).

4.3 Bootstrap recipe `clab-host`

Bootstrap runs a **recipe**: an ordered list of steps, each with a *check* and an *apply* command. For every step the engine runs *check* first (passes → `[skip]`), otherwise *apply* then *check* again (`[ok]` , or `[FAIL]` which aborts the recipe). Steps that do not apply to the chosen options are skipped without running. Re-running is always safe.

#	STEP	CHECK	APPLY	WHEN
1	base packages	`dpkg -s` / `rpm -q` of curl, ca-certificates, rsync, git, python3, jq	`apt-get install` / `dnf install`	always
2	docker engine	`docker info`	**1.1.0:** Docker Engine + compose plugin from the **signed** `download.docker.com` **repository** — Debian/Ubuntu: keyring `/etc/apt/keyrings/docker.asc` + `/etc/apt/sources.list.d/docker.list` ; RHEL family: `dnf config-manager --add-repo …/docker-ce.repo` — then `systemctl enable --now docker` (replaces `curl https://get.docker.com | sh` ; mirrors `deploy/azure/setup-azure-vm.sh`)	always
3	docker group membership	user in group `docker`	`usermod -aG docker <user>`	always
4	containerlab (native)	`containerlab version` reports exactly `AVDLAB_CLAB_VERSION` (0.79.0)	**1.1.0:** download `containerlab_<ver>_linux_<amd64|arm64>.{deb,rpm}` and `checksums.txt` from `https://github.com/srl-labs/containerlab/releases/download/v<ver>/` , `sha256sum -c` the package, then `apt-get install ./pkg` / `dnf install ./pkg` (replaces `curl -sL https://containerlab.dev/setup | bash`); an older install is upgraded, a newer one replaced	mode *native* (default)
5	containerlab image (container mode)	`docker image inspect ghcr.io/srl-labs/clab:0.79.0` (`AVDLAB_CLAB_IMAGE`)	`docker pull`	mode *container*

#	STEP	CHECK	APPLY	WHEN
6	containerlab wrapper (container mode)	`/usr/local/bin/containerlab` is the avdlab wrapper	write a wrapper that runs `docker run --rm --privileged --network host --pid host` with the Docker socket, netns, `/etc/hosts` and the current directory mounted (unchanged in 1.1.0)	mode *container*
7	sysctls	each value ≥ target and `/etc/sysctl.d/90-avdlab.conf` exists	write the file, `sysctl --system`	always
8	remote root	`/opt/avdlab` exists, is owned by the SSH user **and is mode 0750** (1.1.0)	`mkdir -p`, `chown`, `chmod 0750`	always
9	agent image	`docker image inspect <AVDLAB_AGENT_IMAGE>`	`docker pull` (30 min timeout)	always
10	pipx	`command -v pipx`	apt/dnf `pipx` (or pip)	*native toolchain* ticked
11	pyavd (native toolchain)	`pipx list` has `pyavd 6.4.0`	`pipx install "pyavd[ansible]==6.4.0"`	*native toolchain* ticked
12	anta (native toolchain)	`pipx list` has `anta 1.10.0`	`pipx install "anta[cli]==1.10.0"`	*native toolchain* ticked
13	arista.avd collection (native toolchain)	`ansible-galaxy collection list arista.avd` = 6.4.0	`ansible-galaxy collection install arista.avd:==6.4.0`	*native toolchain* ticked

Sysctl targets (also what the probe compares against):

SYSCTL	TARGET	WHY
`fs.inotify.max_user_instances`	64000	many cEOS nodes; the default is too low for > 10 nodes
`fs.inotify.max_user_watches`	1048576	same
`net.ipv4.ip_forward`	1	routing between container networks
`vm.max_map_count`	262144	
`fs.file-max`	2097152	

Steps 1–9 run with sudo; the native toolchain steps run as the SSH user (`~/.local/bin`). The native toolchain is only for people who want to use pyavd/ANTA in a shell on the host; avdlab itself always uses the agent container.

Bootstrap options in the dialog: **containerlab mode** (*native install* or *container wrapper*) and **Also install the native toolchain**. After a fresh `usermod -aG docker`, the group only applies to new logins; avdlab detects whether the user can run Docker and uses sudo for Docker commands when it cannot.

Recipe `act-tools` (for hosts of kind `act`, the ACT tools server) is the subset: base packages, docker engine, docker group membership, remote root, agent image — no containerlab, no sysctls.

After an upgrade to 1.1.0 re-run Bootstrap on every lab host: step 4 replaces whatever containerlab version was installed with the pinned release (the check step accepts only that version) and step 8 tightens `/opt/avdlab` to 0750. `AVDLAB_CLAB_VERSION` must be `x.y.z`; keep `AVDLAB_CLAB_IMAGE` on the same version.

Permissions of the remote lab tree (1.1.0)

Before 1.1.0 the synced lab directory was world-readable and the ANTA report directories were made `a+rwX` so the agent container could write into them. Since 1.1.0:

PATH ON THE LAB HOST	MODE	WHO
`/opt/avdlab` (`AVDLAB_REMOTE_ROOT`)	0750	the SSH user (bootstrap step 8)
`/opt/avdlab/labs`, `/opt/avdlab/labs/<slug>`	0700 at creation (`umask 077`, `chmod 0700`, `tar --no-overwrite-dir` on sync)	the SSH user
`/opt/avdlab/labs/<slug>` after `prepare_agent_dir`	group = the **agent image's gid**, 0750; `anta/` and `anta/psirt/` `g+rwX`	the SSH user + containers started from the agent image

`executors.common.prepare_agent_dir` discovers the gid with `docker run --rm --entrypoint id <agent image> -g` and applies `chgrp -R <gid>` / `chmod 0750` / `g+rwX` from a throw-away root container; the job log shows `[perms] lab dir shared with the agent image (gid N, 0750; reports g+rwX)`. If the gid cannot be determined (custom agent image without `id`), it falls back to world-writable **report directories only** and logs a warning. Existing trees keep their old permissions until the next deploy or sync recreates them.

The ACT `apply-from-tools-server.sh` script uses `ssh -o StrictHostKeyChecking=accept-new` with a per-lab `act/clients/known_hosts` instead of `StrictHostKeyChecking=no` + `/dev/null`, so client host keys are pinned on first contact.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

4.4 Verify

`host.verify` runs `containerlab version` (not on `act` hosts) and starts the agent image once (`python3 -c 'import pyavd, anta; print(...)'`), then re-probes. The job result lists each check with its output tail and the remaining recommendations.

4.5 Mac and Windows lab hosts BETA

containerlab needs a Linux kernel (netlink, veth, network namespaces) and does not run on macOS or Windows natively. On those machines the lab host is a **Linux VM on that machine**, and avdlab treats it exactly like a Linux host. Because those VMs sit behind NAT on the Mac or PC, the host can have a **jump host**.

> **Beta — not yet field-tested.** Probe output and the Feedback button help to fix what breaks.

When you add a host, the dialog asks for the **platform** first (three cards; the choice is stored on the host and the probe double-checks it). The setup steps it shows are:

Mac (OrbStack VM)

Apple Silicon or Intel; the lab runs in an OrbStack Linux VM.

1. On the Mac: install OrbStack (orbstack.dev) and open it once.
2. Terminal: `orb create ubuntu avdlab-vm`
3. Terminal: `orb -m avdlab-vm sudo apt-get install -y openssh-server && orb -m avdlab-vm sudo systemctl enable --now ssh`
4. Give the VM memory in OrbStack → Settings (16 GB+ for an 8-switch fabric).
5. Below: Address `avdlab-vm.orb.local`, SSH user = your Mac user name (OrbStack creates it in the VM with your SSH key), Jump host = `you@<the Mac>`.
6. Save → Probe → Bootstrap. On Apple Silicon use cEOSarm images (New lab switches to arm64 automatically).

Windows (WSL2)

The lab runs in the Containerlab WSL2 distribution.

1. PowerShell: `wsl --install`, then install the Containerlab distribution from containerlab.dev/windows (`wsl --install --from-file clab.wsl`); WSL ≥ 2.4.4.
2. In the distro: `sudo apt-get install -y openssh-server && sudo systemctl enable --now ssh`. Keep Docker Desktop's WSL integration OFF for this distro.
3. Make it reachable: `%UserProfile%\.wslconfig` → `[wsl2] networkingMode=mirrored, memory=20GB`; Windows Firewall rule for TCP 22 — or run OpenSSH on Windows and use it as the Jump host with Address `localhost`.
4. Below: Address = the PC (mirrored) or `localhost` (jump), SSH user = your WSL user, Jump host = `you@<the PC>` if used.
5. Save → Probe → Bootstrap.

The jump host field

FIELD	FORMAT	NOTES
Jump host	`[user@]host[:port]`	user defaults to the host's SSH user, port to 22
Address	the VM **as seen from the jump host**	`avdlab-vm.orb.local` on a Mac; `localhost` (with the distro's sshd port in *Port*) on Windows
Credentials	the same key / password is used on both hops	put the public key on the Mac/PC **and** in the VM

avdlab connects to the jump host first, then opens the connection to the VM through it; host keys of both hops are pinned. All traffic — commands, file sync, terminals, monitoring — uses that tunnelled connection. The field is shown only for the Mac and Windows platforms (choosing *Linux* clears it).

What the probe says on these platforms

DETECTED	HOW	PROBE / BOOTSTRAP BEHAVIOUR
OrbStack VM on macOS	`/etc/orbstack*` or `/opt/orbstack-guest` present	violet *OrbStack VM on macOS · beta* badge; recommendation to give the VM 16 GB+ and to use cEOSarm on Apple Silicon
WSL2 on Windows	`microsoft` / `WSL` in the kernel or `/proc/version`, `/run/WSL`, `WSLInterop`	*WSL2 on Windows · beta* badge; recommendations for Docker inside the distro, sshd + mirrored networking, `.wslconfig` memory; an extra warning when `docker context` points at Docker Desktop
the Mac itself (Darwin)	`uname -s` = Darwin	*macOS (not a lab host)*; bootstrap stops with the OrbStack instructions (address = the VM, jump host = the Mac)
Windows itself (OpenSSH for Windows)	`%OS%` = Windows_NT	*Windows (not a lab host)*; bootstrap stops with the WSL instructions

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

DETECTED	HOW	PROBE / BOOTSTRAP BEHAVIOUR
platform declared ≠ detected	declared *Mac* but plain Linux, declared *Linux* but WSL/OrbStack, …	a recommendation explains the mismatch (e.g. "this address is the Mac itself, not the Linux VM on it")

Picking an arm64 host in **New lab** switches the `ceos_arch` knob to `arm64` (and clears the chosen image so you pick a cEOSarm one).

Not implemented yet (roadmap in `docs/multi-os-hosts.md`): WSL-specific bootstrap steps, and a `local` host kind that runs containerlab through the orchestrator's own Docker socket without SSH.

4.6 Probe output explained

`host.probe` runs a fixed set of independent shell commands (a failing command is tolerated: its raw output is kept and the parsed value is empty), stores the parsed **capabilities** document on the host, and turns it into **recommendations**. The Hosts list shows OS, Docker, containerlab, the number of recommendations and the probe age; clicking a host name opens the **Capabilities** panel:

PANEL SECTION	FIELD	SOURCE COMMAND
System	OS, Family, Kernel, Arch	`/etc/os-release`, `uname -r`, `uname -m`
	CPUs, Memory	`nproc`, `MemTotal` from `/proc/meminfo`
	Disk / free, Disk docker free	`df -Pk /`, `df -Pk /var/lib/docker`
	python3, rsync, pipx	`python3 --version`, `command -v`
	sudo -n	`sudo -n true` (passwordless sudo)
	`/opt/avdlab`	exists or missing (bootstrap also wants mode 0750)
Toolchain	Docker server / client	`docker version`
	user can run docker, in docker group	`docker info` as the user, `id -nG`
	containerlab	`containerlab version`
	cEOS images	`docker images` filtered on `ceos` / `arista`
sysctl	the five values of 4.3	`sysctl -n`
Recommendations	everything below	`recommendations()`

Recommendations you may see, and what to do:

RECOMMENDATION	MEANING / ACTION
`sudo needs a password and none is configured …`	add a sudo password to the host or grant NOPASSWD; bootstrap fails otherwise
`docker is not installed — run bootstrap`	run Bootstrap
`user cannot run docker — bootstrap adds the user to the docker group (re-login required)`	Bootstrap; Docker commands use sudo until then
`containerlab is not installed — run bootstrap`	run Bootstrap
`<sysctl>=<v> is too low for >10 cEOS nodes (want …)` / `below the recommended …` / `unknown`	Bootstrap writes `/etc/sysctl.d/90-avdlab.conf`
`<n> GB RAM: roughly <n> cEOS nodes at 2 GB each`	sizing information, always shown
`only <n> GB free for docker — … want ≥ 20 GB`	free disk space
`no cEOS image present — push one from the Images page`	push an image, or let the first deploy import it
`/opt/avdlab missing — bootstrap creates it`	run Bootstrap (it also sets 0750)
`rsync missing (file sync falls back to a tar pipe)`	informational
`arch aarch64: use cEOSarm images … set the lab knob ceos_arch to arm64`	arm64 host
`unsupported OS '<id>' — bootstrap supports Debian/Ubuntu and RHEL-family only`	use a supported distribution
platform hints (4.5)	Mac/Windows/WSL/OrbStack specifics

The probe also records whether sudo works with the stored password (`sudo_ok`, `sudo_with_password`).

4.7 Host monitor

Each host has a **Monitor** page (`/hosts/<id>`): CPU, load, memory and disk charts, the top containers, a process list, the container list with *Stop* / *Kill* (a container belonging to a deployed lab requires typing a confirmation), and a host shell terminal (operators and admins). The sampler polls every 20 s each host that has a deployed lab or was viewed in the last 10 minutes, over one SSH session per host; samples are kept 2 hours in memory and 7 days in `host_metrics`. The dashboard's *Lab hosts* panel shows CPU, memory and disk bars per host from the same samples (refreshed every 30 s). `AVDLAB_MONITOR_ENABLED=false` disables all background SSH.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

4.8 Images and architecture

cEOS-lab is distributed in three formats; avdlab maps each to an architecture and a Docker tag:

FORMAT	LABEL	ARCHITECTURE	DOCKER TAG
`cEOS`	cEOS-lab (x86_64)	x86_64	`arista/ceos:<version>`
`cEOS64`	cEOS64-lab (x86_64, 64-bit)	x86_64	`arista/ceos:<version>`
`cEOSarm`	cEOSarm-lab (arm64)	arm64	`arista/ceos:<version>-arm64`

- The lab knob `ceos_arch` (`x86_64` | `arm64`) decides which tag the containerlab topology uses; `ceos_version` the version.
- The image picker in New lab and lab Settings lists downloaded images (format, version, architecture, size, which hosts have it), sets both knobs, warns when the image's architecture differs from the lab host's, and shows whether the image is already on the host. *Other…* allows a free-text version.
- **Deploy refuses an architecture mismatch** (`host is x86_64, image is arm64 …: set the lab knob ceos_arch to the host's architecture and rebuild`). x86 cEOS under Rosetta is not supported.
- Getting the image onto the host (`image.push` , or automatically during deploy):

METHOD	WHEN	HOW
import	the image store has the tarball	SFTP upload to `/opt/avdlab/images/` , `docker import <file> <tag>`
remote download	push option *download on the host with ardl*	`ardl get eos --format <fmt> --version <v> --import-docker` on the host, token on stdin (masked in the log)
save/load	the image only exists in the orchestrator's local Docker (`update` profile with `AVDLAB_DOCKER_HOST`)	`docker save` → SFTP → `docker load`

Deploy imports a missing image automatically when the tarball is in the store; otherwise it fails with `image <tag> not on host <name> — push it from Images` .

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 5

User guide

Every page and every lab tab, control by control, with screenshots.

This chapter walks through the UI page by page and, for a lab, tab by tab. Roles matter: **viewers** see everything but every button that changes something is refused by the server (403); **operators** can do everything on this page except user administration, admin settings, the audit log, the in-app update and deleting hosts, and they only see the secrets they created themselves; **admins** can do everything. An admin without an authenticator (1.1.0: MFA is required for the admin role by default) is sent to the Account page until it is set up.

5.0 The typical workflow

#	WHERE	ACTION	RESULT
1	Hosts	*Generate SSH key*, put the public key on the Linux box, *Add host*, *Probe*, *Bootstrap* (`clab-host`), *Verify*	host with Docker + containerlab, recommendations cleared
2	Images	*arista.com token…* → paste token; pick a version in the catalogue → *Download*; optionally *Push* to the host	`arista/ceos:<version>` in the image store
3	Labs → New lab	template `single-dc-l3ls`, keep or change the knobs, name it, pick the host and the image, *Create lab*	lab in state `draft`
4	Lab header	*Build*	configs, docs, ANTA catalogs, topology; state `built`
5	Lab header	*Deploy*	cEOS containers running, eAPI up; state `deployed`
6	Lab header	*Validate*	ANTA report; state `validated` when nothing failed
7	Topology / Devices / Clients	inspect, open terminals, ping, add test hosts	
8	Settings → change a knob → *Save* → *Build* → Runtime *Push config changes*	only changed switches get the new config, no restart	
9	CloudVision page + lab CloudVision tab (optional)	add a target, set the `cloudvision` knob, rebuild, *Onboard*, *Deploy to CloudVision*	devices streaming, workspace submitted
10	Lab header	*Destroy* (and *Delete* when no longer needed)	containers removed; state `destroyed`

Every long action opens the **job drawer** with the live log; the drawer can be reopened from any job link.

5.1 Shell: navigation, top bar, footer

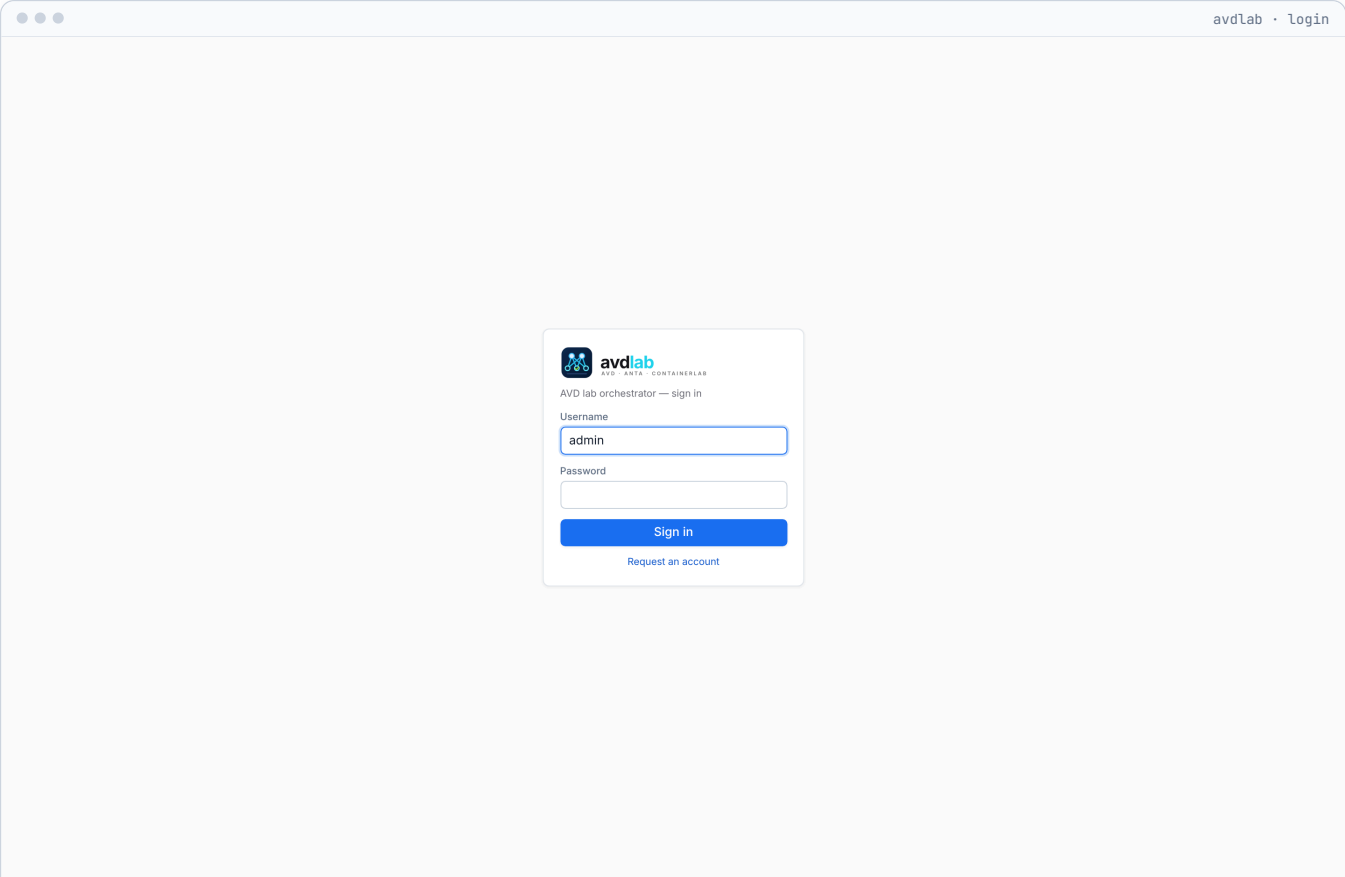
The dark sidebar groups the pages (from `Layout.tsx`):

GROUP	ITEMS (B = HIDDEN IN BEGINNER MODE, A = ADMINS ONLY)
Overview	Dashboard
Provisioning	Hosts, Images, Labs, CloudVision, NetBox (B), Devcontainer (B)
Learn	Getting started, Learn AVD
Admin	Users (A), Admin settings (A), Audit log (A, 1.1.0), Jobs (B), Feedback, Changelog, About

Under the logo: the **version badge** (a dot when the changelog has news, an *update available* pill when a newer release is known). Bottom of the sidebar: the **learning mode** switch. Top bar: breadcrumbs, **GitHub** button (when `github_url` is set), **Beginner/Advanced** toggle, **user menu** (Account, Admin settings for admins, Sign out). Footer: version, Changelog, About, Feedback. A floating **Feedback** button sits bottom-right on every page. Every page and every lab tab is wrapped in an error boundary: a rendering error stays local with *Try again* / *Reload*.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.2 Sign-in, account requests

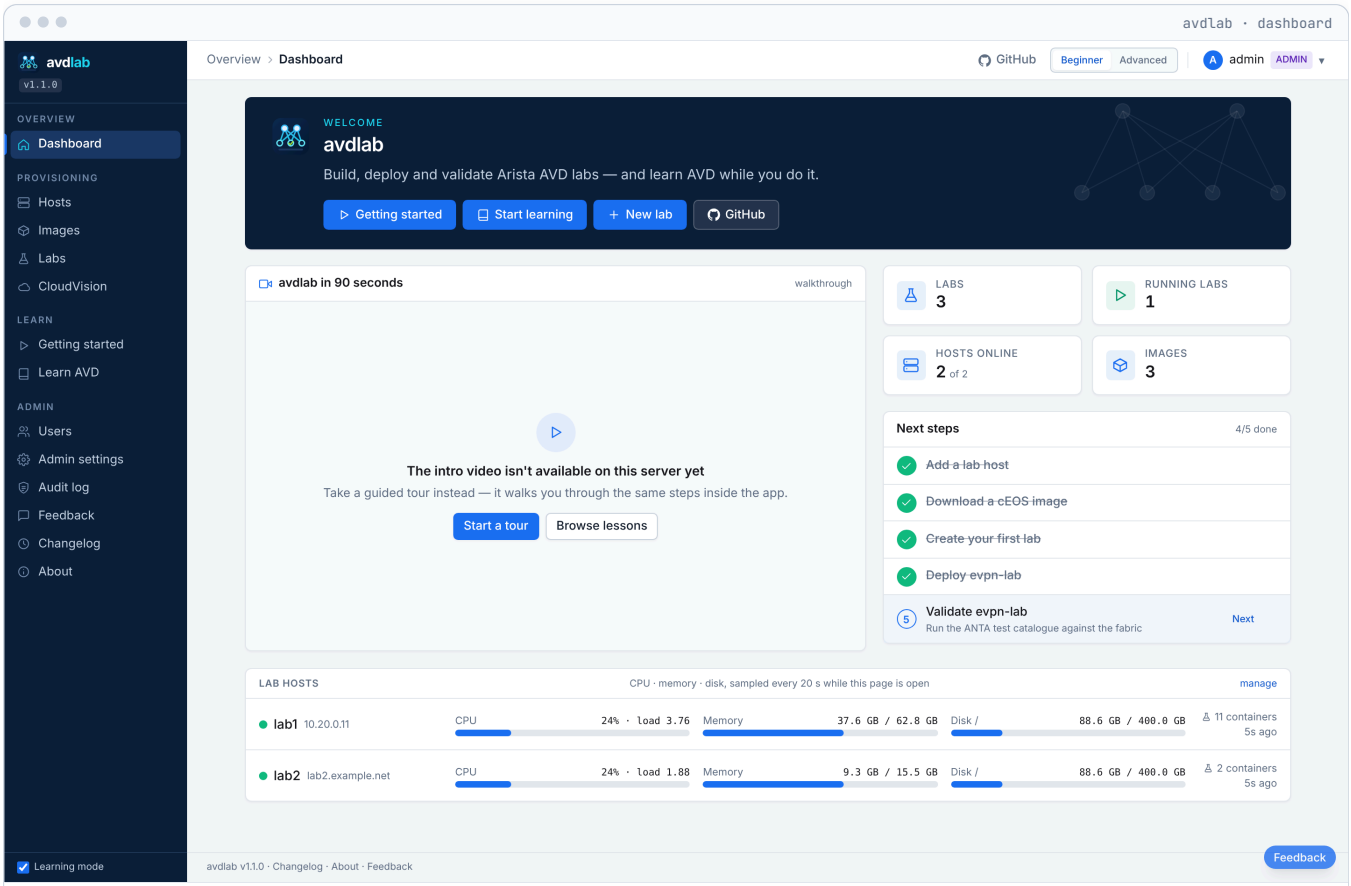


The sign-in page. Account requests go to the admin; MFA is asked for after the password.

- **Sign in** with username and password; with MFA enabled a second step asks for the 6-digit code or a recovery code (`xxxxx-xxxxx`); a code is accepted once (the same code again is refused). A user flagged *must change password* (first sign-in with a temporary password, or a weak env password for the bootstrap admin) must choose a new one before anything else. Every failure — unknown user, wrong password, locked account — shows the same *invalid username or password* message (1.1.0); a lockout is only visible to admins on the Users page. More than 20 attempts per minute from one address are answered *too many attempts: try again later* (429). Each sign-in creates a **session** that expires after 12 hours regardless of activity (Account page lists them). If your role requires two-factor authentication and you have none, you land on the Account page with a banner and only the Account page works until an authenticator is set up.
- **Request an account** (link on the sign-in page, when registration is enabled): username (3–32 characters: lowercase letters, digits, `.`, `_`, `-`), e-mail, full name, reason. The answer is always **"request received"** (1.1.0) — also when the username is taken or already pending, so the form cannot be used to guess account names (admins see such attempts as `registration.duplicate` in the audit log). The page then shows the status of the request (pending → approved / rejected); only `{id, username, status}` is kept in the browser, and the record is removed once decided. Limited to 5 requests per hour per IP. Disabled registration shows a contact address instead.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.3 Dashboard

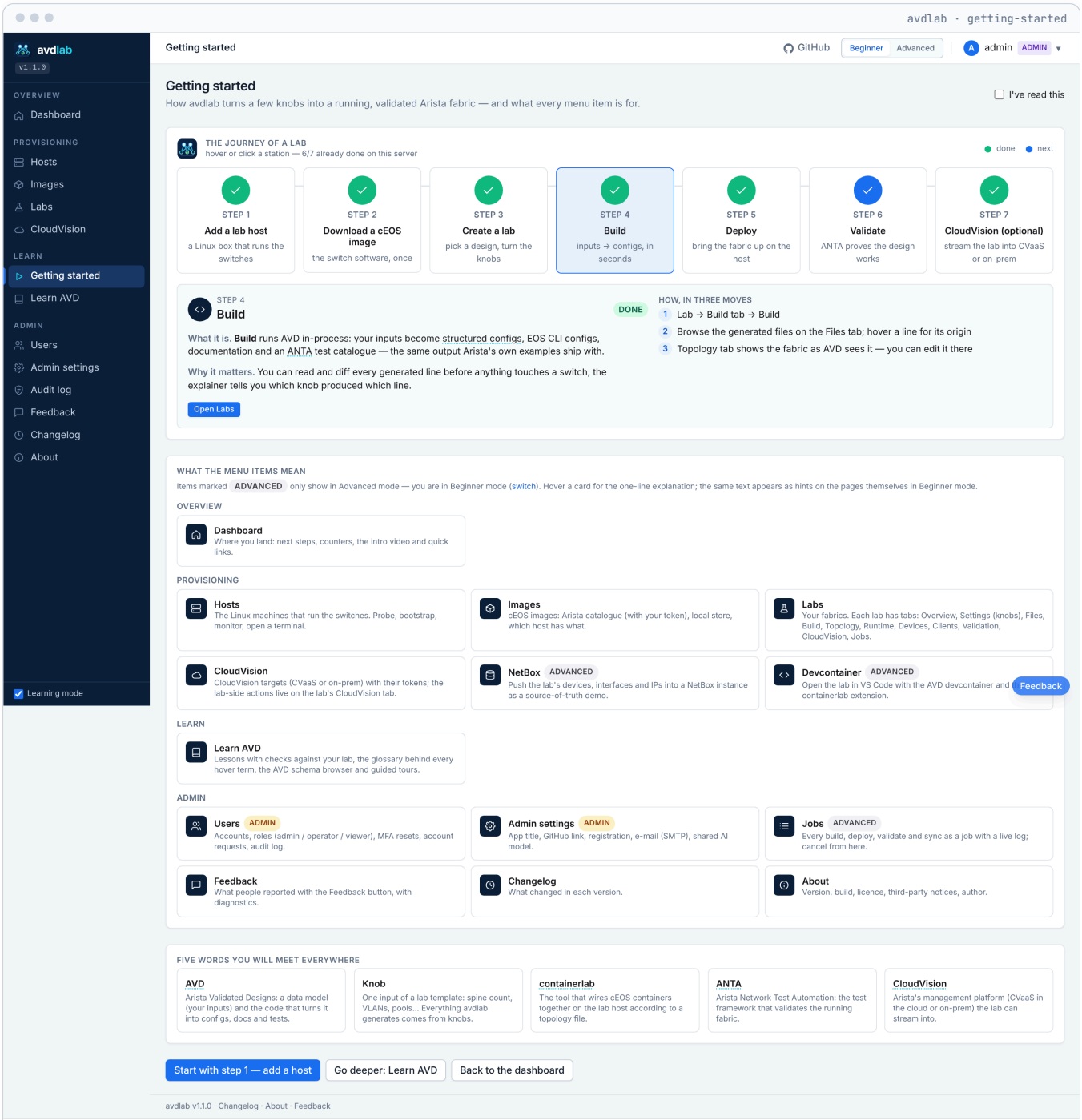


Dashboard in learning mode: the getting-started card, host health bars and the lab list.

Welcome banner with *Start learning*, *New lab* and *GitHub*; the **intro video** (90-second captioned walkthrough; falls back to a guided tour when the file is missing; hidden when `intro_video_url` is empty); tiles for Labs, Running labs, Hosts online, Images; **Next steps** — a checklist that follows what exists (add a lab host → download a cEOS image → create your first lab → deploy it → validate it with ANTA), each linking to the right page; the **Lab hosts** panel with CPU, memory and disk bars and the container count per host (refreshed every 30 s). Advanced mode adds tables of hosts, labs and recent jobs.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.4 Getting started

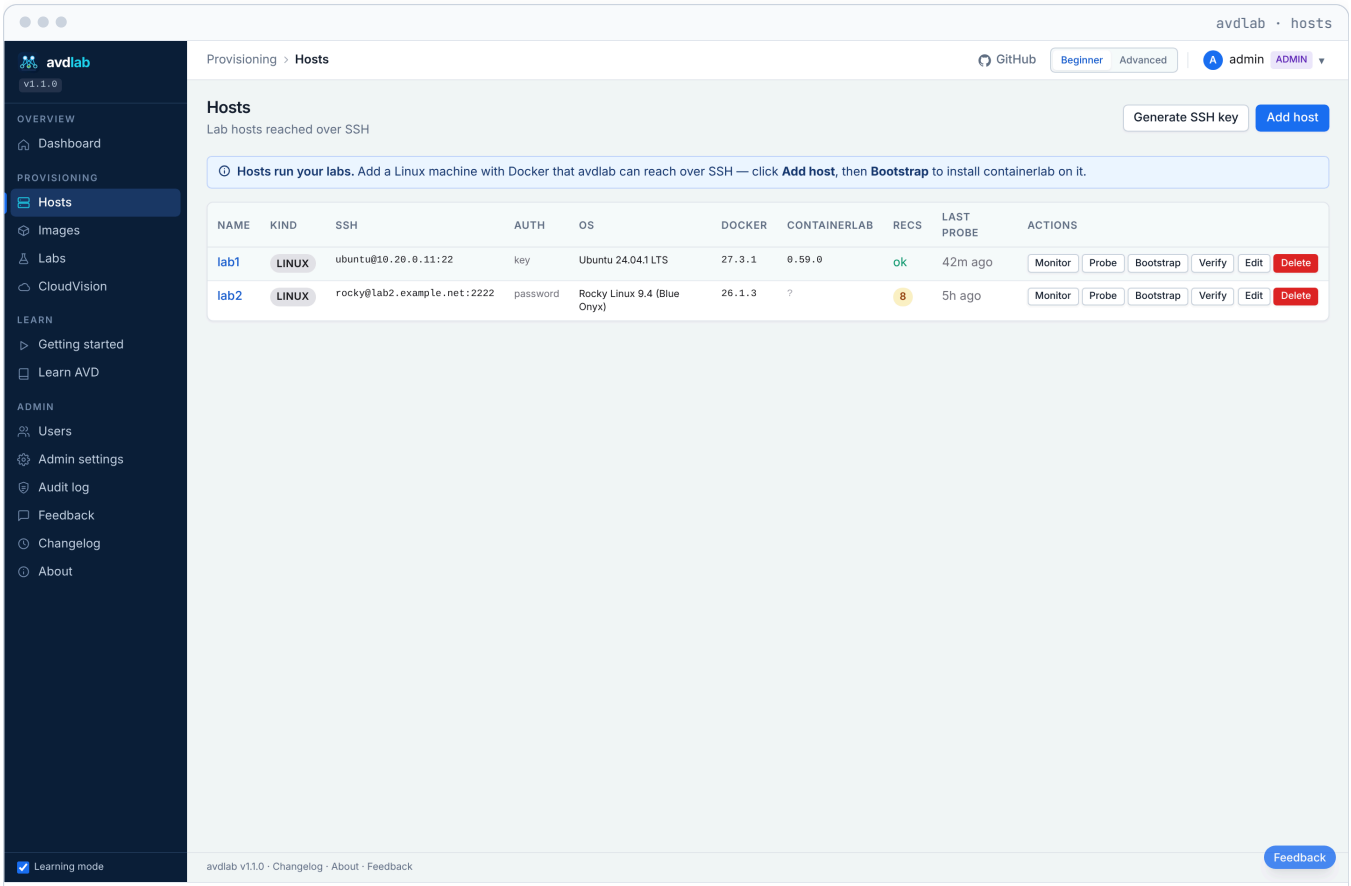


The Getting started journey: seven stations, each one links to the page where it happens.

`/getting-started` presents the seven stations of a lab — host, image, lab, build, deploy, validate, CloudVision — as a journey: for each, *what it is*, *why it matters* and *how, in three moves*, a done-state taken from this server, and a *Guided tour* button. Below: a card per menu item (with *advanced* and *admin* badges) and "five words you will meet everywhere". Step 1 carries the beta note for Mac/Windows lab hosts.

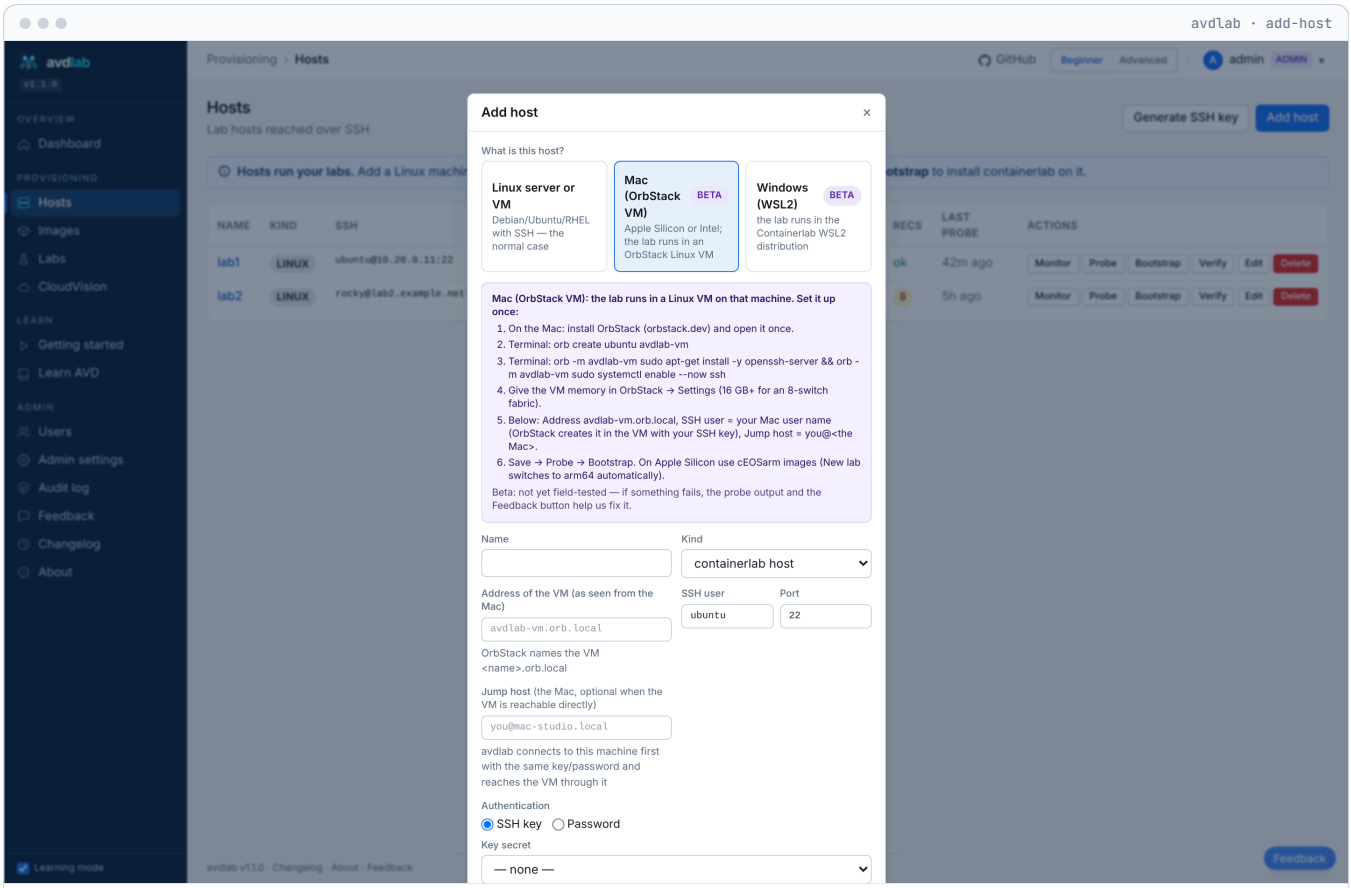
© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.5 Hosts

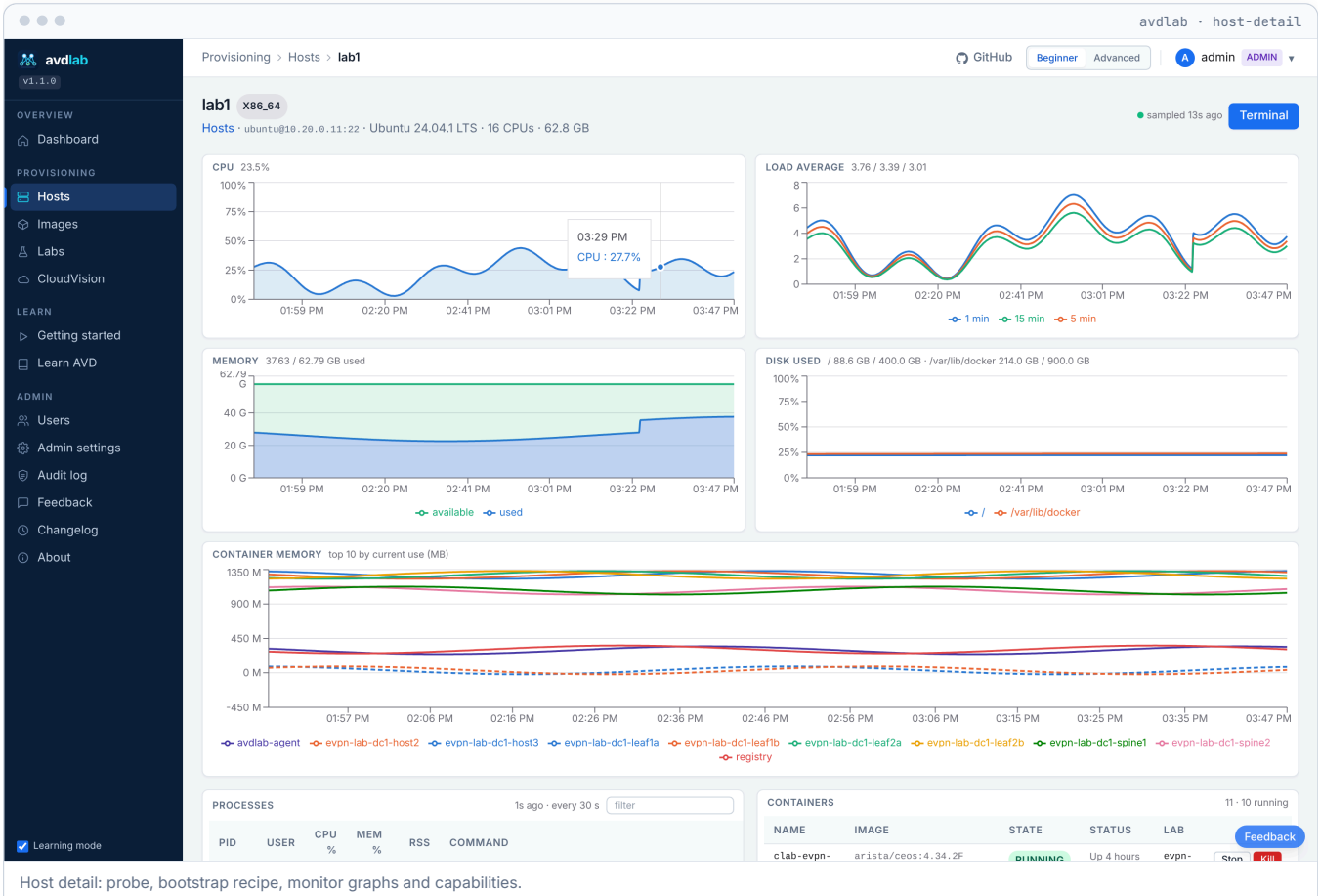


Hosts page with the SSH key panel and one lab host.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



Add host with the platform picker set to Mac (OrbStack VM, beta): the steps and the jump host field appear inline.



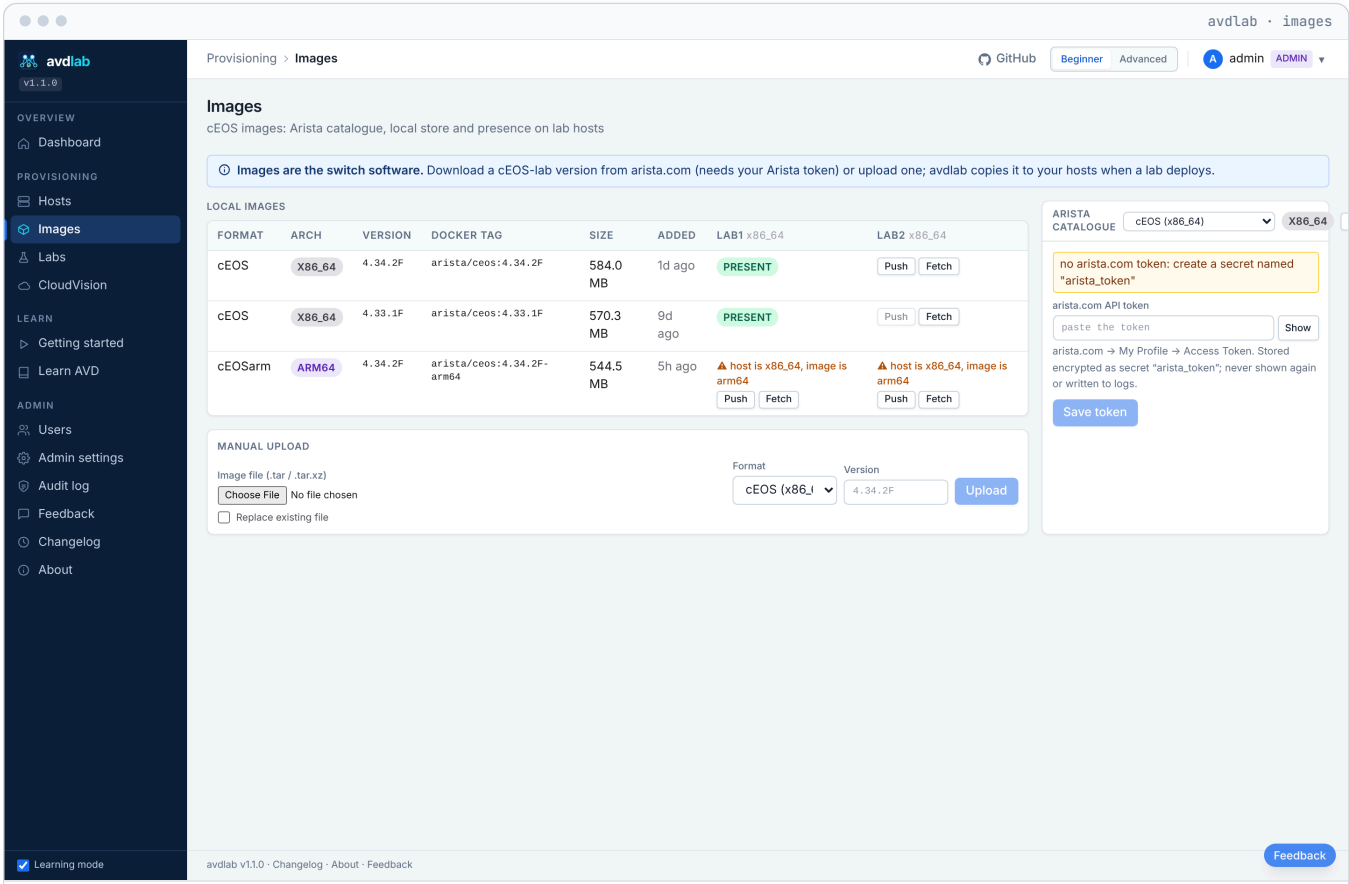
Host detail: probe, bootstrap recipe, monitor graphs and capabilities.

Table columns: Name (with a violet *platform · beta* badge for OrbStack/WSL hosts), Kind, SSH (`user@address:port`), Auth (key / password / none), OS, Docker, containerlab, Recs (number of recommendations, or *ok*), Last probe, Actions.

CONTROL	DOES
Generate SSH key	creates an ed25519 key pair (private key encrypted in the secret store, owned by you with purpose `ssh`), shows the public key and an install command
Add host / Edit	the host dialog: platform card (Linux / Mac (OrbStack VM) beta / Windows (WSL2) beta, with the setup steps), name, kind (*containerlab host* or *act*), address, SSH user and port, jump host (Mac/Windows only; `[user@]host[:port]`, no option-like or shell characters), SSH key or password, sudo password. The secret pickers list **your own** secrets (admins: everyone's, with the owner's name); binding someone else's secret or one with the wrong purpose is refused with *403* and the violation (1.1.0)
Monitor	the host page: metrics charts, processes, containers (Stop/Kill), terminal
Probe	job `host.probe`
Bootstrap	dialog: recipe (`clab-host` or `act-tools`), containerlab mode (native / container wrapper), native toolchain checkbox → job `host.bootstrap`
Verify	job `host.verify`
Delete	removes the host (admins only)
click the name	Capabilities panel: System, Toolchain, sysctl, Recommendations

Details, recipe steps and probe output: chapter 4.

5.6 Images



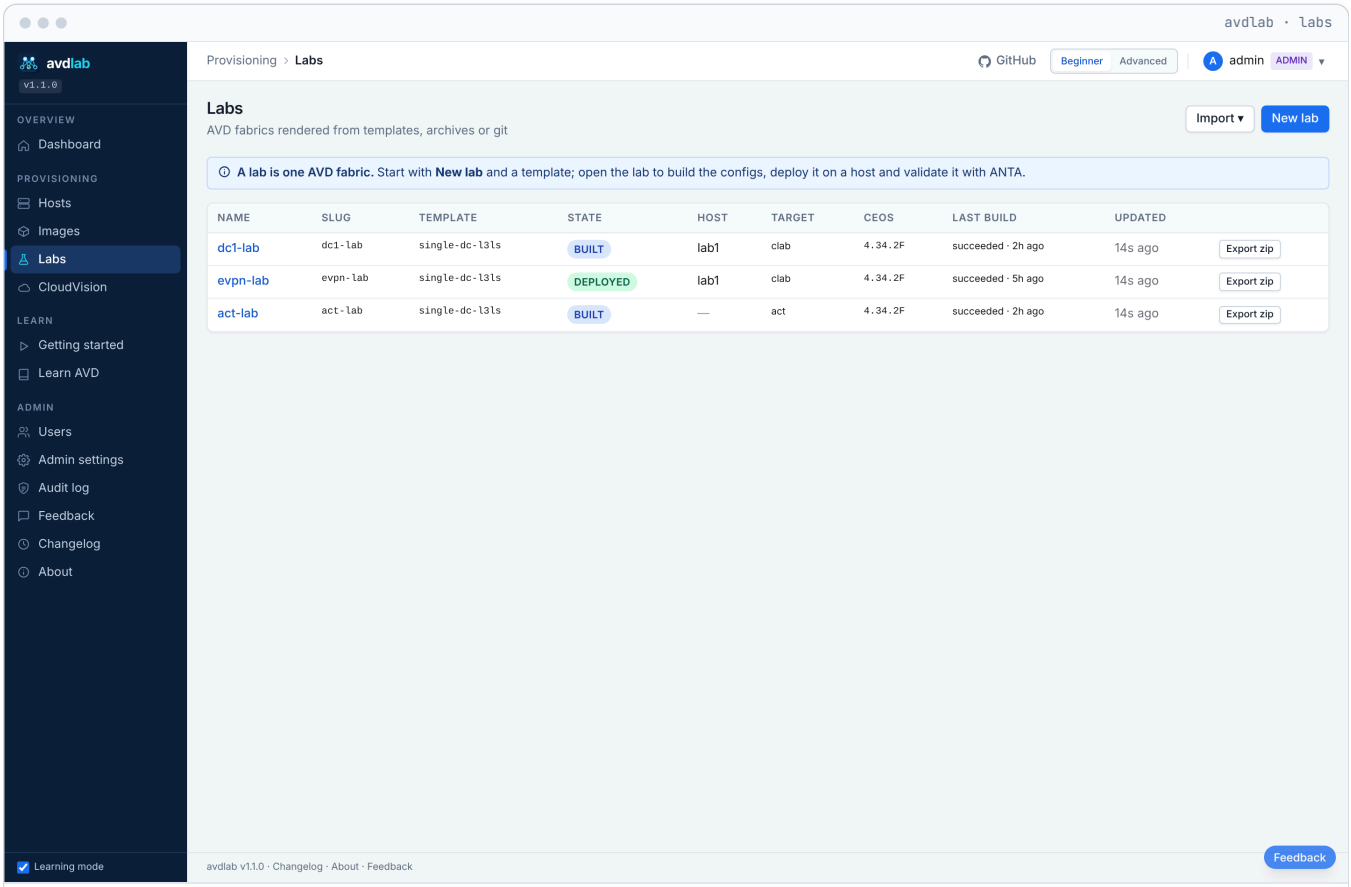
Images: arista.com token, catalogue and locally downloaded cEOS images.

AREA	CONTROLS
Header	**arista.com token…** — always reachable: paste (show/hide while typing), *Replace* (same secret, no duplicates), *Remove*. Stored as secret `arista_token` (purpose `arista`), never shown again and masked in job logs; replacing it drops the cached catalogue.
Local images	Format, Arch, Version, Docker tag, Size, Added, hosts that have it; per host *Push* (or *Fetch* = download on the host with `ardl`); delete
Arista catalogue	format selector (`cEOS`, `cEOS64`, `cEOSarm`), version filter (e.g. `4.34`), *Download* per version (job `image.download`), *downloaded* marker. When arista.com rejects the token: *Replace token* / *Retry*. Without a token: 409 and a prompt to add one. Answers are cached for an hour.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

AREA	CONTROLS
Manual upload	image file (`.tar` , `.tar.gz` or `.tar.xz` ; any other extension is refused), format, version → stored as `<fmt>-lab-<version>.<ext>` (the server derives the name) and `docker import` ed locally when Docker is available. **Replace existing file** checkbox (1.1.0): without it an upload for a version that already has a file answers *409 … already exists: send replace=true*, shown under the form. Uploads above `AVDLAB_UPLOAD_MAX_BYTES` (4 GiB; Caddy caps at 4 GB too) are refused with 413 and the partial file removed

5.7 Labs



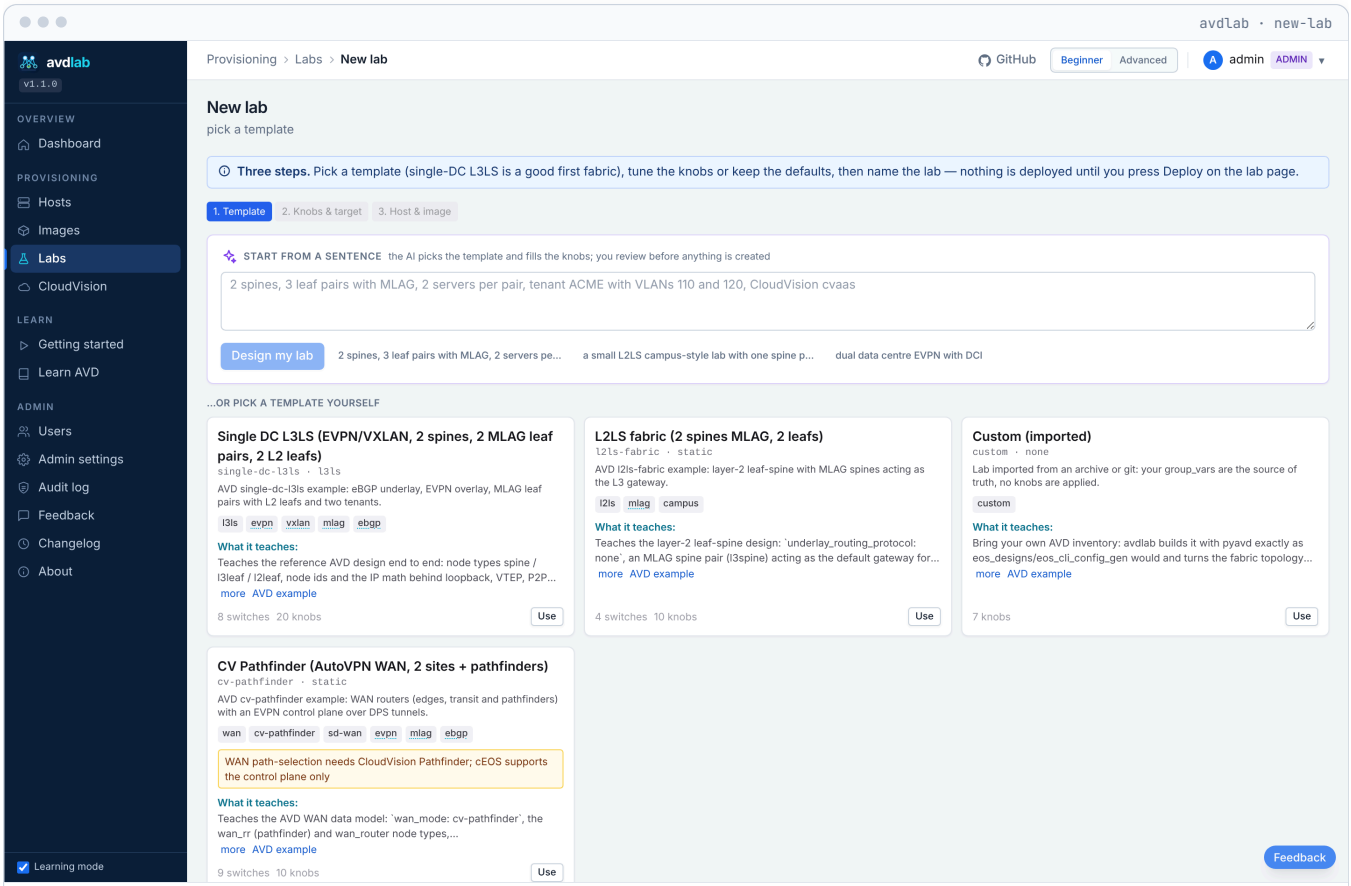
The lab list with state, host, template and the last job.

The list shows Name, Slug, Template, State, Host, Target, cEOS, Last build, Updated, and *Export zip* per lab.

- **New lab** opens the wizard (5.8).
- **Import ▾ → Upload archive (.zip / .tar.gz)** — the archive must contain an Ansible inventory (`inventory.yml` , `inventory.yaml` or `hosts.yml`) and `group_vars/` ; one top-level directory is stripped; `intended/` , `documentation/` , `clab/` , `anta/reports/` and `.git/` are ignored (they are rebuilt). The lab uses template `custom` . Invalid AVD inputs still create the lab (`draft`) with *import warnings* shown on the lab page.
- **Import ▾ → Clone from git** — repository URL (no credentials in the URL; `https://` , `ssh://` , scp-like — local paths and `file://` only when the installer set `AVDLAB_GIT_ALLOW_LOCAL`), branch (validated: letters, digits, `.` , `_` , `/` , `-` , no `..`), authentication (none / access token — refused for `http://` and `git://` remotes / SSH key secret with purpose `git` that you own), optional host → job `lab.import_git` .

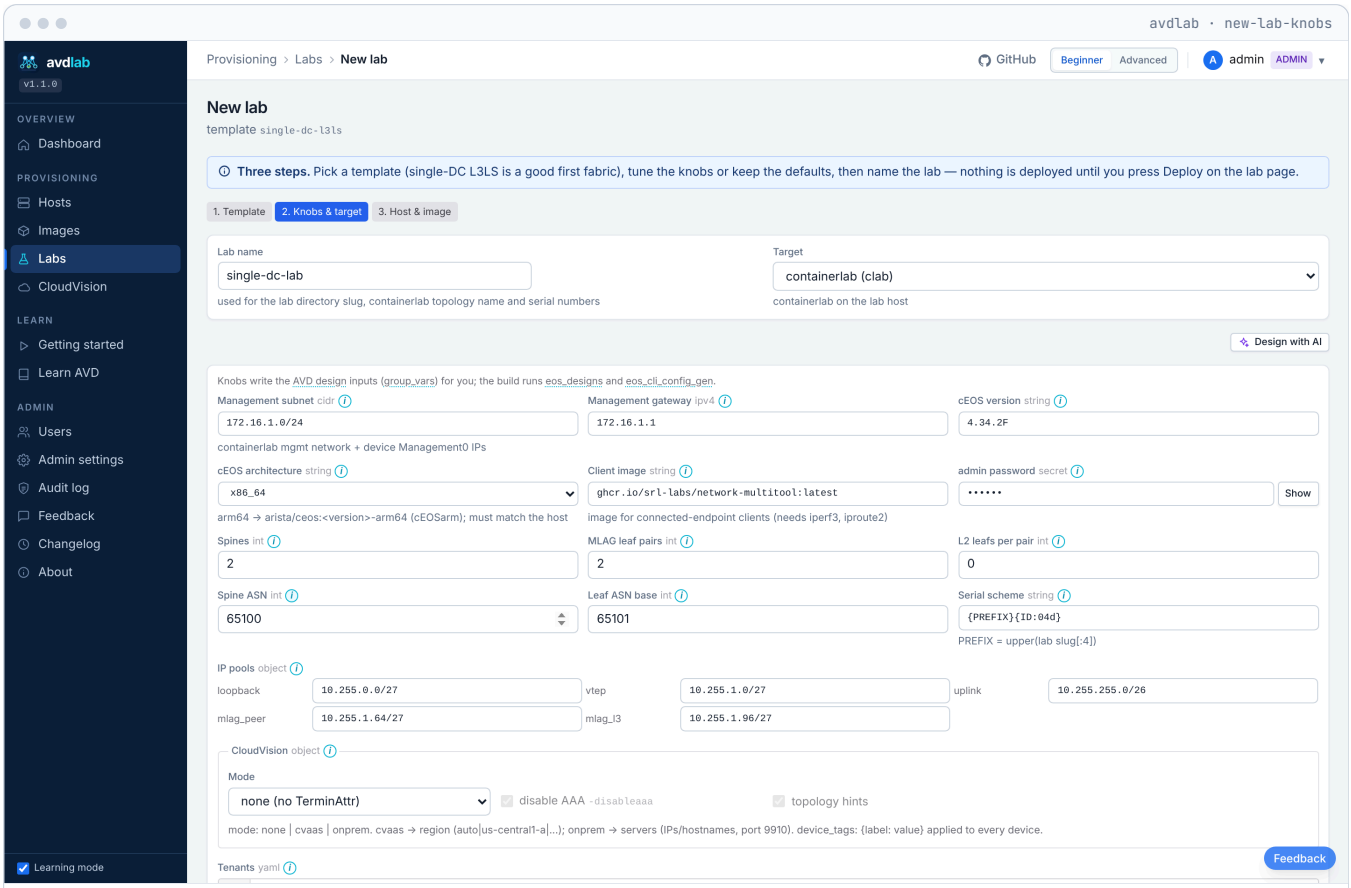
© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.8 New lab



New lab: pick a template, the AI card, or start from a sentence.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



The knobs of single-dc-l3ls after choosing the template.

Three steps: **1. Template**, **2. Knobs & target**, **3. Host & image**. Nothing is created until *Create lab*.

STEP	CONTENTS
1. Template	**Start from a sentence** card (AI: picks the template and fills the knobs, see chapter 6), or *...or pick a template yourself*: cards with title, tags, node count, notes and "What it teaches" with the AVD doc link
2. Knobs & target	lab name (used for the slug, the containerlab topology name and serial numbers), target (*containerlab (clab)* / *Arista Cloud Test (act)*), the typed knob form with inline validation; *Design with AI* to refine
3. Host & image	lab host (optional now, required to deploy; for target act only hosts of kind act), cEOS image picker (lists downloaded images; *Other...* for free text). Picking an arm64 host switches `ceos_arch` to `arm64`. Server-side violations (422) are listed.

Knobs of `single-dc-l3ls` (from `templates/single-dc-l3ls/template.yml`)

KNOB	TYPE	DEFAULT	MEANING
`dc_name`	string	`dc1`	DC name / hostname prefix: `<dc>-spine<n>`, `<dc>-leaf<n>a/b` (L3 MLAG pair), `<dc>-leaf<n>c/d` (L2 leafs); groups use `upper(dc)`
`mgmt_subnet`	cidr	`172.16.1.0/24`	management subnet, also the containerlab management network; spines get .11+, L3 leafs .101+, L2 leafs .151+
`mgmt_gateway`	ipv4	`172.16.1.1`	management gateway
`ceos_version`	string	`4.34.2F`	Docker image `arista/ceos:<version>` used in the containerlab topology
`ceos_arch`	string, choices `x86_64` / `arm64`	`x86_64`	`x86_64` → `arista/ceos:<version>` (cEOS/cEOS64), `arm64` → `arista/ceos:<version>-arm64` (cEOSarm); must match the host CPU
`device_password`	secret	`arista`	password of the `arista` local user (sha512) and of all BGP peer groups (type-7); `ansible_password` follows
`spine_count`	int 1–4	`2`	with more than 2 spines the leaf uplinks use Ethernet1-4 and the MLAG peer-link Ethernet5-6; server ports on Ethernet3-6 shift up automatically (Ethernet5 → Ethernet7)
`leaf_pairs`	int 1–8	`2`	L3 leaf MLAG pairs; spine downlinks extend beyond Ethernet1-8 automatically above 4 pairs
`l2leaf_per_pair`	int 0–2	`1`	0 = none, 1 = single L2 leaf (`<dc>-leaf<n>c`), 2 = an MLAG pair of L2 leafs (c/d) sharing a node group

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

KNOB	TYPE	DEFAULT	MEANING
`pools`	object of pools	loopback `10.255.0.0/27`, vtep `10.255.1.0/27`, uplink `10.255.255.0/26`, mlag_peer `10.255.1.64/27`, mlag_l3 `10.255.1.96/27`	loopback (Loopback0, spines then leafs), vtep (Loopback1), uplink (/31 P2P), mlag_peer (Vlan4094), mlag_l3 (Vlan4093); AVD pool syntax is validated
`spine_asn`	int	`65100`	spine ASN
`leaf_asn_base`	int	`65101`	first leaf-pair ASN; pair n gets `leaf_asn_base + n - 1`
`serial_scheme`	string	`{PREFIX}{ID:04d}`	Python format: PREFIX = upper(lab slug[:4]), ID = position in the inventory, also HOST, TYPE, SLUG; written to each node's `serial_number` and to the cEOS `ceos-config` file
`tenants`	yaml	from the example's `NETWORK_SERVICES` `tenants`	`tenants:` list of the AVD network services model, validated with pyavd
`servers`	yaml	from the example's `CONNECTED_ENDPOINTS` `servers`	`servers:` list; endpoints become linux containers in the topology
`dns_servers`	yaml	`["192.168.1.1"]`	resolvers of the switches (management VRF); use public resolvers (1.1.1.1, 8.8.8.8) when the lab must reach CVaaS
`cloudvision`	object	`{mode: none, region: auto, servers: [], disable_aaa: true, topology_hints: true, device_tags: {}}`	mode `none` / `cvaas` / `onprem`; cvaas → region (the AVD 6.4 list, e.g. `euwest-2`, `us-4`, `apnortheast-1`); onprem → servers (port 9910); `device_tags` applied to every device. Mode `none` strips the example's TerminAttr config.
`client_image`	string	empty	Docker image of the linux client containers; needs iperf3, iproute2 (bond/vlan) and ping; empty = `ghcr.io/srl-labs/network-multitool:latest`
`device_tags`	yaml	`{}`	CloudVision tags per device `{hostname: {label: value}}`, merged after the global tags (per-device wins); unknown hostnames are rejected
`provisioning`	object	`{mode: startup-config, ztp_source: intended, ztp_server_ip: null}`	`startup-config` (containerlab mounts intended configs) or `ztp` (cEOS boots into ZTP; a `ztp` node runs dnsmasq + HTTP; option 67 serves `<node>.cfg`, or with `ztp_source: cloudvision` a bootstrap script running the CloudVision target's ZTP script / writing the onboarding token); empty `ztp_server_ip` = highest free IP of the management subnet
`segmentation`	yaml	`{}`	MSS foundation: `router segment-security` with applications, policies and VRF segments; rendered only on the L3 leafs that carry each VRF
`act`	object	`{os_version: 4.33.1F, username: cvpadmin, password: cvp123!, internet_access: true, tools_server_ip: ""}`	used for target act: digital-twin build (vEOS-lab, Management1), ACT topology with a tools server and one ubuntu node per client; the ACT credentials are also added as a local user. Since 1.1.0 `act.password` is stored encrypted like a secret knob and always displayed as `********`; leaving the mask in place keeps the stored value, a newline in the password is refused

The static templates expose a subset (`mgmt_subnet`, `mgmt_gateway`, `ceos_version`, `ceos_arch`, `device_password`, `serial_scheme`, mostly `dns_servers`, `cloudvision`, `client_image`, `device_tags`, `provisioning`, `segmentation`, `act`; `single-dc-l3ls-ipv6` adds `mgmt_ipv6_subnet`); `custom` only the topology-shaping knobs.

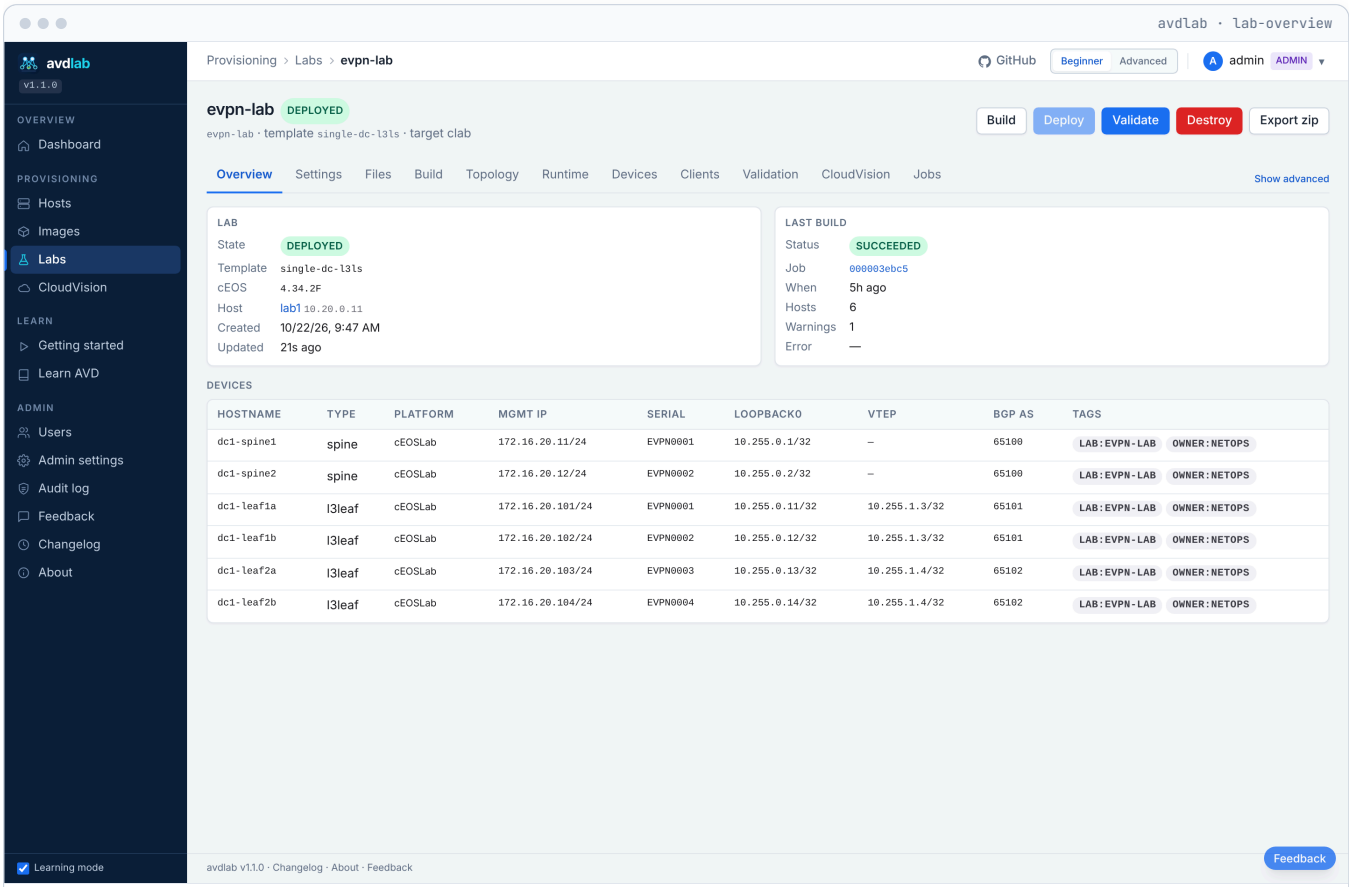
5.9 A lab: header and tabs

The **header** shows name, state badge, slug, template and target, with *Build*, *Deploy*, *Validate*, *Destroy* (confirmation), *Export zip* and *Delete* (only when not running). Which buttons are enabled depends on the state (chapter 1 §1.4). The header *Deploy* uses `reconfigure=true, wait_eapi=true, wait_timeout=600`; the header *Validate* uses extra fabric validation and skips `VerifyNTP` and `VerifyReloadCause`.

Tabs: Overview, Settings, Files, Build, Topology, Runtime, Devices, Clients, Validation, CloudVision, MSS (A), NetBox (A), ACT (A, only for target act), Git & VS Code (A), Jobs. (A) = hidden in Beginner mode; *Show advanced* at the end of the tab row switches mode.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Overview



Lab overview: state, actions, the last jobs and the targets.

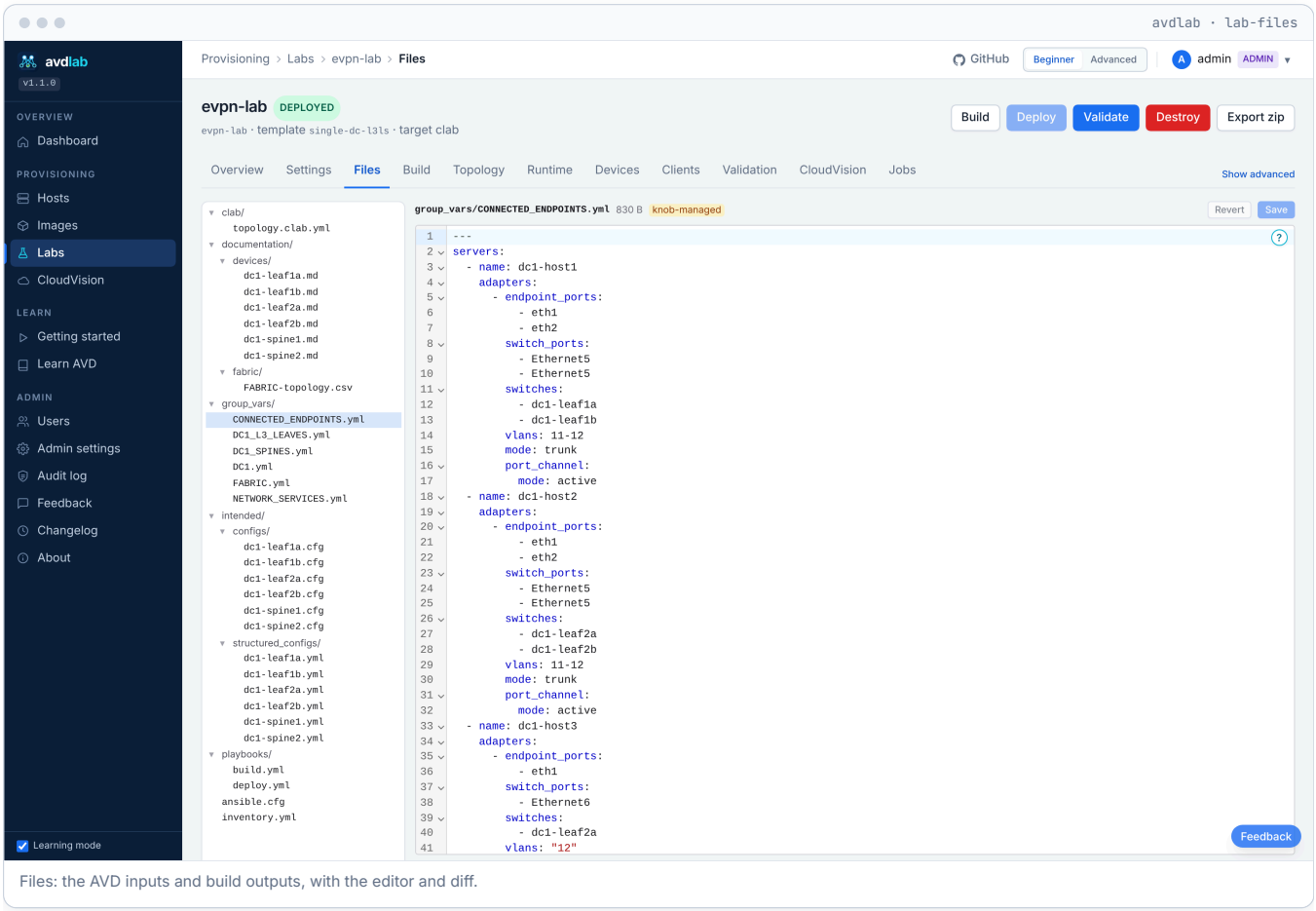
Lab facts (state, template, cEOS, host, created/updated), **Last build** (status, job link, host count, warnings, error) and the **Devices** table from the structured configs: hostname, type, platform, mgmt IP, serial, Loopback0, VTEP, BGP AS, CloudVision tags.

Settings

A sticky bar at the top with **Save** (validates, re-renders, commits), **Reset**, an unsaved-changes counter and **Design with AI**. Fields: **Target** (locked while the lab runs), **Host**, **cEOS image** (image picker; also sets `ceos_arch`), **CloudVision target** (enabled only when the `cloudvision` knob mode is not `none`). Then the typed **knob form**: structured sub-forms for `cloudvision`, `provisioning`, `act`, a device-tags editor, YAML editors for `tenants` / `servers`, (i) help popovers in learning mode, and *custom* markers for knobs whose files were edited by hand. Violations (422) are shown next to the knob. Saving a knob change of a built lab resets it to `draft`. Saving replaces hand edits of knob-managed files.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Files

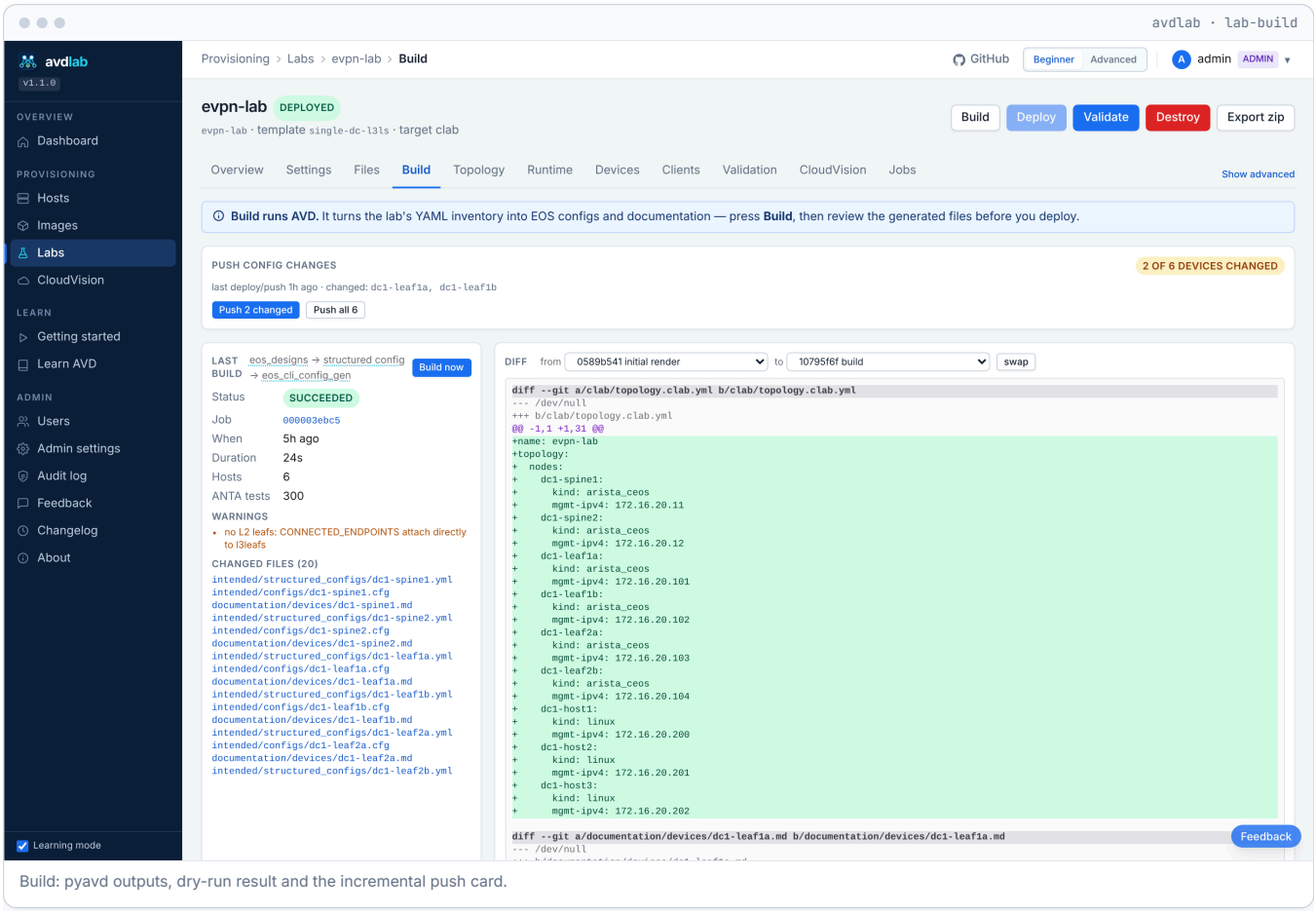


Files: the AVD inputs and build outputs, with the editor and diff.

A file tree of the lab directory and an editor. Editable: `inventory.yml`, `ansible.cfg`, `group_vars/`, `host_vars/`, `playbooks/`, `anta/user_catalogs/`; read-only (generated): `intended/`, `documentation/`, `clab/`, `anta/`. Files marked **knob-managed** (`inventory.yml`, `group_vars/`) are validated with pyavd before saving and mark the related knobs as custom; YAML syntax is checked for every `.yml`. Each save is a commit; *Revert* discards unsaved edits. In learning mode the config viewer offers click-to-explain (which AVD input produced a line, with the lab's real values).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

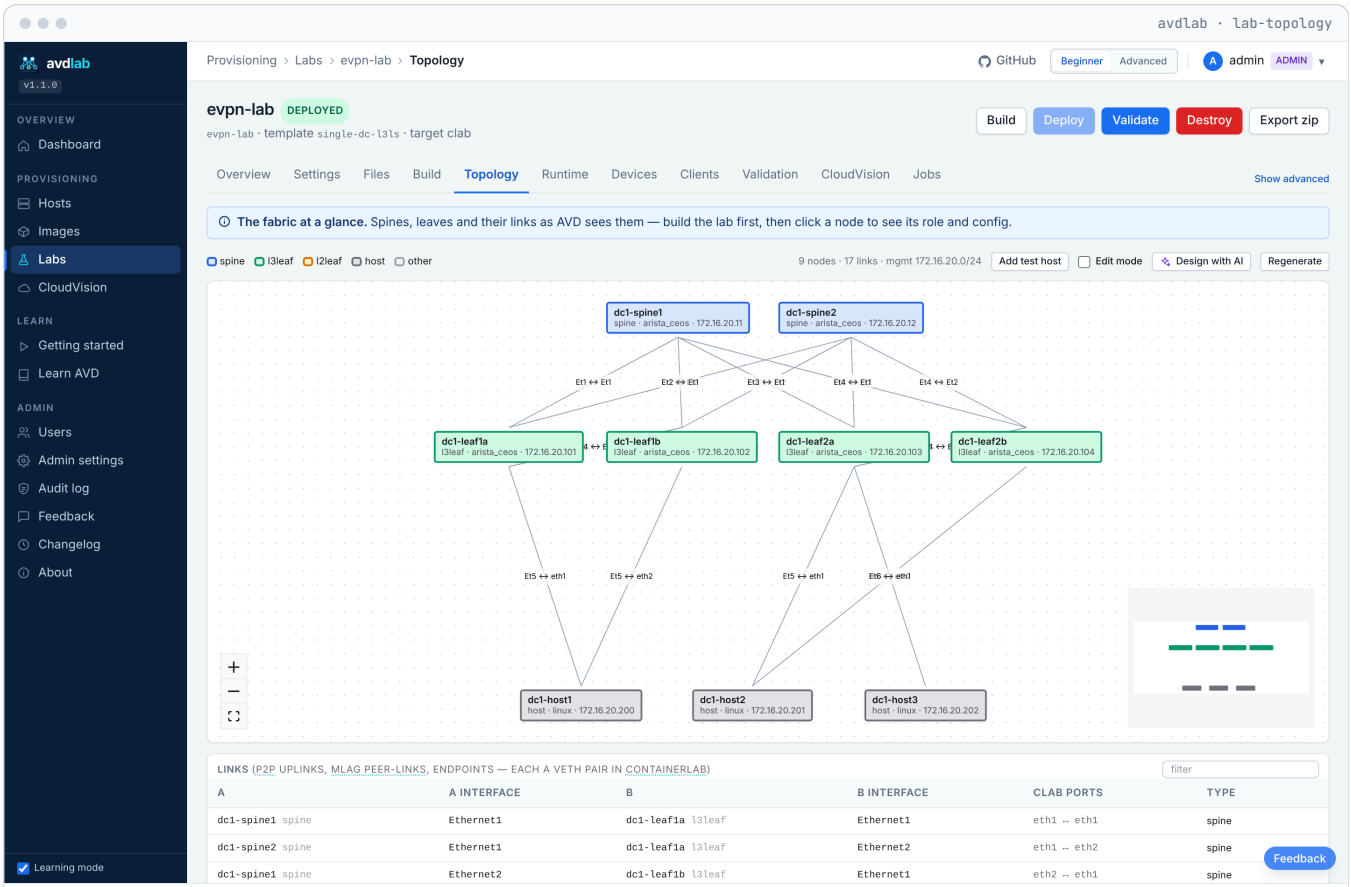
Build



Build: pyavd outputs, dry-run result and the incremental push card.

Push config changes card (when the lab runs), **Last build** (status, job, duration, hosts, ANTA test count, warnings, error, changed files linking to Files, previous builds) with *Build now*, a **Diff** between any two revisions (from / to / swap) and the **History** of commits (rev, message, author, date, files, *diff* to the previous one).

Topology



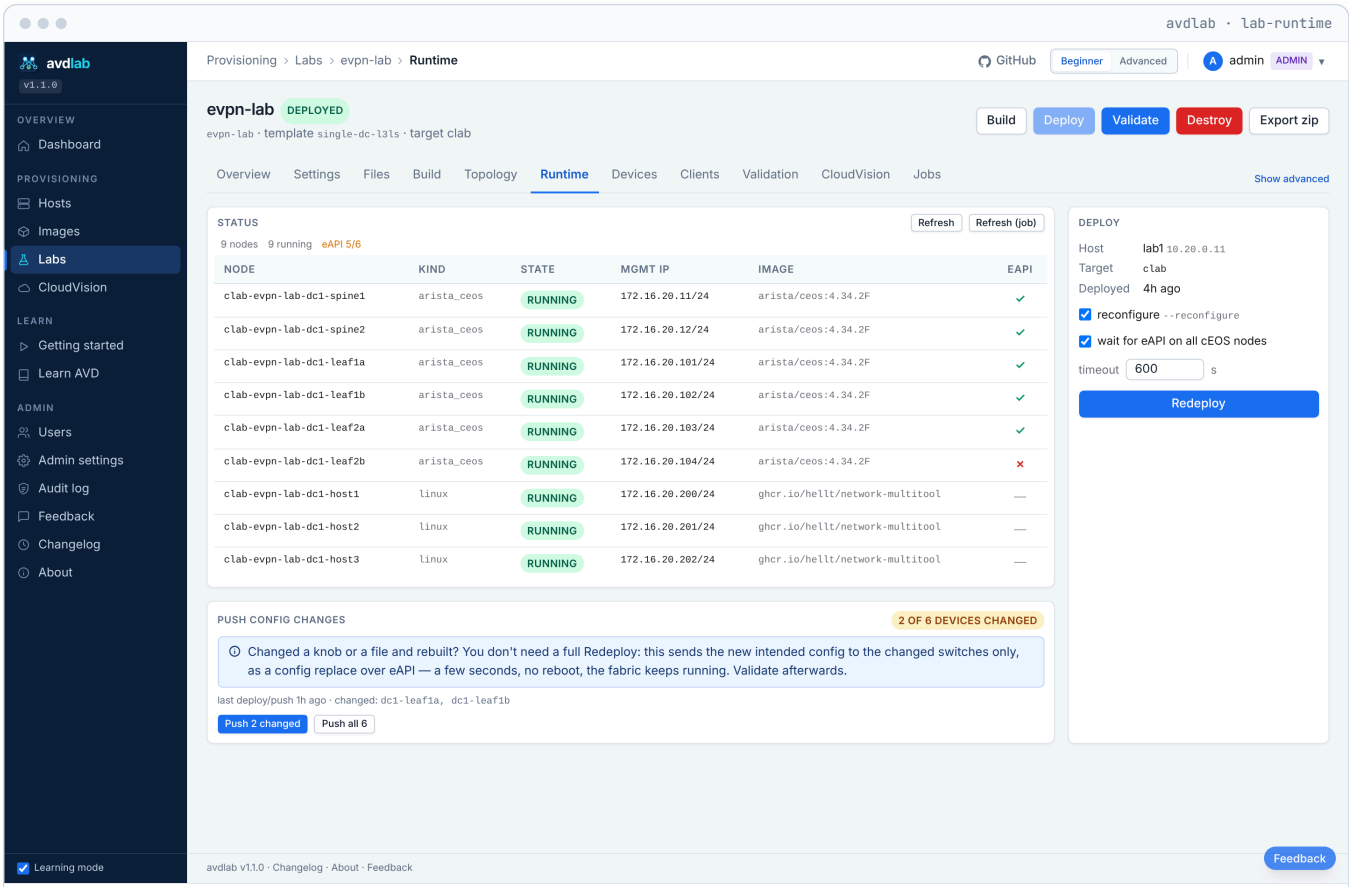
Topology editor: spines, leaf pairs, test hosts and links; edits are input changes.

The fabric graph as AVD sees it (React Flow): spines, leafs, L2 leafs, endpoints, with a links table (P2P uplinks, MLAG peer-links, endpoints — each a veth pair; A/B interfaces and the containerlab port names). Controls: filter, *Regenerate* (job `lab.topology`, rebuilds `clab/` from the last build), **Add test host**, **Design with AI**, *Auto-layout*, *lock layout* (positions are saved per lab in `clab/layout.json`).

Edit mode turns the graph into an editor of the AVD inputs: drag the switch end of a blue uplink onto another switch to move it to a free port (`set_uplink`), *Add server* (name, type, attach to a leaf pair or single leaf, NIC, mode access/trunk, VLANs, native VLAN, LACP port-channel for MLAG), right-click a server to remove it, *+ leaf pair*, *− last leaf pair*, L2 leafs per pair and spine count (l3ls templates only). Changes queue as **pending operations**; a dry run shows the changed files and the diff; *Apply* validates with pyavd, commits, sets a built lab back to `draft` and offers *Build now* / *Later*. Click a node to inspect it (role, config, uplinks with *move…*, free ports — click a free port to plug in a test host).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Runtime

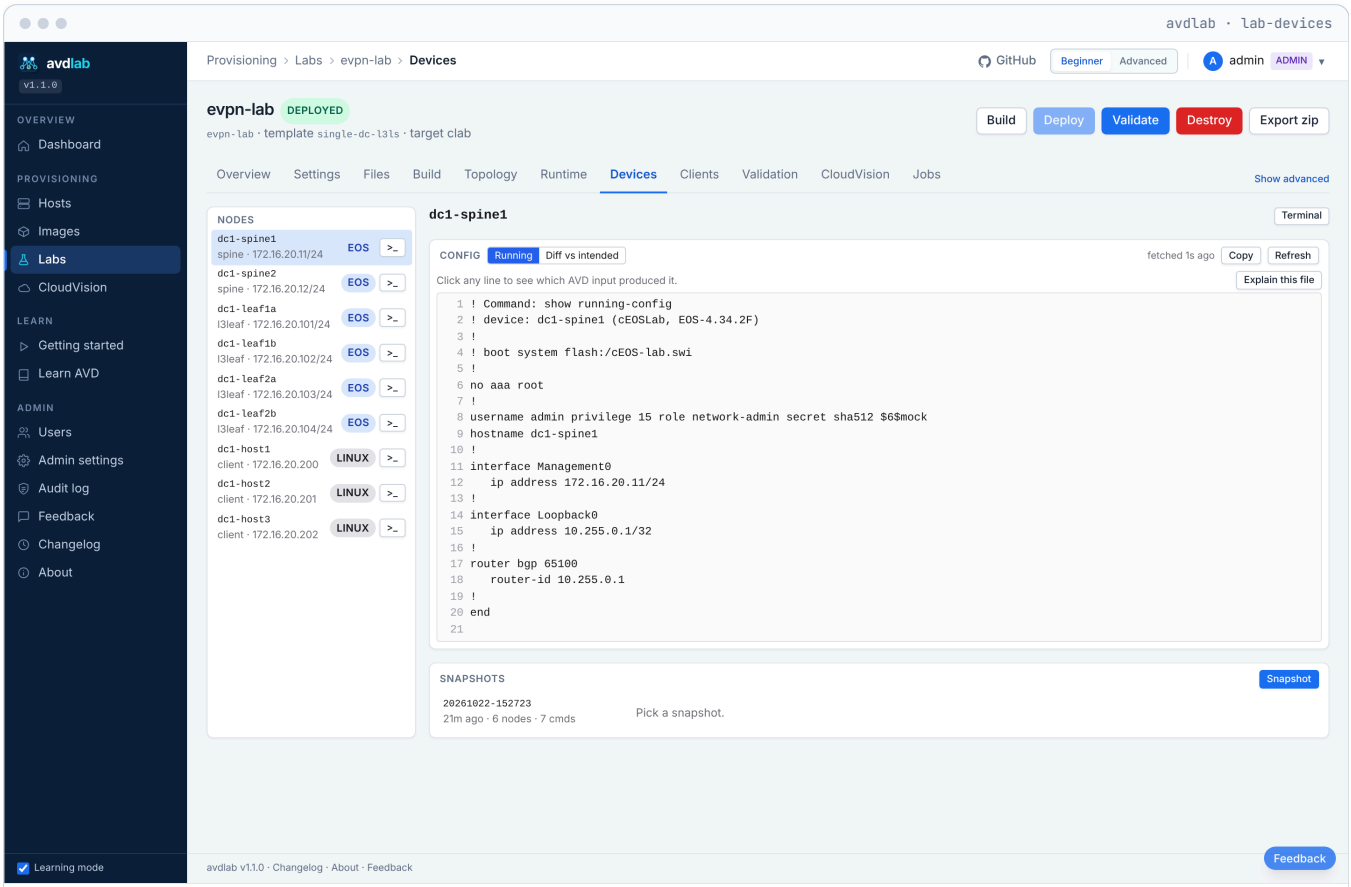


Runtime: the containers on the lab host, console, and the push-configs card.

Status table from `containerlab inspect` plus an eAPI probe per node: node, kind, state, mgmt IP, image, eAPI ✓/×, with a summary (nodes, running, eAPI n/m). *Refresh* runs the inline status (≤ 20 s; a cached result is used when the host does not answer); *Refresh (job)* runs `lab.status` as a job. **Deploy** panel: host, target, deployed-at, options *reconfigure* (`--reconfigure`), *wait for eAPI on all cEOS nodes*, *timeout* (60–3600 s), and *Deploy* / *Redeploy*. In ZTP mode a **ZTP panel** shows per node MAC, IP, DHCP lease, bootfile fetched, eAPI (polled during the deploy, refreshed from the dnsmasq leases and HTTP log).

Push config changes (containerlab labs that are running): after a *Build*, the card lists the switches whose intended config changed since they last received it (*changed*), and switches with no record (*not tracked yet* — deployed before 0.10.1, or new). *Push N changed* sends only those, *Push all N* every switch; both run `lab.push_configs` : an `eos_config` **config replace over eAPI** from the agent container — seconds, no container restart, the fabric keeps running. Validate afterwards.

Devices

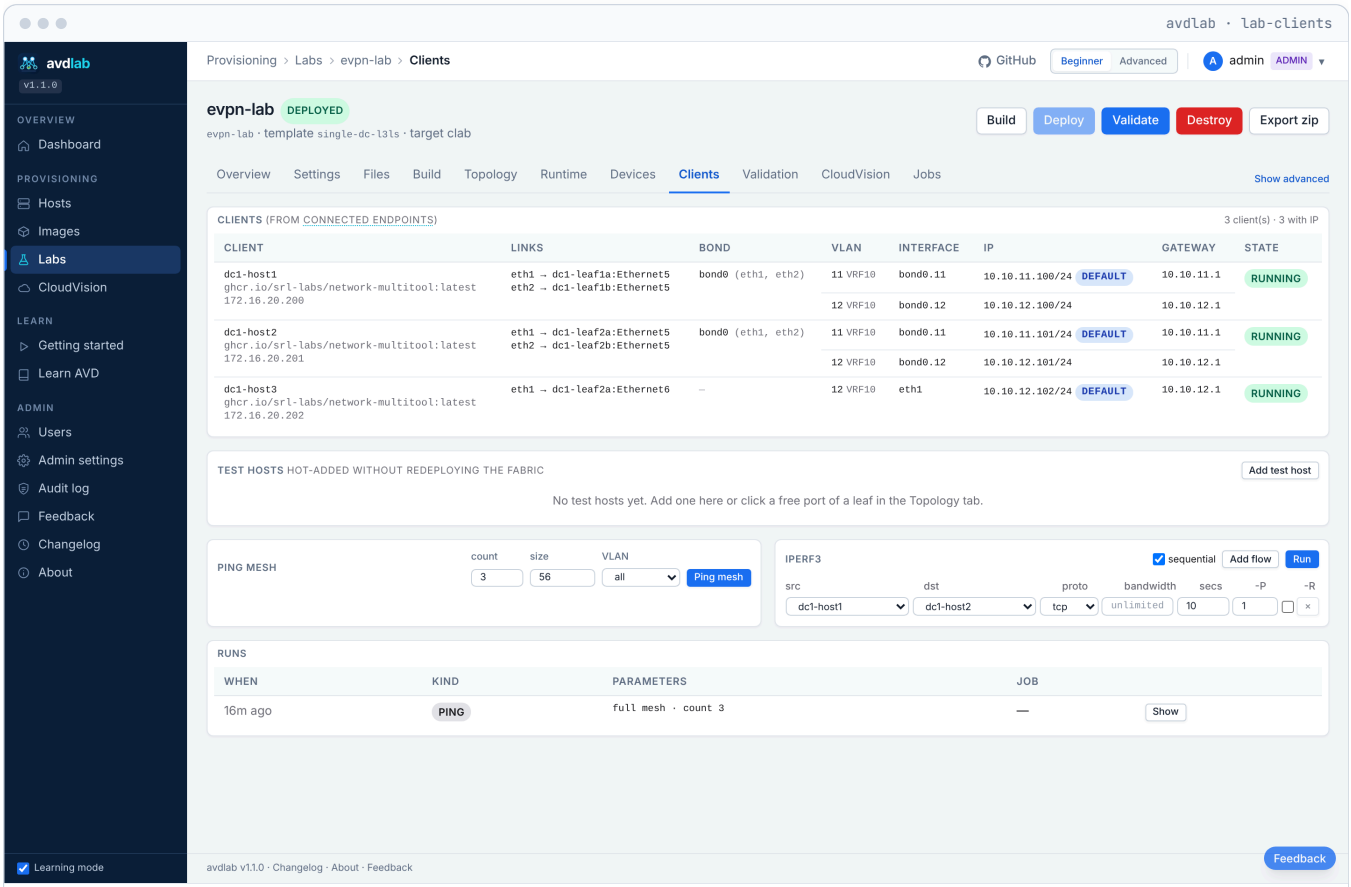


Devices: per switch management IP, version, eAPI and web terminal.

The node list (switches and linux clients). Per switch: **Running** config (`show running-config`, fetched inline), **Diff vs intended** (normalised unified diff; "Running config matches the intended config" or a *drift* badge), the config explainer, and a **Terminal** (EOS `Cli` or `bash` through `docker exec`, over the SSH connection; operators and admins). Linux clients open a bash terminal. **Snapshots**: pick nodes and commands (default `show running-config`, `show version`, `show lldp neighbors`, `show ip bgp summary`, `show bgp evpn summary`, `show mlag`, `show vxlan vni`) → job `lab.snapshot` stores the outputs under `snapshots/<timestamp>/<node>/` in the lab repo (committed) and lists them.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Clients & test hosts



Clients & test hosts: add Linux test hosts behind leaf ports and run traffic.

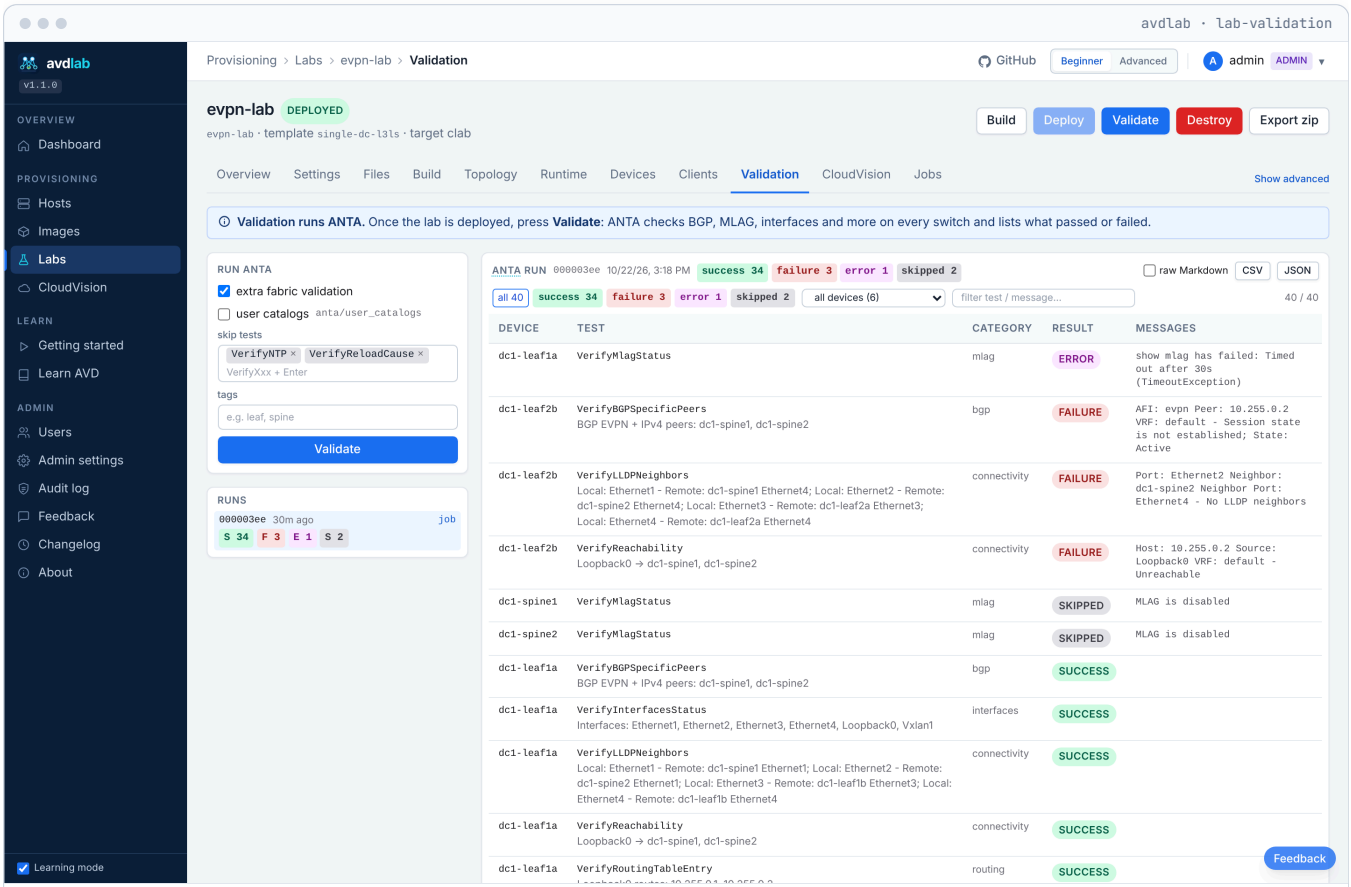
Clients are derived from AVD connected endpoints: each becomes a linux container whose plan shows links, bond (802.3ad for port-channels), VLAN sub-interfaces for trunks, addresses (`.100 + index` in each SVI subnet), gateway (the anycast SVI address) and state. Traffic tests (lab must be running):

TEST	OPTIONS	RESULT
Ping mesh	count (default 3), size (56), VLAN, all pairs or selected	loss matrix src ↓ / dst →, 0 % / partial / 100 %
iperf3	flows (src, dst, proto tcp/udp, bandwidth, seconds, …), sequential	per flow sent / received / retransmits / jitter / loss
Runs	history of all ping, iperf and MSS runs with parameters and job	detail view

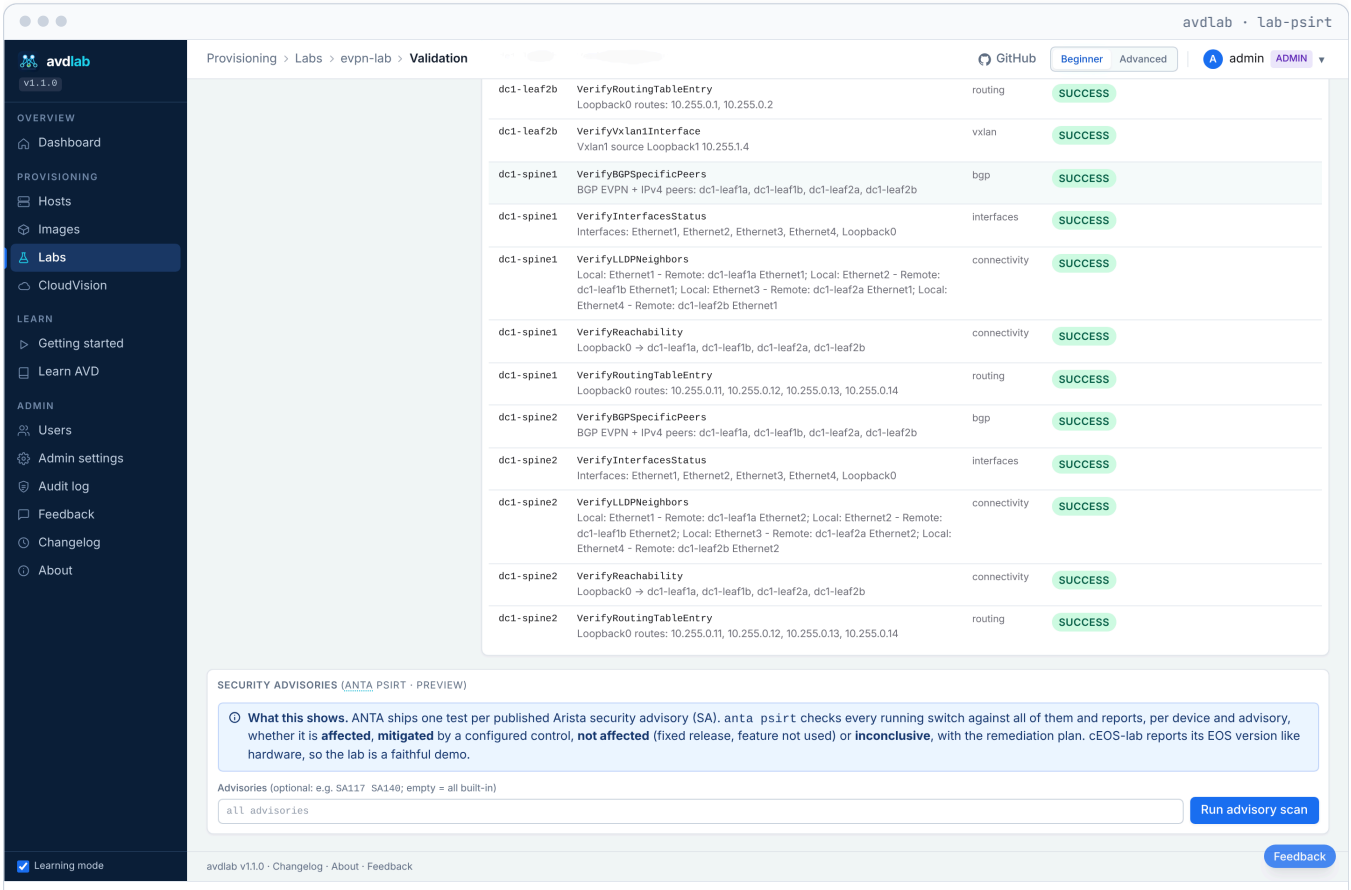
Test hosts are hot-added without redeploying: *Add test host* (name, leaf, switch port, mode access/trunk, VLANs, IP *auto* or manual, image) adds the endpoint to the AVD inputs, rebuilds only the affected switch(es), pushes their new config, starts the container and connects it with `containerlab tools veth create`. Per test host: *ping*, *iperf*, *exec*, *Open terminal*, remove. If a switch is not actually running (e.g. after a host reboot) the add is refused with the name of the switch and a hint to Redeploy; a remove still completes with a warning. A hot-added port may not appear as `EthernetN` on the switch until a full redeploy (the job result says `interface_visible: false`).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Validation



Validation: ANTA runs, catalogs and reports.

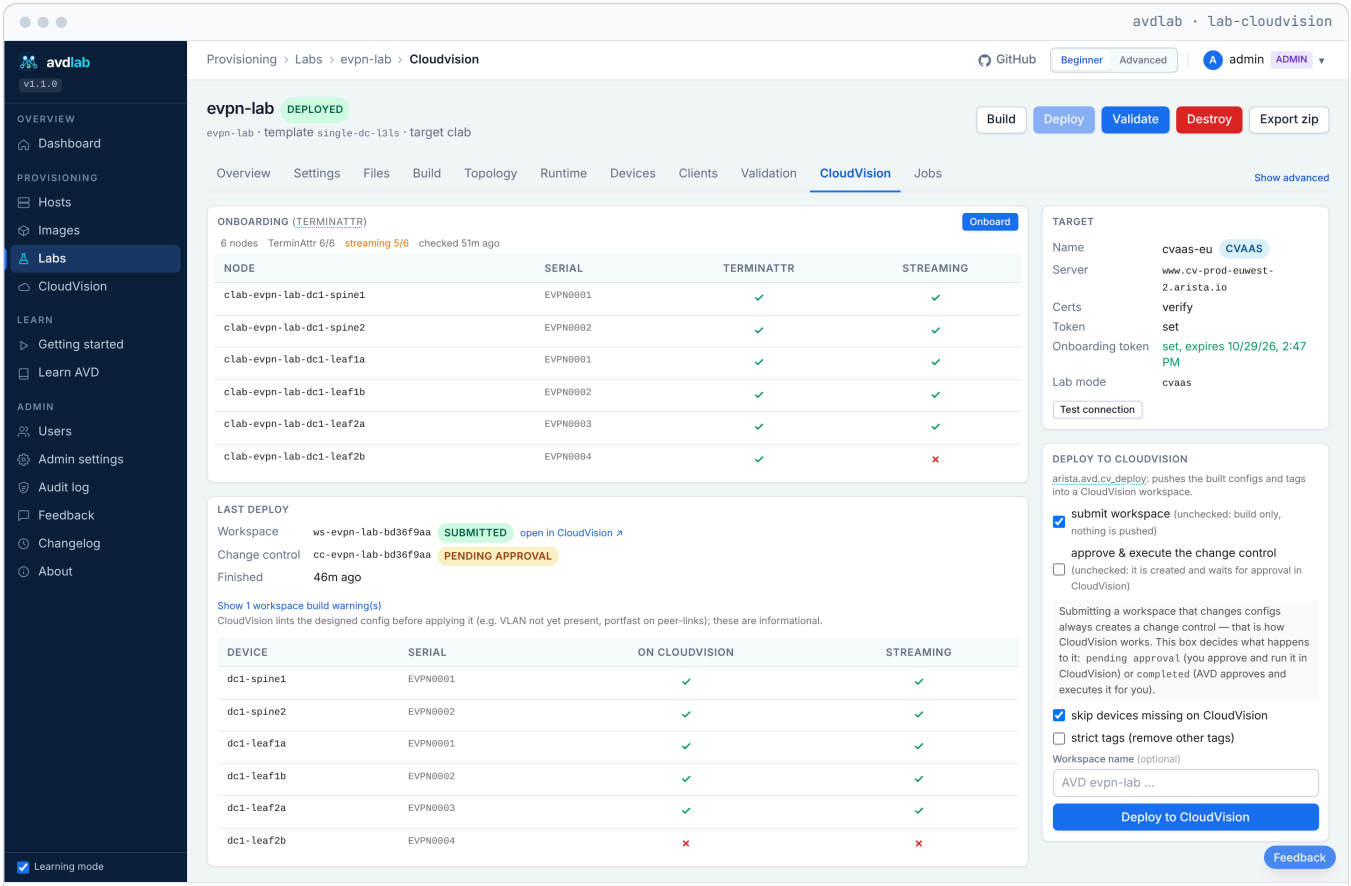


The ANTA PSIRT card (preview): a security-advisory scan of the running switches.

Run ANTA with options: extra fabric validation, skip tests (default `VerifyNTP`, `VerifyReloadCause`; type `VerifyXxx` + Enter), tags (e.g. `leaf`, `spine`), and *anta/user_catalogs* (include your own catalogs from the Files tab). **Runs** lists every validation with totals; a run opens the **results table** (device, test, category, result, messages) with status, device and text filters, and downloads (JSON, CSV, Markdown report).

Security advisories (ANTA PSIRT, preview) card: *Run advisory scan* (optionally only some advisories, e.g. `SA117 SA140`; empty = all built-in) runs `anta psirt` with the dedicated ANTA 1.10 image against the running switches (job `lab.psirt`) and shows device × advisory: **affected**, **mitigated** (by a configured control), **not affected** (fixed release, feature not used) or **inconclusive**, with severity, vulnerability details, remediation plan and the Markdown report. cEOS-lab reports its EOS version like hardware, so the lab is a faithful demo. Reports are kept under `anta/psirt/` in the lab repo.

CloudVision



CloudVision tab: target, onboarding, workspace and change-control deploy.

Prerequisites: a CloudVision target (5.10), the `cloudvision` knob set to `cvaas` or `onprem` (rebuild after changing it), the target bound in Settings, and public DNS for CVaaS (`dns_servers`).

CARD	CONTENTS
Target	name, kind, server, verify-certs, token and onboarding-token status, *Test connection*
Onboarding	*Onboard* (job `lab.cv_onboard`, lab must be running): writes the onboarding token into every cEOS node at the token file the build rendered (via `docker exec -i`, plus a copy on flash), restarts TerminAttr, then polls TerminAttr and (with a service-account token) the CloudVision inventory. Table: node, serial, TerminAttr ✓/×, streaming ✓/×; TerminAttr log tails for nodes that do not stream.
Deploy to CloudVision	`arista.avd.cv_deploy` (job `lab.cv_deploy`, lab built or running) pushes the built configs and tags into a CloudVision workspace. Options below.
Last deploy	workspace (id, state, *open in CloudVision ↗*), change control, devices (exists on CloudVision, streaming), errors, collapsed CloudVision lint warnings

Deploy options and their semantics:

OPTION	DEFAULT	MEANING
submit workspace	on	unchecked: the workspace is only built, nothing is pushed
approve & execute the change control	off	a submitted workspace with changes always creates a change control; unchecked → it waits in **pending approval** (you approve and run it in CloudVision), checked → AVD approves and executes it (**completed**)
skip devices missing on CloudVision	on	devices not (yet) in the inventory are skipped instead of failing
strict tags (remove other tags)	off	tags not set by AVD are removed
Workspace name	empty	optional name

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Outcomes explained on the tab: **Deploy failed before a workspace was created** (job link, typical causes — the 15 s pre-flight from the orchestrator turns an expired/rejected token or an unreachable server into a precise message before the playbook runs), or **Workspace submitted, nothing to push** — no change control is created when the designed config equals the running config, which is the normal answer right after a deploy with the same intended configs.

MSS (advanced)

Segmentation (MSS foundation — design to be refined): edit **applications** (name, protocol tcp/udp/icmp/any, ports), **policies** (rules: application → forward/drop, log) and **VRFs & segments** (prefixes, interfaces, "from → policy", fallback); the **policy matrix** per VRF (preview of unsaved changes), **Rendered on** (which L3 leafs carry the VRF and get `router segment-security`). **Verify** (job `lab.mss_verify`): auto-derived or explicit cases (src, dst, application, expect allow/deny) probed between clients with ping / `nc` / iperf3; results pass/fail per case.

NetBox (advanced)

Pick a **NetBox target** (bound to the lab), options *dry run — nothing written*, *delete stale objects tagged avdlab*:, include clients; **Sync** (job `lab.netbox_sync`, lab must be built). Last sync: created / updated / deleted / unchanged / skipped per object type and *Open in NetBox ↗*. The sync creates sites/locations/racks, devices with their AVD serials (plus clients as role `server`), interfaces (LAGs with members, switchport mode and VLANs, VRFs, management `mgmt_only`), VLANs, VRFs (with RD), prefixes (pools, SVI subnets, P2P /31s), IPs (anycast for SVI virtual IPs and shared VTEPs, `loopback` role for Loopback0, management IP as primary IPv4) and cables, tagged `avdlab` and `avdlab:<slug>`; it is idempotent. The site is the AVD `dc_name` (else the fabric name), so two labs with the same name share a site — give each lab its own `dc_name` or NetBox.

ACT (advanced, target act only)

The ACT topology (prepared / not prepared, node table with role, node type, version, IP), *Download topology YAML* to upload in act.arista.com, client setup scripts with an "apply all from the tools server" command, and next steps (add the tools server as a host of kind act, Bootstrap with `act-tools`, then Deploy and Validate as usual).

Git & VS Code (advanced)

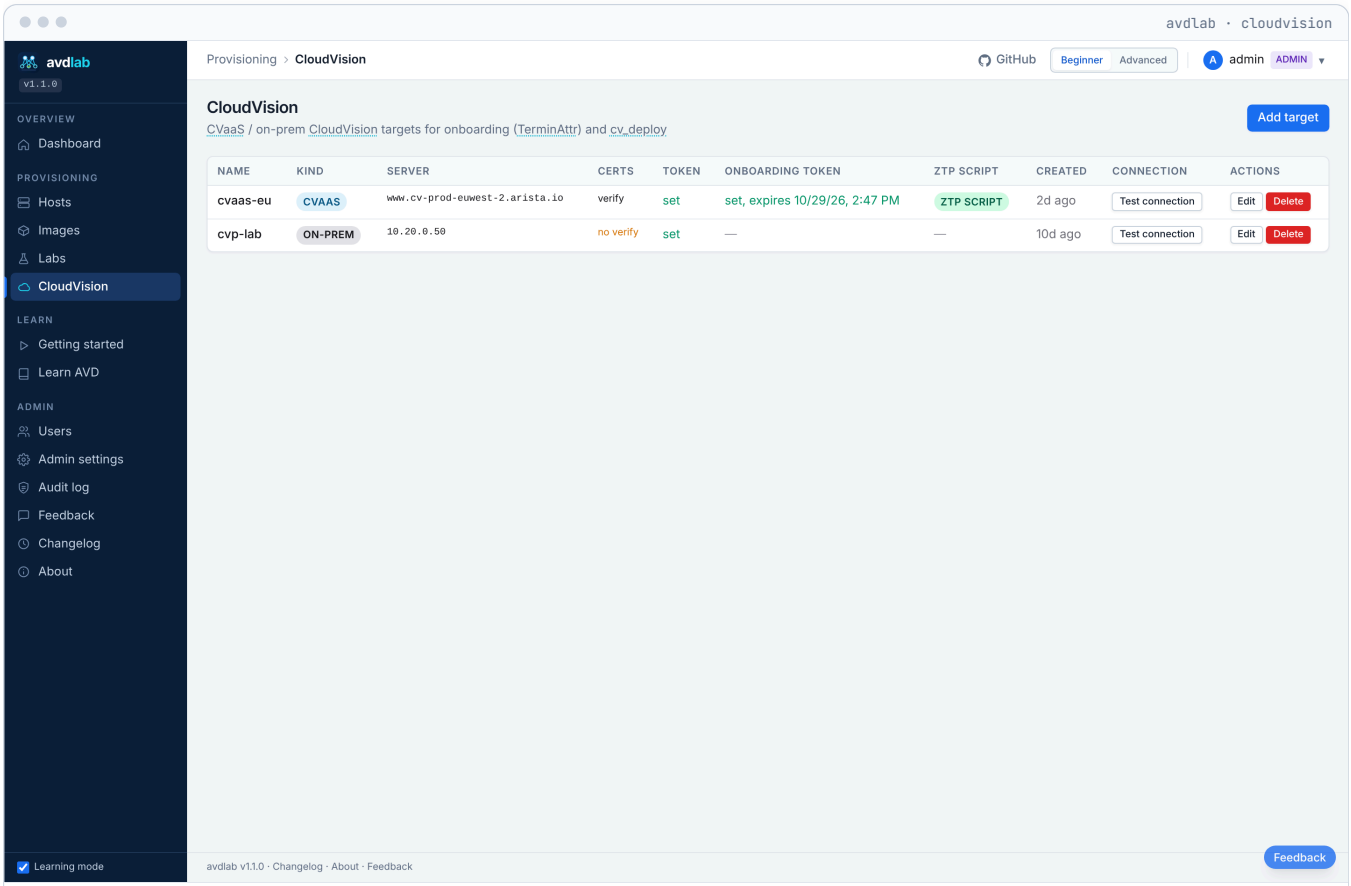
Git remote: URL (no credentials in the URL; local paths and `file://` need `AVDLAB_GIT_ALLOW_LOCAL`), branch (validated on save and again on every push/pull — *invalid git branch name* otherwise), authentication (none / access token over **HTTPS only** — write-only, sent as `x-access-token` for GitHub or `oauth2` for GitLab; refused for `http://` and `git://` / SSH key secret that you own with purpose `git`); *Push* (optional force; job `lab.git_push`), *Pull* (`git pull --ff-only`, then re-validated; job `lab.git_pull`), ahead/behind from the last fetch, last sync. The token or key never appears in the job log (`***`). **Open in VS Code**: a Remote-SSH `vscode://` link to `/opt/avdlab/labs/<slug>` on the host, the topology path for the containerlab extension, a devcontainer clone link, the SSH command; *sync* (job `lab.sync`) copies the lab to the host without deploying.

Jobs

All jobs of this lab with status, type, duration and log links (same as the global Jobs page, filtered).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.10 CloudVision page (targets)



CloudVision targets: CVaaS regions or an on-prem CVP, with a service-account token.

Table: name, kind, server, certs, token, onboarding token, ZTP script, created, *Test connection* (inline REST inventory call, 15 s → ok / devices seen / error), Edit, Delete. **Add target**: name; kind *CVaaS* (server = the regional API endpoint of your tenant, e.g. `www.cv-prod-….arista.io`, or *custom…*) or *on-prem* (CVP cluster address); verify TLS certificates; **service-account token** (for cv_deploy and the inventory checks), **onboarding token** with optional expiry (for TerminAttr), optional **ZTP bootstrap script** — all write-only; each can be removed. Generate the service-account token in CloudVision under Settings → Access Control → Service Accounts.

5.11 NetBox page (targets)

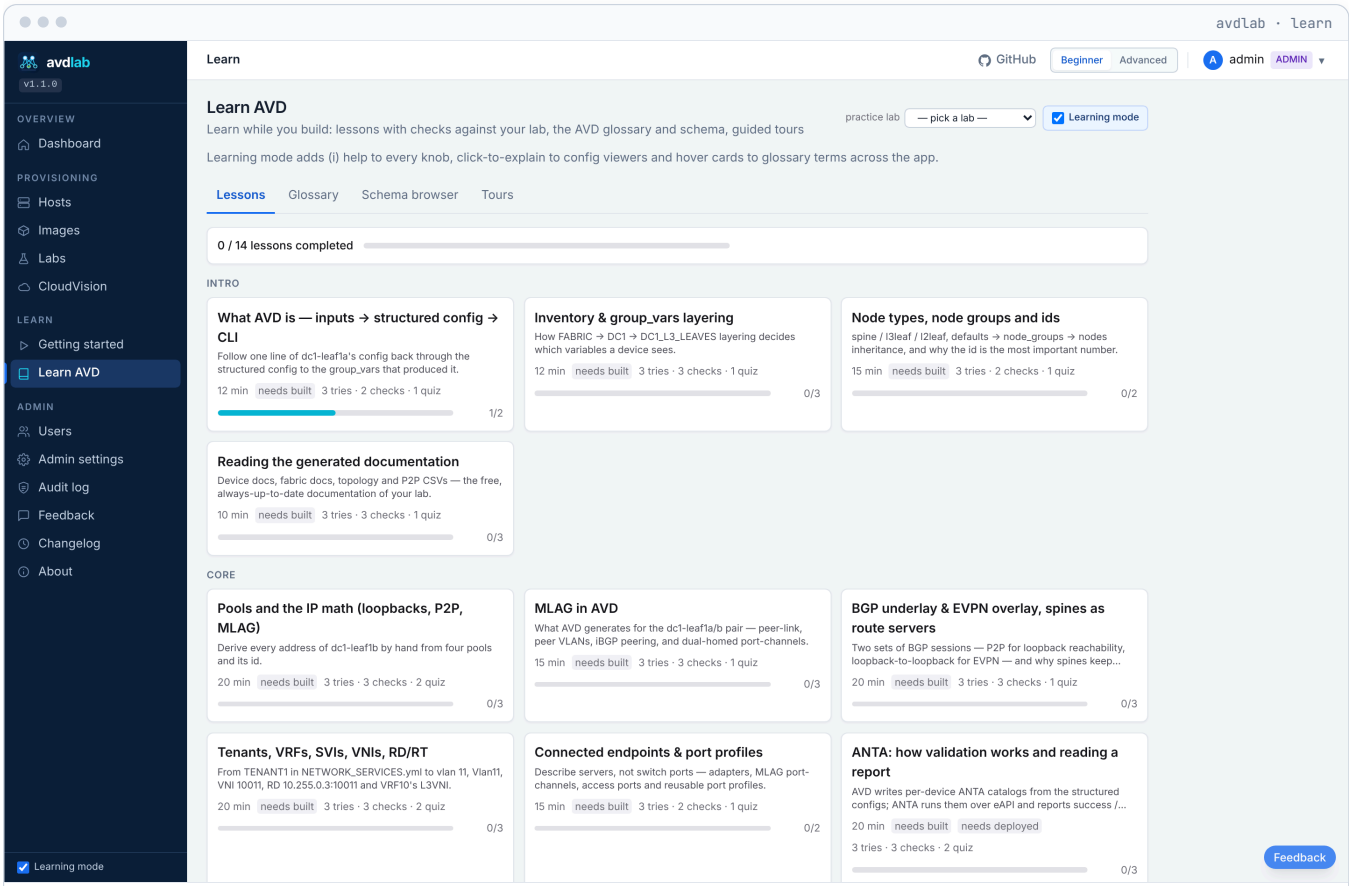
Table: name, URL, certs, token, labs, created, *Test connection* (pynetbox `status()`, 15 s → ok / version / error), Edit, Delete. **Add target**: name, base URL (`http(s)://host[:port]`), verify TLS certificates, API token (write-only). Both token formats work: legacy 40-hex tokens and NetBox 4.5+ `nbt_<id>.<secret>` tokens (sent as `Bearer`); a pasted `Token` / `Bearer` prefix is stripped. The compose profile `netbox` starts a local NetBox (chapter 3).

5.12 Devcontainer (advanced)

Downloads a zip for VS Code Dev Containers: mode **local-clab** (containerlab inside the devcontainer, Docker-in-Docker; needs a strong x86_64 machine or a Codespace) or **remote-host** (tooling only; labs run on the SSH lab host); optional AVD version (blank = server default). The page lists the bundle contents.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.13 Learn

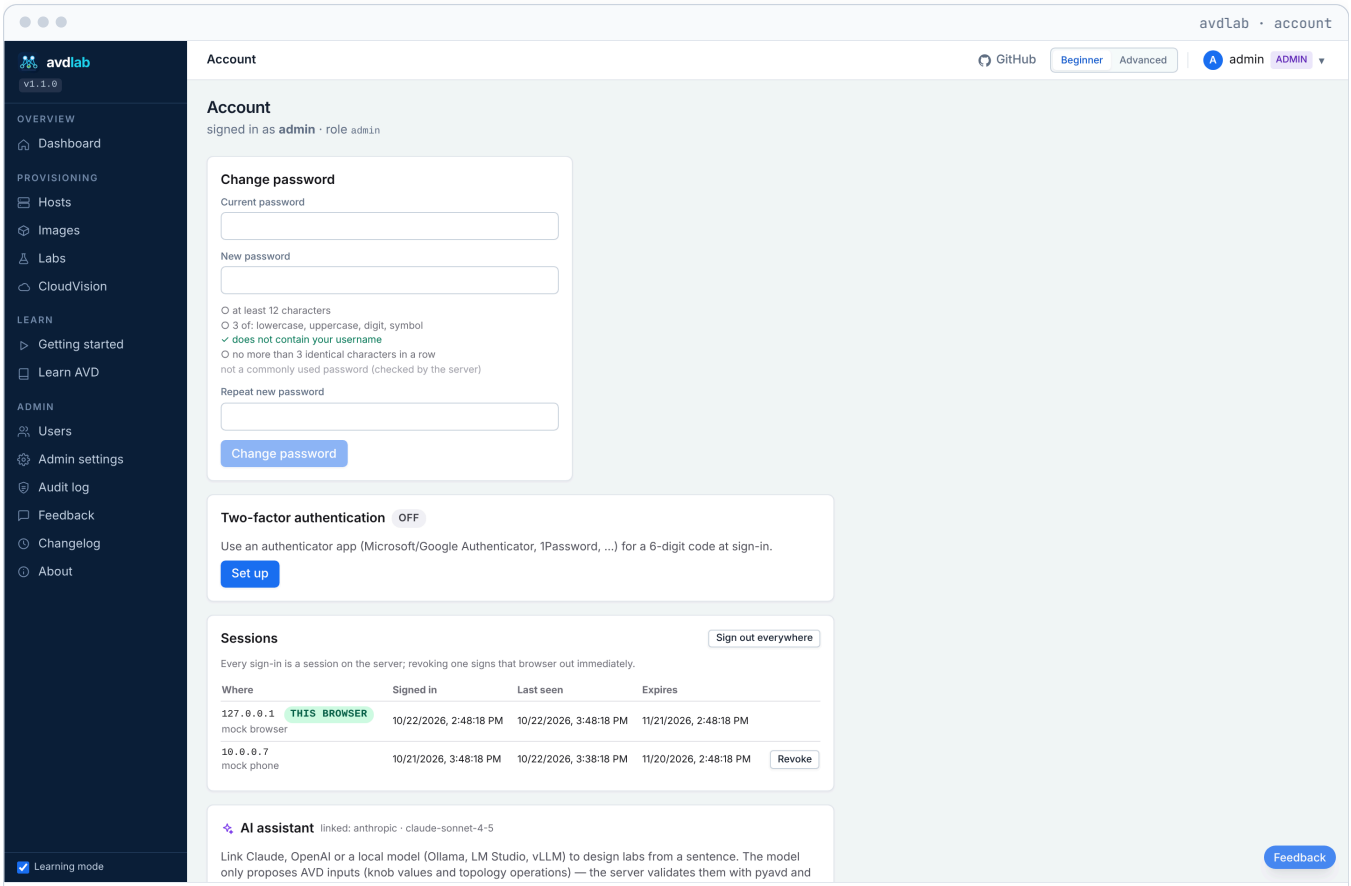


Learn AVD: guided curriculum with the lab as the exercise.

`/learn` has four sections: **Lessons** (14 lessons, from "What is AVD" to segmentation; each mixes text with *try* actions the UI performs, *checks* evaluated against your practice lab on the server, and quizzes; progress is stored per user), **Glossary** (54 terms, the same cards that appear on hover), **Schema browser** (search and look up keys of the AVD 6.4 `eos_designs` and `eos_cli_config_gen` schemas) and **Tours** (guided tours per page). The learning-mode switch adds (i) help to every knob, click-to-explain to config viewers and hover cards to glossary terms across the app.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.14 Account

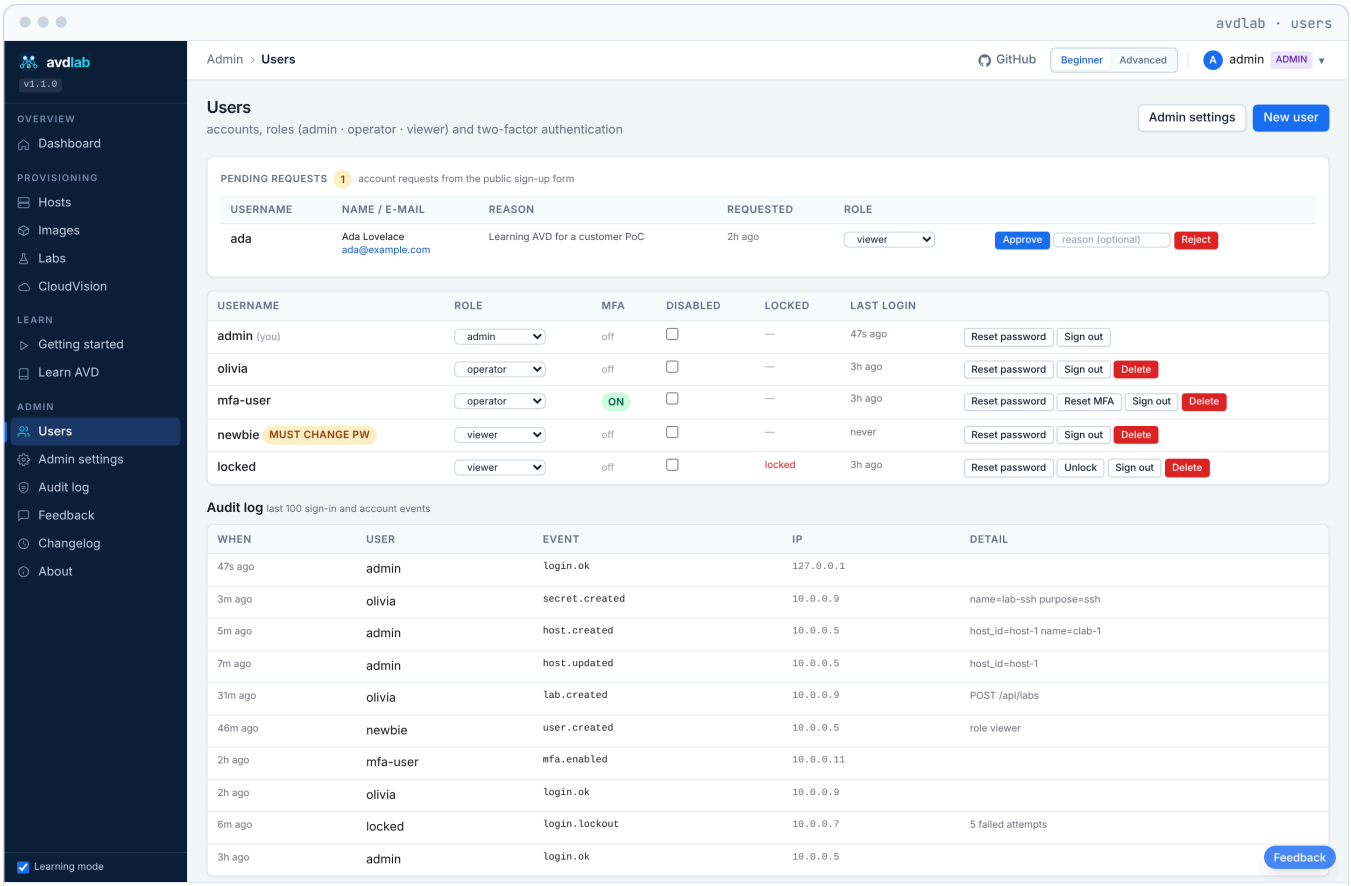


Account: password, TOTP MFA, and (1.1.0) the list of your sessions with Sign out everywhere.

Signed in as / role. **Two-factor required** banner (1.1.0): when your role is in `mfa_required_roles` (admins by default) and you have no authenticator yet, an amber banner says *Two-factor authentication is required for your role* and the rest of the app answers 403 until you set it up; a toast with the same text appears when you were redirected here. **Change password** with a live policy checklist (length, character classes, no username, not common — checked by the server); success signs out your other sessions. **Two-factor authentication**: *Set up* shows a QR code and the key for manual entry → enter the 6-digit code → *Enable* → 10 **recovery codes**, shown once (*Copy codes*, *Download*); *Disable MFA* needs your password and a current code; *Regenerate recovery codes* needs your password (regenerating invalidates the old codes). **Sessions** (1.1.0): a table of everywhere you are signed in — the current one marked, IP address, browser (user agent), created, last seen, expires (12 hours after sign-in) — with *Revoke* per row and **Sign out everywhere** (revokes all sessions including this one). **AI assistant**: link your own model (provider, base URL, model, API key — write-only), *Test connection* lists the provider's models, *Remove*; a base URL on a private or loopback address is refused unless an admin allowed it (chapter 6); viewers cannot use the designer.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.15 Users (admin)

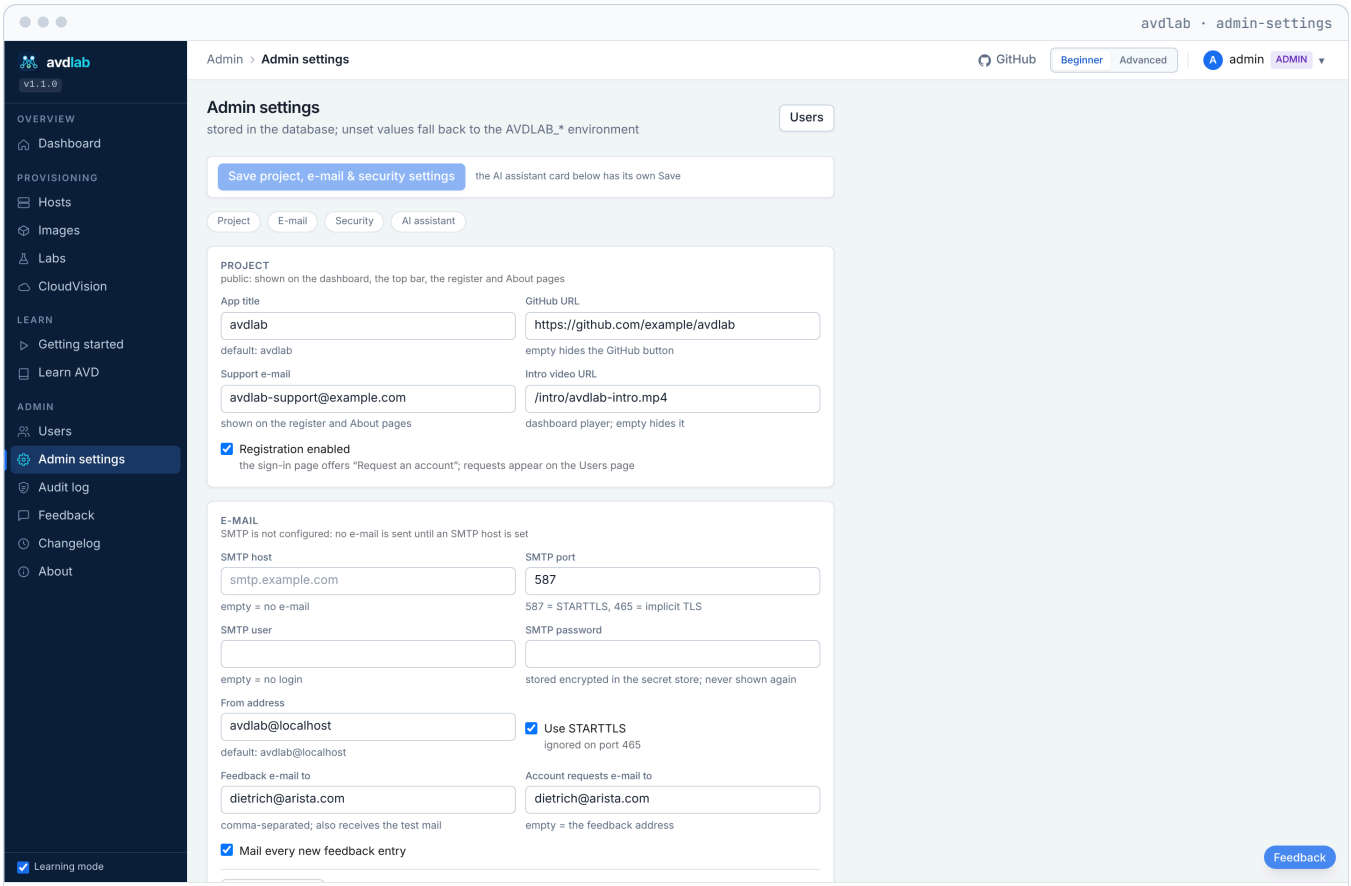


Users (admin): roles, pending registrations, MFA state.

Pending requests (from the public form): username, name / e-mail, reason, requested; *Approve* with a role (creates the account with a 20-character temporary password, e-mailed to the applicant when SMTP is configured and always shown **once** to the admin) or *Reject* with an optional reason. **Users**: *New user* (username, role, optional password — empty generates a temporary one), and per user role, disabled, MFA, locked, last login, *must change pw* (the API rows also carry `mfa_required`); actions *Reset password*, *Reset MFA*, *Unlock*, *Sign out* (1.1.0: revokes every session of that user without disabling the account — the incident-response instrument), *Delete* (not yourself, not the last admin). Disabling a user, resetting a password or MFA signs that user out everywhere (1.1.0). **Audit log**: the last 100 sign-in and account events (when, user, event, IP, detail); the full trail with actor, filters and export is the separate Audit log page (5.16a). The header links to Admin settings.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.16 Admin settings (admin)



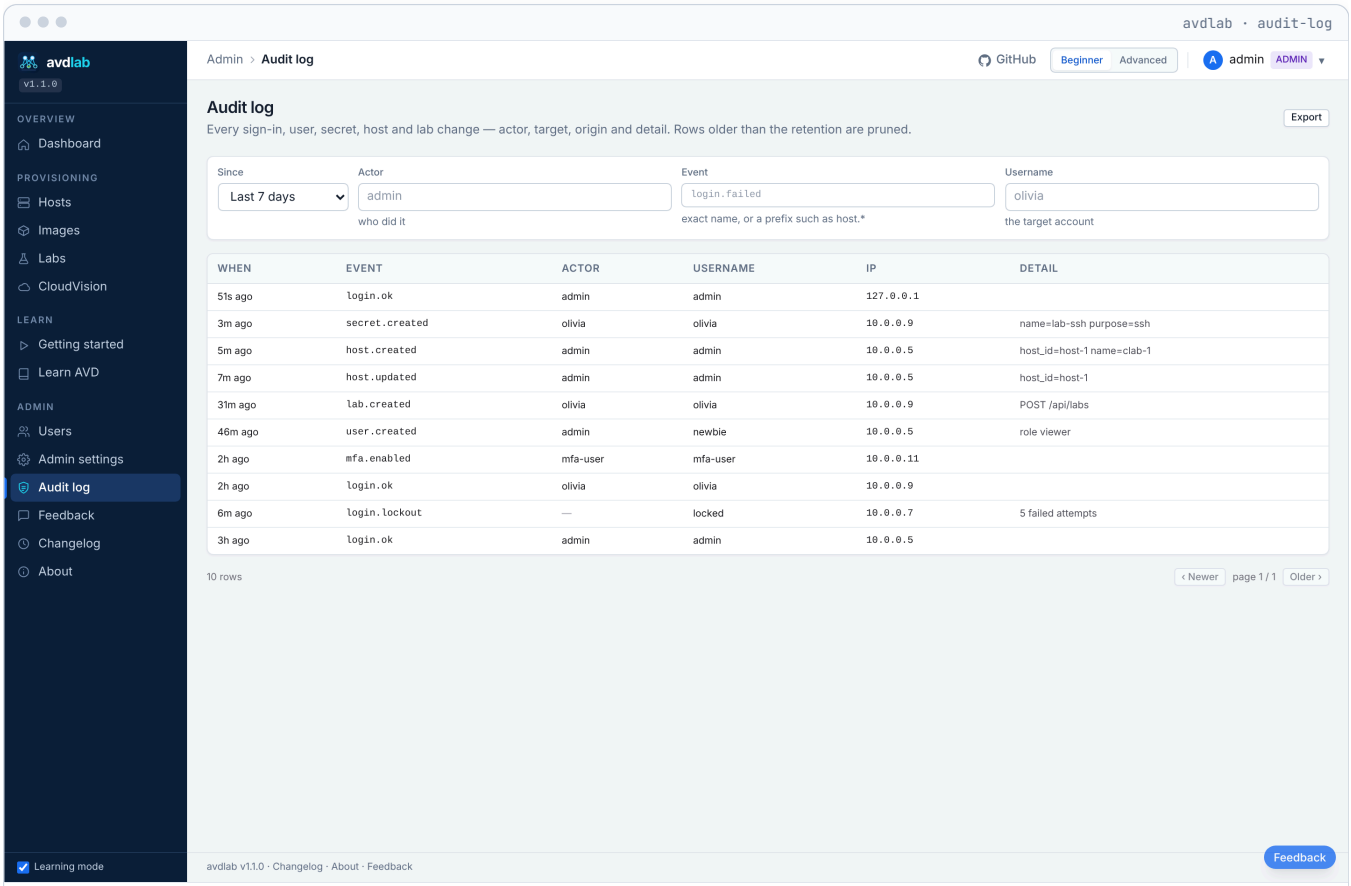
Admin settings: project, e-mail, the 1.1.0 Security section (MFA policy, private AI endpoints) and the shared AI link.

A sticky Save/Reset bar at the top. Sections: **Project** (app title, GitHub URL, support e-mail, intro video URL, registration enabled), **E-mail** (SMTP host, port — 587 STARTTLS / 465 implicit TLS, user, password (encrypted, never returned), from address, use STARTTLS, feedback e-mail to, account-requests e-mail to, mail every new feedback entry, *Send test mail* which shows the SMTP error if it fails — since 1.1.0 a login over a plaintext connection is refused with a clear message unless the installer set `AVDLAB_SMTP_ALLOW_INSECURE`) and **AI assistant** (its own Save: shared default provider, base URL, model, API key, *Allow users without their own link to use this model*, *Test connection*). Stored values win over the `AVDLAB_*` defaults; a reset returns a key to its env default.

The **Security** section (1.1.0) holds two policy keys: **Roles that must enrol an authenticator** (checkboxes per role, `mfa_required_roles`, default admin only; none checked opts out) and **Allow private AI endpoints** (`ai_allow_private_urls`, default off: an AI base URL on a loopback / private / link-local address is refused as SSRF; switch it on for a local LLM on the LAN). Like every admin setting the database value wins over `AVDLAB_MFA_REQUIRED_ROLES` / `AVDLAB_AI_ALLOW_PRIVATE_URLS`.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

5.16a Audit log (admin, 1.1.0)



Audit log (1.1.0): who did what, filters by actor / event / user, JSON-lines export.

`/admin/audit` (sidebar *Admin › Audit log*) shows who did what, when and from where: every sign-in and lockout, logout everywhere and session revocation, MFA change, account request (including duplicates), user administration, secret creation / deletion / binding, host and lab creation / change / deletion, lab file writes and git configuration, feedback deletion, job kills and AI applies. Columns: time, **actor** (who did it; empty for anonymous events such as a failed login), **target** username, event, IP, detail. Filters: *since* (1 h / 24 h / 7 d / 30 d / all), *actor*, *event* (exact, or a prefix like `host.*`), *username*; 100 rows per page with newer / older paging. **Export** downloads the current selection as JSON lines (`GET /api/audit/export`, newest first, up to 100 000 rows) — do this before rows age out of the retention (`AVDLAB_AUDIT_RETENTION_DAYS`, 365 days by default). Operators and viewers see an explanatory empty state and no sidebar entry.

5.17 Feedback

The **Feedback** button (every page) opens a form: kind (bug / suggestion / question), title, description, page, *Include diagnostics* (version, browser, recent failed jobs with secrets masked, host capabilities — previewable). Sending stores the entry, writes `/data/feedback/<id>.md`, optionally opens a GitHub issue (`AVDLAB_FEEDBACK_GITHUB_REPO` + secret `github_token`) and mails `feedback_mail_to` when SMTP is configured. The **E-mail** button opens your mail client with the feedback address, a subject and the page context prefilled — it works without SMTP. The `/feedback` page lists entries (kind, title, page, received, status new/triaged/done, GitHub issue link, delete).

5.18 Changelog and About

Changelog renders `CHANGELOG.md` per version (Added / Changed / Fixed / Security) with the running version marked, plus the **Updates** panel (current/latest, *Check now*, *Update now* — admins only since 1.1.0 — when self-update is possible, otherwise the reason). In image mode a **release image** row in amber shows `image_error` when the release is not digest-pinned (`<image>@sha256:…`); such a release cannot be applied until the publisher adds the digest. **About**: version, git sha, build date, Python and platform (from the authenticated `GET /api/meta/build`), pinned AVD/ANTA versions, author, GitHub link, copyright and licence (all rights reserved), third-party notices.

5.19 Jobs (advanced)

`/jobs` lists the latest jobs of all labs and hosts (type, status, lab/host, created, duration); `/jobs/<id>` shows parameters, result, error and the live log with a *Kill* button while it runs or waits. A job started while 4 others run stays **queued** (its log says `job queued: 4 of 4 job slots busy, n waiting`)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

and starts when a slot frees up; *Kill* works on a queued job too. Secrets in logs appear as `***`; a log that reached 20 MiB ends with `[log truncated]` while the job itself continues. Job directories on disk are kept for 90 days.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 6

AI designer

Design with AI and Start from a sentence: tools, dry run, repair, providers and privacy.

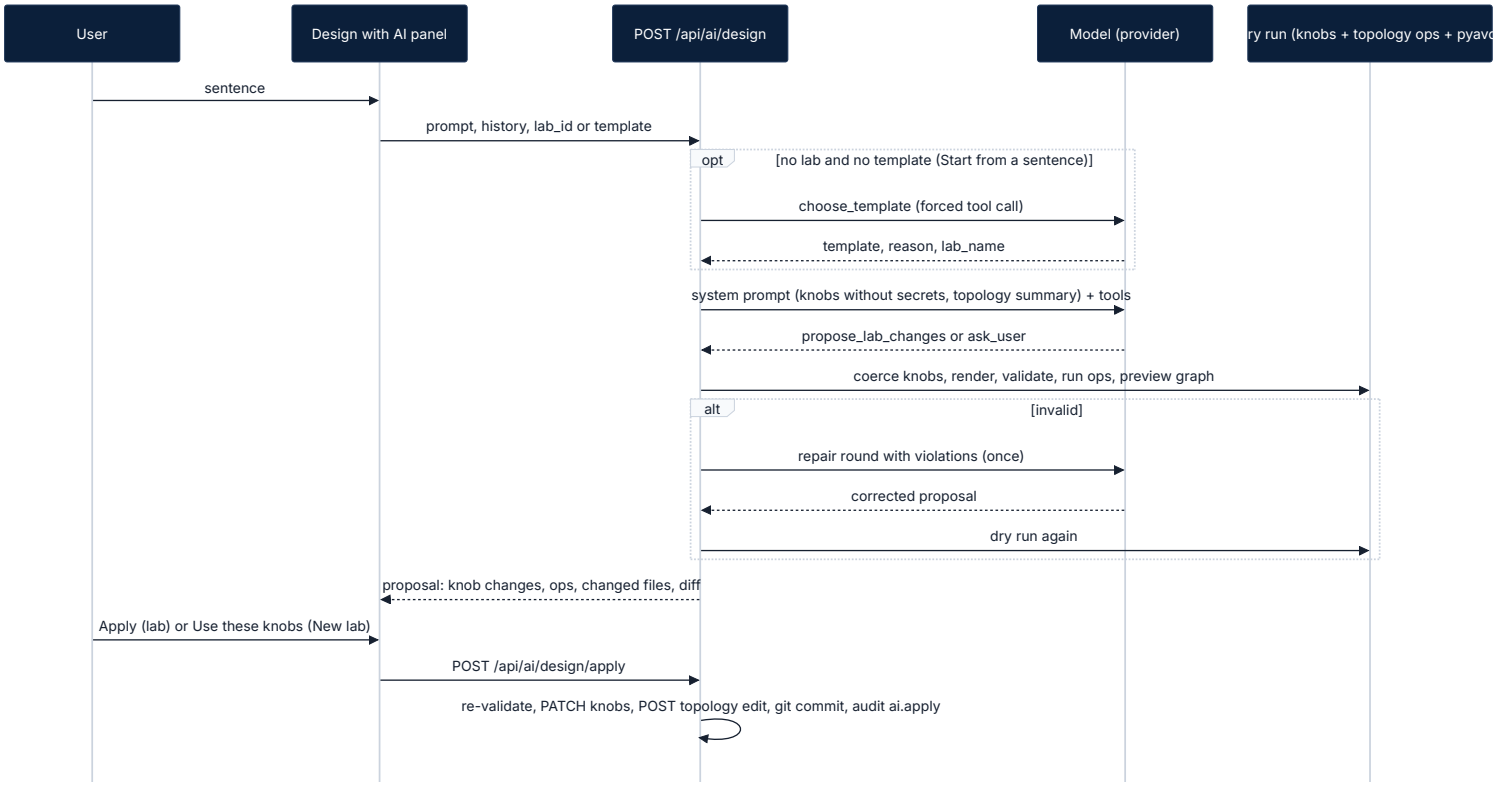
The AI designer turns a sentence — *"3 spines, 4 leaf pairs with MLAG, 2 servers per pair, tenant ACME with VLANs 110/120, CloudVision cvaas"* — into a **proposal of validated AVD inputs** that you review before anything changes. It is optional, bring-your-own-model, and follows the project's one rule: **the AVD inputs stay the source of truth and the model never writes YAML**.

Where it appears:

ENTRY POINT	MODE	WHAT *APPLY* DOES
Design with AI button on the lab **Topology** and **Settings** tabs	lab mode: the model sees the lab's knobs and topology and may propose knob changes **and** topology operations	*Apply* changes the lab (git commits, audit event)
Design with AI in step 2 of **New lab**	template mode: the model sees the chosen template's knobs	*Use these knobs* fills the wizard's knob form; nothing is created until *Create lab*
Start from a sentence card in step 1 of **New lab**	from scratch: the model first picks the template, then proposes knobs	the wizard jumps to step 2 with the knobs filled, a suggested lab name and a note explaining the template choice

6.1 How a design call works

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



AI designer sequence

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

1. **Resolve the model.** The user's own link (Account → AI assistant) wins; otherwise the admin's shared default is used if *Allow users without their own link to use this model* is on. No model → `409 {code: "no_ai_link"}` and the panel offers *Link a model on your Account page*.
2. **Rate limit.** 30 design calls per user per hour (`AVDLAB_AI_RATE_LIMIT`), counted in memory → `429`. *Start from a sentence* uses two model calls and counts twice.
3. **Choose a template** (from-scratch only). One forced `choose_template` tool call over the template catalogue (every template except `custom`: name, title, description, tags, node count, generator, notes, first knob keys); the tool description tells the model to prefer `single-dc-l3ls` for any generic spine/leaf or EVPN/VXLAN request. The answer carries the template, a one-sentence reason and a slug-like lab name. If the model does not call the tool with a known template, avdlab falls back to `single-dc-l3ls` (lab name `ai-lab`).
4. **Build the system prompt.** Fixed rules and AVD vocabulary, the template (name, title, generator, description, notes), every knob with type, range, help, the first pitfalls from its `learn` block and its **current value** — except secret knobs, which are listed as "secret: never set, never shown" — and, in lab mode, the current lab (name, state, target), the topology operations available, and a **topology summary** (hostnames → node type, connected endpoints with their switch ports, up to 16 free ports per leaf). Knob values longer than 4000 characters and the summary beyond 6000 characters are truncated.
5. **Call the model** with tool choice "must call a tool" and two tools (below). The conversation history (up to 40 turns) is cleaned: only user/assistant turns, consecutive same-role turns merged, each turn capped at 8000 characters.
6. **Dry run** the proposal through the same code paths as the Settings form and the topology editor: sanitise (secret or masked knobs dropped with a note, op shapes checked, at most 50 ops), coerce knobs, render the inputs, validate with pyavd, run the topology ops in order on a copy, rebuild the topology preview. The result: `valid`, `violations`, `changed_files`, unified `diff`, preview `graph`, knob changes (old → new), warnings.
7. **Repair round.** If the dry run fails, the violations (up to 40) are sent back to the model once, asking for a corrected complete proposal. The panel marks a repaired proposal ("the first proposal failed validation; the model fixed it"). A second failure is returned as an invalid proposal with its violations — never applied.
8. **Review.** The panel shows the summary, *Assumptions* (values the model chose itself, plus server notes), *Knob changes* (old → new), *Topology operations*, the changed files and the diff. You can *Refine* (send another message; the history is kept), *Discard*, or apply.
9. **Apply** (`POST /api/ai/design/apply`, lab mode). The proposal is sanitised and **dry-run again** (all or nothing: never half-applied); refused with 409 while `lab.build`, `lab.deploy` or `lab.destroy` runs for the lab. Knob changes go through `PATCH /api/labs/{id}`, topology ops through `POST /api/topology/{id}/edit` — so every change is a git commit of the lab exactly as if you had made it by hand — and an `ai.apply` event is written to the audit log. A built lab goes back to `draft`: press *Build*.

6.2 The tools the model may call

TOOL	ARGUMENTS	NOTES
`propose_lab_changes`	`knobs` (partial knob values — JSON schema generated from the template descriptor: types, min/max, choices, help), `topology_ops` (lab mode only, ≤ 50), `summary`, `assumptions`	list/YAML knobs (`tenants`, `servers`, `dns_servers`) replace the whole value, so the model must include entries it wants to keep
`ask_user`	`question`	one clarifying question when an essential detail is missing and there is no sensible default
`choose_template`	`template` (enum of template names), `reason`, `lab_name`	only in the from-scratch step

Topology operations (the ops of the topology editor, same argument shapes as `POST /api/topology/{id}/edit`):

OP	ARGUMENTS	AVAILABLE
`set_uplink`	node, index, switch, switch_port	all templates
`add_endpoint`	name, type (default `servers`), adapters	all templates
`update_endpoint`	name, adapters	all templates
`remove_endpoint`	name	all templates
`add_leaf_pair` / `remove_last_leaf_pair`	—	l3ls templates (`single-dc-l3ls`)
`set_l2leaf_per_pair`	n	l3ls templates
`set_spine_count`	n	l3ls templates

For a custom lab (generator `none`) only topology ops can change it. In template mode (New lab) count ops are folded into the knobs.

Rules the system prompt gives the model: never write YAML, CLI or files; nothing changes until the user presses Apply; ask at most one question, otherwise propose and list every own choice in `assumptions`; only include knobs it changes; never set secret knobs (tell the user to set them in Settings); answer in the user's language. It also carries AVD vocabulary (node types, MLAG pairs sharing a VTEP, tenants → VRFs → SVIs with `ip_address_virtual` and `enabled: true`, L2VNI = `mac_vrf_vni_base` + VLAN, how a dual-homed server is one adapter on both MLAG leafs with `port_channel: {mode: active}`, CloudVision modes and the need for public DNS with CVaaS).

Local models without proper tool calling sometimes answer with the JSON as text; the server accepts a JSON object in the text that looks like a `propose_lab_changes` / `ask_user` call. If the model answers only in prose, the panel shows the text.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

6.3 Providers

PROVIDER	BASE URL	MODEL (EXAMPLE)	KEY
Anthropic (Claude)	empty (`https://api.anthropic.com`)	`claude-sonnet-4-5`	Anthropic API key, header `x-api-key` , `anthropic-version: 2023-06-01`
OpenAI	empty (`https://api.openai.com/v1`)	`gpt-4o`	OpenAI API key (Bearer)
Ollama	`http://<host>:11434/v1` (private address: needs `ai_allow_private_urls` , see 6.4)	`llama3.1` , `qwen2.5`	none
LM Studio	`http://<host>:1234/v1` (same)	the loaded model	none
vLLM	`http://<host>:8000/v1` (same)	the served model	optional
Azure OpenAI	`https://<resource>.openai.azure.com/openai/deployments/<deployment>`	the deployment's model	Azure key (header `api-key` ; `api-version` query parameter added)

Two client implementations, plain `httpx` , no SDKs: `anthropic` (Messages API, `POST {base}/v1/messages`) and `openai` (Chat Completions, `POST {base}/chat/completions` with `tools` / `tool_choice` , temperature 0.2) which covers every OpenAI-compatible server. Responses are limited to 4096 tokens; each request times out after `AVDLAB_AI_TIMEOUT` seconds (180). **Test connection** lists the provider's models (`GET /v1/models` or `/models` ; Azure: a one-token completion) and says whether the configured model is in the list; the model field then offers the listed models.

Private base URLs (1.1.0). The API is the one making these requests, from inside your network, so a base URL is a potential server-side request forgery vector. A base URL — per-user link or shared default — whose host is or resolves to a loopback, link-local, RFC 1918, CGNAT (`100.64/10`) or cloud-metadata address (`169.254.169.254`), or that does not resolve at all, is refused with `422 base_url resolves to a private address (…): not allowed unless an admin enables ai_allow_private_urls` . Public SaaS endpoints need nothing. For a model server on your LAN or on the orchestrator host itself (Ollama, LM Studio, vLLM) an admin turns the override on once: `PUT /api/admin/settings {"ai_allow_private_urls": true}` (or `AVDLAB_AI_ALLOW_PRIVATE_URLS=true` in `.env` ; the database value wins). The check runs when the link is saved and tested, not on every design call.

Ollama example (on the lab host or any machine the orchestrator can reach):

```
ollama pull qwen2.5:14b          # a model with tool calling; small models propose less reliably
OLLAMA_HOST=0.0.0.0 ollama serve # then Account → AI assistant → OpenAI-compatible, http://<host>:11434/v1, qwen2.5:14b
```

(an admin must have enabled `ai_allow_private_urls` first, otherwise saving the link answers 422).

6.4 Configuration

WHERE	WHO	WHAT
Account → AI assistant	operators, admins	own link: provider, base URL (no private addresses unless allowed), model, API key (write-only, encrypted in the secret store with purpose `ai` ; empty field keeps the stored key, *Remove* deletes link and key), *Test connection* with the unsaved form values
Admin settings → AI assistant	admins	shared default provider / base URL / model / API key (reserved secret `ai_default_api_key`), *Allow users without their own link to use this model*, *Test connection*
`PUT /api/admin/settings`	admins	`ai_allow_private_urls` (1.1.0; no form field yet): allow private / loopback base URLs for every link
`.env`	installer	`AVDLAB_AI_DEFAULT_PROVIDER` , `AVDLAB_AI_DEFAULT_BASE_URL` , `AVDLAB_AI_DEFAULT_MODEL` , `AVDLAB_AI_DEFAULT_API_KEY` , `AVDLAB_AI_SHARED_ALLOWED` (false), `AVDLAB_AI_RATE_LIMIT` (30), `AVDLAB_AI_TIMEOUT` (180), `AVDLAB_AI_ALLOW_PRIVATE_URLS` (false) — defaults only, the admin page / API wins

Roles. Design calls are `POST` requests, so viewers are refused (403); the Account card says so. Linking a model is also a write. Operators and admins can design and apply.

6.5 Privacy

Each design call sends to the configured provider:

SENT	NOT SENT
the template descriptor: knob keys, types, ranges, help, pitfalls	secret knobs (device password and any other `secret` -type knob)
the `act` knob with the password field masked (`********`) when the lab targets ACT	the ACT node password
the lab's current knob values without secret knobs	onboarding tokens, service-account tokens, NetBox tokens
a topology summary: hostnames, node types, endpoints, free switch ports	SSH keys, host addresses and credentials
lab name, state, target (lab mode)	job logs, running configs, ANTA results
your chat messages and the conversation history	the API key itself in any prompt (it is only a request header)

API keys are only ever put in request headers and every provider error passes through a masking function, so a key echoed by a provider or proxy never reaches a log line or an HTTP response (`502 {message}` carries the masked provider error). When lab data must not leave your network, use a

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

local model (Ollama, LM Studio, vLLM; an admin enables `ai_allow_private_urls`). When an admin enables the shared model, every user's lab data is sent to that provider — the admin form says so.

ACT password. `act.password` is an encrypted secret knob since 1.1.0 and is masked everywhere in the API; the designer applies the same mask (`mask_act_knob`) before building its prompt, so the model sees `********`. A proposal that echoes the mask keeps the stored password; a proposal that sets a new one is dropped like any other secret.

6.6 Limits and behaviour to expect

LIMIT	VALUE
prompt length	8000 characters
history	40 turns, 20000 characters each on input, 8000 used per turn
ops per proposal	50
design calls	30 per user per hour (in memory; restarts reset it)
repair rounds	1
provider timeout	`AVDLAB_AI_TIMEOUT` (180 s)
response size	4096 tokens
typical duration	from a sentence: two model calls plus a pyavd dry run — "up to a minute"

Errors you may see:

RESPONSE	MEANING
`409 no_ai_link`	no own link and no shared model allowed
`429 AI design rate limit reached`	wait or raise `AVDLAB_AI_RATE_LIMIT`
`502 {message}`	the provider failed (bad key, unknown model, timeout, HTTP error); the message is the provider's, masked. Since 0.10.1 the panel shows this message instead of a bare "HTTP 502".
`422 invalid proposal` / `n violation(s)` on apply	the proposal no longer validates against the current lab (e.g. it changed since) — refine and try again
`422 base_url resolves to a private address …` on saving / testing a link	the model server is on a LAN, loopback or metadata address (1.1.0) — ask an admin to enable `ai_allow_private_urls`, or use a public endpoint
`403 mfa_required`	your role must have an authenticator; set it up on the Account page first
`409 lab.build is running for this lab` on apply	wait for the job

Small local models propose less reliably (missing `enabled: true` on SVIs, wrong adapter shapes); the repair round fixes many such cases, and nothing invalid is ever applied.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 7

API reference

REST and WebSocket routes per router, roles, and all job types.

Everything the SPA does goes through a JSON REST API under `/api` plus a few WebSockets; you can script avdlab with the same calls. This chapter is derived from the routers registered in `backend/avdlab/main.py` (182 routes in 1.1.0: 179 HTTP operations and 3 WebSockets) and their docstrings. FastAPI also serves its generated OpenAPI description at `/docs` and `/openapi.json` (not behind a session).

7.1 Conventions

TOPIC	CONVENTION
Base path	`/api` — the tables below omit it
Authentication	session cookie from `POST /api/auth/login` (+ `POST /api/auth/mfa/verify` when MFA is on). The cookie carries a server-side session id (1.1.0): it is valid for 7 days of activity but at most 12 h after sign-in (`AVDLAB_SESSION_ABSOLUTE_MAX_AGE`), and `POST /auth/logout` , a revocation or an admin action can end it earlier. A script keeps the cookie jar: `curl -c jar -b jar -H 'content-type: application/json' -d '{"username":"admin","password":"…"}' https://host/api/auth/login`
Anonymous	`401 {"detail": "not authenticated"}` except the public routes
Role refused	`403 {"detail": "role viewer may not POST /api/labs"}` (RoleGuard)
MFA required (1.1.0)	`403 {"message": "mfa_required", "detail": "mfa_required"}` on everything but `/auth/*` and `/meta*` while a user whose role is in `mfa_required_roles` has no authenticator; WebSockets close `4403`
Origin refused (1.1.0)	`403 {"detail": "origin not allowed"}` when a mutating call or WebSocket upgrade carries an `Origin` that is neither the request host nor a CORS origin; requests without `Origin` (curl) are unaffected
Secret ownership (1.1.0)	`403 {"message": …, "violations": [{"path", "message"}]}` when a secret is bound that the caller does not own or whose purpose does not fit; `404` for another user's secret on `GET` / `DELETE /secrets/{id}`
Rate limited (1.1.0)	`429 {"detail": "too many attempts: try again later"}` + `Retry-After` on `/auth/login` and `/auth/mfa/verify` (20 per minute per IP); `429` also on `/register` (5 per hour per IP) and `/ai/design` (30 per user per hour)
Validation errors	`422 {"detail": {"message": str, "violations": [{"host"?, "path"?, "message"}]}}` — knob coercion, pyavd schema, YAML syntax, topology ops (`op_index` , `hint`), `jump_host` format, git URL / branch, secret `purpose` , control characters in a secret knob
Too large (1.1.0)	`413 {"detail": {"message", "size", "max_bytes"}}` on `/images/upload` above `AVDLAB_UPLOAD_MAX_BYTES` ; Caddy answers its own `413` above 16 MB elsewhere
Conflicts	`409` with a message, often `{"message": "lab.deploy is already running for this lab", "job_id": …}` ; `409 {"message", "file"}` on an upload that would overwrite a file; `409 {"message", "reason"}` when a self-update is impossible
Jobs	long actions answer `202 {"job_id": "<32 hex>"}` ; follow with `GET /jobs/{id}` or `WS /jobs/{id}/stream` . A job beyond the concurrency limit stays `queued` until a slot frees up
Inline budgets	status 20 s, running config 25 s, client exec 20 s, CV/NetBox test 15 s, ZTP status 15 s
Secrets	write-only everywhere: request bodies may carry `token` , `api_key` , `value` , `smtp_password` ; responses only carry `has_*` / `configured` flags. A masked secret knob (device password, `act.password`) comes back as `********` and is kept when sent back unchanged; a value with a newline, CR or NUL is `422`
Logs	job logs mask every registered secret with `***` ; a log that hit `AVDLAB_JOB_LOG_MAX_BYTES` ends with `[log truncated]`
Ids	32-character hex strings

Role column. `public` = no session needed; `viewer` = any signed-in user (read); `viewer (self)` = any signed-in user, acting on their own account; `operator` = operator or admin; `admin` = admin only. The rules are those of `core/security.py` `role_denial()` (chapter 2 §(f)); `admin` also covers `/audit*` , and the update routes check the role in the router.

7.2 WebSockets

PATH	ROLE	FRAMES
`/jobs/{job_id}/stream`	viewer	server → client JSON `{"line": "[HH:MM:SS] …"}` for the existing log, then live lines, then `{"done": true}`
`/hosts/{host_id}/terminal?cols=&rows=`	operator	`$SHELL -l` on a PTY over SSH
`/labs/{lab_id}/nodes/{node}/terminal?shell=cli|bash&cols=&rows=`	operator	`docker exec -it <node> Cli` or `bash` (linux clients: bash or sh; ACT nodes: SSH from the tools server)

Terminal frames are binary in both directions (client → stdin, stdout → client); a JSON text frame `{"resize": {"cols": n, "rows": n}}` resizes the PTY; any other text frame is written to stdin. Close codes: `4401` not authenticated, revoked or expired session, `4403` role not allowed / `mfa_required`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

/ `origin not allowed`. Session open/close is logged, never the content; a terminal error shows a generic message in the browser and the detail goes to the server log (1.1.0).

7.3 Routes per router

Auth — `api/auth.py`

METHOD	PATH	ROLE	PURPOSE
POST	`/auth/login`	public	sign in `{username, password}` → user (with `mfa_required`), or `{mfa_required: true}` for the second step; every failure incl. a locked account is `401 {"detail": "invalid username or password"}` (no `423` since 1.1.0); `429` + `Retry-After` when rate-limited. Creates a `sessions` row
POST	`/auth/mfa/verify`	public	second sign-in step `{code}` (TOTP or recovery code); a TOTP step is accepted once (replay guard); `429` when rate-limited
POST	`/auth/logout`	public	revoke the current session and clear the cookie
POST	`/auth/logout-all`	viewer (self)	1.1.0: revoke every session of the caller → `{ok, revoked}` (audit `logout.all`)
GET	`/auth/sessions`	viewer (self)	1.1.0: the caller's sessions `[{id, current, ip, user_agent, created_at, last_seen_at, expires_at}]`
POST	`/auth/sessions/{sid}/revoke`	viewer (self)	1.1.0: revoke one own session → `{ok, id}`; `404` for a session that is not the caller's (audit `session.revoked`)
GET	`/auth/me`	viewer	current user (role, MFA, `mfa_required`, must_change_password, recovery codes left)
GET	`/auth/policy`	public	password policy parameters for the live checklist
POST	`/auth/password`	viewer (self)	change own password `{current_password, new_password}`; revokes the other sessions
POST	`/auth/mfa/setup`	viewer (self)	start TOTP enrolment → QR code + key
POST	`/auth/mfa/enable`	viewer (self)	confirm `{code}` → 10 recovery codes (shown once)
POST	`/auth/mfa/disable`	viewer (self)	`{password, code}` → MFA off
POST	`/auth/mfa/recovery-codes`	viewer (self)	`{password}` → new recovery codes (old ones invalid)

Jobs — `api/jobs.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/jobs/types`	viewer	registered job types
GET	`/jobs`	viewer	list jobs `?lab_id=&host_id=&limit=100`, newest first
GET	`/jobs/{job_id}`	viewer	one job: type, status, params, result, artefacts, error, timestamps
GET	`/jobs/{job_id}/log`	viewer	full log `{log}` (secrets `***`, possibly ending with `[log truncated]`)
POST	`/jobs/{job_id}/cancel`	operator	kill a queued/running job → `{cancelled, was_running}` (`was_running: false` for a job still waiting for a slot; audit `job.cancelled`)
WS	`/jobs/{job_id}/stream`	viewer	existing log, then live lines `{line}`, then `{done: true}`

Secrets — `api/secrets.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/secrets`	viewer	names and kinds (never values) of the **caller's own** secrets; admins get every row. Each row: `id, name, kind, meta, created_at, purpose, own` (+ `created_by`, `created_by_name` for admins). Legacy rows (NULL owner) are admin-only
POST	`/secrets`	operator	create `{name, kind, value, meta?, purpose?}` → 201; `purpose` ∈ `ssh, host, git, arista, cloudvision, netbox, ai, generic` (default from the kind; `422` otherwise); reserved names `smtp_password`, `ai_default_api_key`, `github_token`, `system:*` are `403` for non-admins (audit `secret.created`)
DELETE	`/secrets/{secret_id}`	operator	delete an own secret → 204; `404` for another user's row unless admin (audit `secret.deleted`)

Audit — `core/audit.py` (1.1.0)

METHOD	PATH	ROLE	PURPOSE
GET	`/audit`	admin	`?since=&until=&actor=&event=&username=&limit=100&offset=0` → `{items, total, limit, offset}`; `event` exact or prefix form `host.*`; `limit` ≤ 1000; rows `{id, at, username, actor, event, ip, user_agent, detail}` newest first
GET	`/audit/export`	admin	same filters, no paging; `application/x-ndjson`, one JSON object per line, newest first, ≤ 100 000 rows

Hosts — `hosts/api.py` , `hosts/monitor.py`

METHOD	PATH	ROLE	PURPOSE
POST	`/hosts/keys/generate`	operator	new ed25519 key pair → `{secret_id, name, public_key}` (owned by the caller, purpose `ssh`)
POST	`/hosts/devcontainer`	operator	devcontainer zip `{mode: local-clab|remote-host, avd_version?}`
GET	`/hosts/recipes`	viewer	bootstrap recipes and their steps
GET	`/hosts`	viewer	hosts with a summary (OS, docker, containerlab, recommendations, arch, platform)
POST	`/hosts`	operator	create `{name, kind, address, ssh_port, ssh_user, ssh_key_secret_id, password_secret_id, sudo_password_secret_id, jump_host, platform}` . 1.1.0: `403 {message, violations}` for a secret the caller does not own or with an incompatible purpose (`ssh` for the key, `host` for the passwords; admins bypass); `422 {message, violations: [{path: "jump_host"}]}` for a `jump_host` that is not `[user@]host[:port]` , has option-like (`-…`) or shell characters, or a port outside 1–65535 (audit `host.created`)
GET	`/hosts/{host_id}`	viewer	one host
PATCH	`/hosts/{host_id}`	operator	update host fields; same `403` / `422` rules as `POST` (audit `host.updated`)
DELETE	`/hosts/{host_id}`	admin	delete a host (audit `host.deleted`)
GET	`/hosts/{host_id}/capabilities`	viewer	last probe document + `last_probe_at`
POST	`/hosts/{host_id}/probe`	operator	202 job `host.probe`
POST	`/hosts/{host_id}/bootstrap`	operator	202 job `host.bootstrap` `{recipe, options}`
POST	`/hosts/{host_id}/verify`	operator	202 job `host.verify`
WS	`/hosts/{host_id}/terminal`	operator	host shell `$SHELL -l` on a PTY (`?cols=&rows=`)
GET	`/hosts/{host_id}/metrics`	viewer	points + per-container stats; marks the host watched for 10 min
GET	`/hosts/{host_id}/processes`	viewer	process list
GET	`/hosts/{host_id}/containers`	viewer	`docker ps -a` with lab labels
DELETE	`/hosts/{host_id}/containers/{name}`	admin	stop (`force=false`) or `rm -f` (`force=true`); live lab containers need `{confirm: <name>}`

Images — `images/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/images/formats`	viewer	`[{fmt, arch, label}]` for cEOS / cEOS64 / cEOSarm
GET	`/images/token`	viewer	arista.com token status (never the value)
PUT	`/images/token`	operator	create/replace the token `{value}` ; drops the catalogue cache
DELETE	`/images/token`	operator	remove the token
GET	`/images/catalog`	viewer	`?fmt=&refresh=` versions from arista.com (1 h cache; 409 no token, 502 rejected/unreachable)
GET	`/images`	viewer	image store
GET	`/images/{image_id}`	viewer	one image
DELETE	`/images/{image_id}`	operator	delete an image
POST	`/images/download`	operator	202 job `image.download` `{fmt, version, force?}`
POST	`/images/{image_id}/push/{host_id}`	operator	202 job `image.push` `{options: {remote_download?, force?}}`
POST	`/images/upload`	operator	multipart `file, fmt, version, replace?` → 201. 1.1.0: the stored file is `<images_dir>/<fmt>-lab-<version>.<ext>` whatever the client name; only `.tar.xz` , `.tar.gz` , `.tar` extensions (`422`); `413 {detail: {message, size, max_bytes}}` above `AVDLAB_UPLOAD_MAX_BYTES` (checked on `Content-Length` and on the streamed bytes, `.part` removed); `409 {detail: {message, file}}` when the file exists and `replace` is not `true`

Labs — `labs/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/labs/templates`	viewer	template descriptors: knobs (with `learn`), tags, node_count, notes
GET	`/labs`	viewer	labs (`knobs.act.password` always `********`)
POST	`/labs`	operator	create `{name, template, host_id?, ceos_version?, knobs, target?}` → 201 or 422 violations (audit `lab.created`); a secret knob containing a newline/CR/NUL is `422`
GET	`/labs/{lab_id}`	viewer	one lab (knobs with secrets masked, last build job, host)
PATCH	`/labs/{lab_id}`	operator	`{knobs?, host_id?, ceos_version?, name?, cv_target_id?, netbox_target_id?, target?}` — re-render, validate, commit; sending `********` back keeps the stored secret knob
DELETE	`/labs/{lab_id}`	operator	delete a lab (409 while running or while a job runs; audit `lab.deleted`)
GET	`/labs/{lab_id}/files`	viewer	file tree with `editable` / `knob_managed` flags
GET	`/labs/{lab_id}/files/{path:path}`	viewer	file content
PUT	`/labs/{lab_id}/files/{path:path}`	operator	save `{content, message?}` (YAML + pyavd checks, commit; audit `lab.file_written`)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

METHOD	PATH	ROLE	PURPOSE
POST	`/labs/{lab_id}/build`	operator	202 job `lab.build`
GET	`/labs/{lab_id}/history`	viewer	git commits of the lab
GET	`/labs/{lab_id}/diff`	viewer	`?a=&b=&path=` unified diff between revisions
GET	`/labs/{lab_id}/devices`	viewer	device table from structured configs (type, mgmt IP, serial, loopbacks, AS, tags)
POST	`/labs/{lab_id}/deploy`	operator	202 job `lab.deploy` `{reconfigure, wait_eapi, wait_timeout}`
POST	`/labs/{lab_id}/destroy`	operator	202 job `lab.destroy`
GET	`/labs/{lab_id}/status`	viewer	inline status (20 s budget, cached fallback)
POST	`/labs/{lab_id}/status/refresh`	operator	202 job `lab.status`
POST	`/labs/{lab_id}/validate`	operator	202 job `lab.validate`
GET	`/labs/{lab_id}/validations`	viewer	validation runs with totals
GET	`/labs/{lab_id}/validations/{run_id}/report`	viewer	`?format=json|md|csv` report of a run

Lab runtime — `executors/lab_routes.py`, `executors/push.py`, `executors/psirt.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/labs/{lab_id}/clients`	viewer	client plans + container state, mgmt IP, hot_added, deployed
POST	`/labs/{lab_id}/traffic/ping`	operator	202 job `lab.traffic_ping`
POST	`/labs/{lab_id}/traffic/iperf`	operator	202 job `lab.traffic_iperf`
GET	`/labs/{lab_id}/traffic/runs`	viewer	ping / iperf / mss runs, newest first
GET	`/labs/{lab_id}/traffic/runs/{run_id}`	viewer	one run with results
GET	`/labs/{lab_id}/nodes/{node}/running-config`	viewer	inline `show running-config` (25 s)
GET	`/labs/{lab_id}/nodes/{node}/config-diff`	viewer	intended vs running `{diff, identical}`
POST	`/labs/{lab_id}/snapshot`	operator	202 job `lab.snapshot` `{commands}`
GET	`/labs/{lab_id}/snapshots`	viewer	snapshot list
GET	`/labs/{lab_id}/snapshots/{snap}/{node}/{cmd_slug}`	viewer	one snapshot output (text/plain)
WS	`/labs/{lab_id}/nodes/{node}/terminal`	operator	node terminal `?shell=cli|bash&cols=&rows=`
GET	`/labs/{lab_id}/push/plan`	viewer	`{changed, unchanged, unknown, total, since, available}`
POST	`/labs/{lab_id}/push`	operator	202 job `lab.push_configs` `{hosts?}`
POST	`/labs/{lab_id}/psirt`	operator	202 job `lab.psirt` `{tests?}`
GET	`/labs/{lab_id}/psirt`	viewer	last scan summary + rows
GET	`/labs/{lab_id}/psirt/report.md`	viewer	Markdown report of the last scan

Test hosts — `executors/client_routes.py`

METHOD	PATH	ROLE	PURPOSE
POST	`/labs/{lab_id}/clients`	operator	hot-add a test host `{name, switch, switch_port, vlans, mode, ip, image?, deploy}` → 202 job `lab.client_deploy`
DELETE	`/labs/{lab_id}/clients/{name}`	operator	remove → 202 job `lab.client_remove` (200 when not deployed)
POST	`/labs/{lab_id}/clients/{name}/exec`	operator	inline `{cmd}` in the client (20 s)
POST	`/labs/{lab_id}/clients/{name}/ping`	operator	inline `{dst, count}`
POST	`/labs/{lab_id}/clients/{name}/iperf`	operator	inline `{dst, seconds, proto, reverse, bandwidth?}`

Topology — `topology/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/topology/{lab_id}`	viewer	graph JSON (nodes, links, layout); 404 until built
GET	`/topology/{lab_id}/clab`	viewer	raw `clab/topology.clab.yml`
POST	`/topology/{lab_id}/regenerate`	operator	202 job `lab.topology`
GET	`/topology/{lab_id}/layout`	viewer	node positions `{positions, locked}`
PUT	`/topology/{lab_id}/layout`	operator	save positions (committed)
GET	`/topology/{lab_id}/ports`	viewer	used/free ports and peers per node
POST	`/topology/{lab_id}/edit`	operator	`{ops, dry_run}` → graph, changed_files, diff, warnings; 422/409 with violations

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Lab integrations (git, import/export, VS Code) — `labs/integrations_api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/labs/{lab_id}/git`	viewer	remote config, ahead/behind, last sync
PUT	`/labs/{lab_id}/git`	operator	`{url, branch, auth_kind, secret_id|null, token|null}`. 1.1.0: `422` for a `file://` URL or local path (unless `AVDLAB_GIT_ALLOW_LOCAL`), a URL starting with `-`, an invalid branch name (`^[A-Za-z0-9._/-]{1,200}$`, no `..`, `@{`, `//`, leading `-` / `/`, trailing `/`, `.`, `.lock`) or `auth_kind: token` with `http://` / `git://`; `403 {message, violations}` for an SSH key secret the caller does not own or without purpose `git`; a `token` creates the caller's secret `git-token-<slug>` (purpose `git`) (audit `lab.git_configured`)
DELETE	`/labs/{lab_id}/git`	operator	remove the remote (audit `lab.git_removed`)
POST	`/labs/{lab_id}/git/push`	operator	202 job `lab.git_push` `{force}`; the job re-checks the branch (`invalid git branch name …`)
POST	`/labs/{lab_id}/git/pull`	operator	202 job `lab.git_pull` (same check)
POST	`/labs/import`	operator	multipart archive (zip / tar.gz), name, host_id? → 201 custom lab (+ import_warnings) (audit `lab.created`)
POST	`/labs/import/git`	operator	202 job `lab.import_git`; same `422` / `403` rules as `PUT …/git`
GET	`/labs/{lab_id}/export`	viewer	zip of the lab (without .git and ANTA reports)
GET	`/labs/{lab_id}/open-links`	viewer	VS Code Remote-SSH / devcontainer links, SSH command, hints
POST	`/labs/{lab_id}/sync`	operator	202 job `lab.sync`

CloudVision — `cloudvision/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/cv/targets`	viewer	CloudVision targets (token flags only)
POST	`/cv/targets`	operator	create `{name, kind: cvaas|onprem, server, verify_certs, token?, onboarding_token?, onboarding_token_expires_at?, ztp_script?}`
GET	`/cv/targets/{target_id}`	viewer	one target
PATCH	`/cv/targets/{target_id}`	operator	update; an empty token removes the stored one
DELETE	`/cv/targets/{target_id}`	operator	delete
POST	`/cv/targets/{target_id}/test`	operator	inline REST inventory call (15 s) → `{ok, devices_seen, error}`
POST	`/labs/{lab_id}/cv/onboard`	operator	202 job `lab.cv_onboard`
POST	`/labs/{lab_id}/cv/deploy`	operator	202 job `lab.cv_deploy`
GET	`/labs/{lab_id}/cv/status`	viewer	`{mode, target, onboarding, last_deploy}`

NetBox — `netbox/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/netbox/targets`	viewer	NetBox targets
POST	`/netbox/targets`	operator	create `{name, url, token, verify_certs}`
GET	`/netbox/targets/{target_id}`	viewer	one target
PATCH	`/netbox/targets/{target_id}`	operator	update
DELETE	`/netbox/targets/{target_id}`	operator	delete (409 while bound to labs)
POST	`/netbox/targets/{target_id}/test`	operator	inline `status()` (15 s) → `{ok, version, error}`
POST	`/labs/{lab_id}/netbox/sync`	operator	202 job `lab.netbox_sync`
GET	`/labs/{lab_id}/netbox/status`	viewer	`{target, last_sync}`

ZTP, MSS, ACT — `labs/ztp_api.py`, `labs/mss_api.py`, `labs/act_api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/labs/{lab_id}/ztp`	viewer	ZTP timeline (15 s): per node MAC, IP, lease, bootfile, eAPI
GET	`/labs/{lab_id}/mss`	viewer	segmentation knob, policy matrix, rendered_on, last verify
POST	`/labs/{lab_id}/mss/verify`	operator	202 job `lab.mss_verify` `{cases|null}`
GET	`/labs/{lab_id}/act`	viewer	ACT topology info, nodes, clients, hints
GET	`/labs/{lab_id}/act/topology`	viewer	ACT topology YAML download
POST	`/labs/{lab_id}/act/prepare`	operator	re-run ACT post-processing

Learn — `learn/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/learn/schema/search`	viewer	`?q=&schema=&limit=` fuzzy key search

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

METHOD	PATH	ROLE	PURPOSE
GET	`/learn/schema/{name}`	viewer	`?key=` one schema entry
GET	`/learn/explain`	viewer	`?lab_id=&node=&line=` explain one config line
POST	`/learn/explain`	operator	`{lab_id, node, lines?, all?}` batch explain
GET	`/learn/lessons`	viewer	lesson list with progress
GET	`/learn/lessons/{lesson_id}`	viewer	parsed lesson
POST	`/learn/lessons/{lesson_id}/check`	operator	`{lab_id, checkpoint_id}` → `{passed, detail}`
GET	`/learn/progress`	viewer	passed checkpoints
GET	`/learn/glossary`	viewer	glossary terms
GET	`/learn/tours`	viewer	tours keyed by page

Meta and updates — `meta/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/meta`	public	`name, version, copyright, license, changelog_latest` plus the public admin settings (app title, GitHub URL, support/feedback e-mail, registration flag, intro video URL). 1.1.0: `git_sha`, `build_date`, `avd_version`, `anta_version`, `python` are **no longer here**
GET	`/meta/build`	viewer	1.1.0: `{name, version, git_sha, build_date, avd_version, anta_version, python, platform}` (the About page reads it)
GET	`/meta/changelog`	public	parsed `CHANGELOG.md`
GET	`/meta/notices`	public	licence and third-party notices
GET	`/meta/update`	viewer	update status: current, latest, available, mode, image, `image_error` (1.1.0: why the release image cannot be applied, e.g. not digest-pinned — then `can_self_update: false` with that `reason`), can_self_update, reason, last_update
POST	`/meta/update/check`	**admin**	force an update check (`403` for operators since 1.1.0)
POST	`/meta/update/apply`	**admin**	202 job `app.update` `{force?, to?}`; `409 {message, reason}` when impossible (no docker access, unpinned image, no newer release without `force`, a job running or queued)
GET	`/meta/update/log`	viewer	`{log, last_update}`

Feedback — `feedback/api.py`

METHOD	PATH	ROLE	PURPOSE
POST	`/feedback`	operator	`{kind, title, body, page?, include_diagnostics}` → 201
GET	`/feedback`	viewer	`?status=` list
GET	`/feedback/diagnostics`	viewer	preview of the diagnostics
GET	`/feedback/{fid}`	viewer	one entry
PATCH	`/feedback/{fid}`	operator	`{status: new|triaged|done}`
DELETE	`/feedback/{fid}`	operator	delete (audit `feedback.deleted`)

Users — `api/users.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/users`	admin	users; each row carries `mfa_required` (1.1.0: role demands an authenticator the user lacks) besides `mfa_enabled`, `locked_until`, …
GET	`/users/audit`	admin	`?limit=100` recent audit rows (with `actor`); the full query API is `GET /audit`
POST	`/users`	admin	create `{username, role, password?}` (no password → temporary password returned once)
GET	`/users/requests`	admin	account requests `?status=pending|approved|rejected|all` (default pending)
POST	`/users/requests/{req_id}/approve`	admin	`{role}` → account + temporary password (once)
POST	`/users/requests/{req_id}/reject`	admin	`{reason?, notify}` (mail the applicant when SMTP is configured)
PATCH	`/users/{user_id}`	admin	`{role?, disabled?}`; disabling revokes the user's sessions
DELETE	`/users/{user_id}`	admin	delete (not self, not the last admin)
POST	`/users/{user_id}/reset-password`	admin	new temporary password (once); revokes the user's sessions
POST	`/users/{user_id}/mfa/reset`	admin	clear MFA; revokes the user's sessions
POST	`/users/{user_id}/unlock`	admin	clear lockout
POST	`/users/{user_id}/sessions/revoke`	admin	1.1.0: revoke every session of that user and bump `session_version` → `{ok, revoked}` (audit `user.sessions_revoked`)

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Registration and admin settings — `api/registration.py`

METHOD	PATH	ROLE	PURPOSE
POST	`/register`	public	public account request → `202 {id, status: "pending", message: "request received"}` for every accepted submission, also for a taken or pending username (1.1.0, no enumeration; audited `registration.duplicate`; the fake id then answers 404 on the status route); 403 disabled, 429 rate limit
GET	`/register/status/{req_id}`	public	`{status}` of a request (404 unknown)
GET	`/admin/settings`	admin	effective admin settings (`has_smtp_password`); 1.1.0 adds `mfa_required_roles` (list) and `ai_allow_private_urls` (bool)
PUT	`/admin/settings`	admin	partial update; `null` resets to env default; `smtp_password: ""` clears; `{"mfa_required_roles": ["admin","operator"]}` , `{"ai_allow_private_urls": true}`
POST	`/admin/settings/test-mail`	admin	send a test mail → `{ok, error, to}`

AI designer — `ai/api.py`

METHOD	PATH	ROLE	PURPOSE
GET	`/ai/link`	viewer	own link (no key), shared availability, effective source
PUT	`/ai/link`	operator	`{provider, base_url?, model, api_key?}` (key write-only); `422` when `base_url` resolves to a private / loopback / metadata address and `ai_allow_private_urls` is off (1.1.0)
DELETE	`/ai/link`	operator	remove link and key
POST	`/ai/link/test`	operator	probe the provider → `{ok, models, model_listed, error}` (same `422` rule)
POST	`/ai/design`	operator	`{lab_id?, template?, prompt, history?, current_knobs?}` → proposal / question / text
POST	`/ai/design/apply`	operator	`{lab_id, knobs, topology_ops}` → re-validated, applied, committed
GET	`/admin/ai`	admin	shared AI default (no key)
PUT	`/admin/ai`	admin	update the shared default (same `422` rule for `base_url`)
POST	`/admin/ai/test`	admin	probe the shared model

7.4 Job types

`GET /jobs/types` lists the 27 registered types. Parameters are the JSON body of the endpoint that starts the job (stored as the job's `params`); defaults in parentheses.

Semantics shared by every job type (1.1.0). At most `AVDLAB_JOB_MAX_CONCURRENT` (4) jobs run at once; a job submitted beyond that is created normally (`202 {job_id}`) but stays `queued` first-in-first-out — its log starts with `job queued: 4 of 4 job slots busy, n waiting` — and `POST /jobs/{id}/cancel` on it answers `{cancelled: true, was_running: false}`. Every secret the job reveals is masked as `***` in `GET /jobs/{id}/log` and on the WebSocket; a log that reaches `AVDLAB_JOB_LOG_MAX_BYTES` (20 MiB) ends with `[log truncated]` while the job runs on. `<data>/jobs/<id>` directories older than `AVDLAB_JOB_LOG_RETENTION_DAYS` (90) are removed; the `jobs` row, `result` and `error` stay readable.

JOB TYPE	STARTED BY	PARAMETERS	RESULT (MAIN FIELDS)
`host.probe`	`POST /hosts/{id}/probe`	—	capabilities document without raw outputs, `recommendations`
`host.bootstrap`	`POST /hosts/{id}/bootstrap`	`recipe` (`clab-host` | `act-tools`), `options: {clab_mode: native|container, native_toolchain: false}`	`family` , `steps` (`skip` / `ok` / `FAIL`), `applied` , `recommendations`
`host.verify`	`POST /hosts/{id}/verify`	—	`ok` , `checks` , `recommendations`
`image.download`	`POST /images/download`	`fmt` (`cEOS` | `cEOS64` | `cEOSarm`), `version` , `force` (false)	image row: path, size, sha256, docker tag
`image.push`	`POST /images/{id}/push/{host_id}`	`options: {remote_download: false, force: false}`	method (`import` , `remote_download` , `save_load`), host
`lab.build`	`POST /labs/{id}/build`	—	`hosts` , `warnings` , `changed_files` , `anta_tests` , topology summary, commit
`lab.topology`	`POST /topology/{id}/regenerate`	—	regenerated `clab/` from the last build
`lab.deploy`	`POST /labs/{id}/deploy`	`reconfigure` (true), `wait_eapi` (true), `wait_timeout` (600, 10–7200 s)	`nodes` , `ready` , `pending` , `duration`
`lab.destroy`	`POST /labs/{id}/destroy`	—	`destroyed`
`lab.status`	`POST /labs/{id}/status/refresh`	—	`deployed` , `nodes` (state, mgmt IP, image, eapi)
`lab.validate`	`POST /labs/{id}/validate`	`extra_fabric_validation` (true), `skip_tests` (`["VerifyNTP","VerifyReloadCause"]`), `user_catalogs` (false), `tags` ([])	`totals` , `by_device` , `report_paths` , `validation_run_id` , `ansible_rc`
`lab.sync`	`POST /labs/{id}/sync`	—	`remote_dir` , `files`
`lab.push_configs`	`POST /labs/{id}/push`	`hosts` (null = changed + not tracked)	`pushed` , `skipped`

JOB TYPE	STARTED BY	PARAMETERS	RESULT (MAIN FIELDS)
`lab.psirt`	`POST /labs/{id}/psirt`	`tests` (e.g. `["SA117","SA140"]`; empty = all)	per device × advisory assessments, summary, report paths
`lab.snapshot`	`POST /labs/{id}/snapshot`	`commands` (7 defaults: running-config, version, LLDP, BGP, EVPN, MLAG, VXLAN; 1–50)	snapshot id, nodes, commands
`lab.traffic_ping`	`POST /labs/{id}/traffic/ping`	`pairs` (null = full mesh), `count` (3, ≤ 100), `size` (56), `vlan`	loss matrix, run id
`lab.traffic_iperf`	`POST /labs/{id}/traffic/iperf`	`flows: [{src, dst, proto tcp|udp, bandwidth, duration (10), parallel (1), reverse, vlan}]` (1–64), `sequential` (true)	per-flow throughput, retransmits, jitter, loss
`lab.client_deploy`	`POST /labs/{id}/clients`	`name`, `switch`, `switch_port` (`EthernetN`), `vlans`, `mode` (access), `ip` (auto), `image`, `deploy` (true)	client, push hosts, `interface_visible`, gateway ping
`lab.client_remove`	`DELETE /labs/{id}/clients/{name}`	—	removed client, push hosts, warnings
`lab.mss_verify`	`POST /labs/{id}/mss/verify`	`cases: [{src, dst, application, expect allow|deny, vlan?}]` or null (auto-derive)	passed, failed, per-case observed
`lab.cv_onboard`	`POST /labs/{id}/cv/onboard`	—	`nodes` (TerminAttr running, streaming), `complete`
`lab.cv_deploy`	`POST /labs/{id}/cv/deploy`	`submit_workspace` (true), `run_change_control` (false), `skip_missing_devices` (true), `strict_tags` (false), `workspace_name`	workspace, change control, devices, errors, warnings
`lab.netbox_sync`	`POST /labs/{id}/netbox/sync`	`delete_stale` (false), `dry_run` (false), `include_clients` (true)	`created`, `updated`, `deleted`, `unchanged`, `skipped`, `netbox_url`
`lab.git_push`	`POST /labs/{id}/git/push`	`force` (false)	last sync
`lab.git_pull`	`POST /labs/{id}/git/pull`	—	fast-forward result, re-validation
`lab.import_git`	`POST /labs/import/git`	`name`, `url`, `branch`, `auth_kind`, `secret_id` / `token`, `host_id`	new custom lab
`app.update`	`POST /meta/update/apply` (admin)	`force` (false), `to`, `image` (taken from the update status; must be digest-pinned in image mode, optionally cosign-verified)	from, to, mode, helper

Act labs use the same `lab.deploy` / `lab.destroy` / `lab.status` / `lab.validate` types; the executor is chosen from the lab's target. Jobs that refuse to run concurrently on one lab answer 409 at the API before a job is created.

7.5 Example: build, deploy and validate from a script

```
H=https://avdlab.example.com/api; J='-H content-type:application/json'
curl -s -c jar $J -d '{"username":"ops","password":"…"}' $H/auth/login
LAB=$(curl -s -b jar $J -d '{"name":"demo","template":"single-dc-l3ls","host_id":"<host id>","knobs":{"leaf_pairs":3}}' $H/labs | jq -r .id)
JOB=$(curl -s -b jar -X POST $H/labs/$LAB/build | jq -r .job_id)
until [ "$(curl -s -b jar $H/jobs/$JOB | jq -r .status)" != running ]; do sleep 5; done
curl -s -b jar $J -d '{"reconfigure":true,"wait_eapi":true,"wait_timeout":600}' $H/labs/$LAB/deploy
# … wait, then
curl -s -b jar $J -d '{}' $H/labs/$LAB/validate
```

With MFA enabled the login answers `{"mfa_required": true}` and needs `POST /auth/mfa/verify {"code": "123456"}` with the same cookie jar; for automation use an operator account without MFA on a trusted network (operators are not in the default `mfa_required_roles`), or keep MFA and supply the code. A script's session ends 12 hours after sign-in at the latest — sign in again when a call answers `401`. Do not send an `Origin` header from scripts (or send the site's own), and expect `429` + `Retry-After` when retrying a wrong password in a loop.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

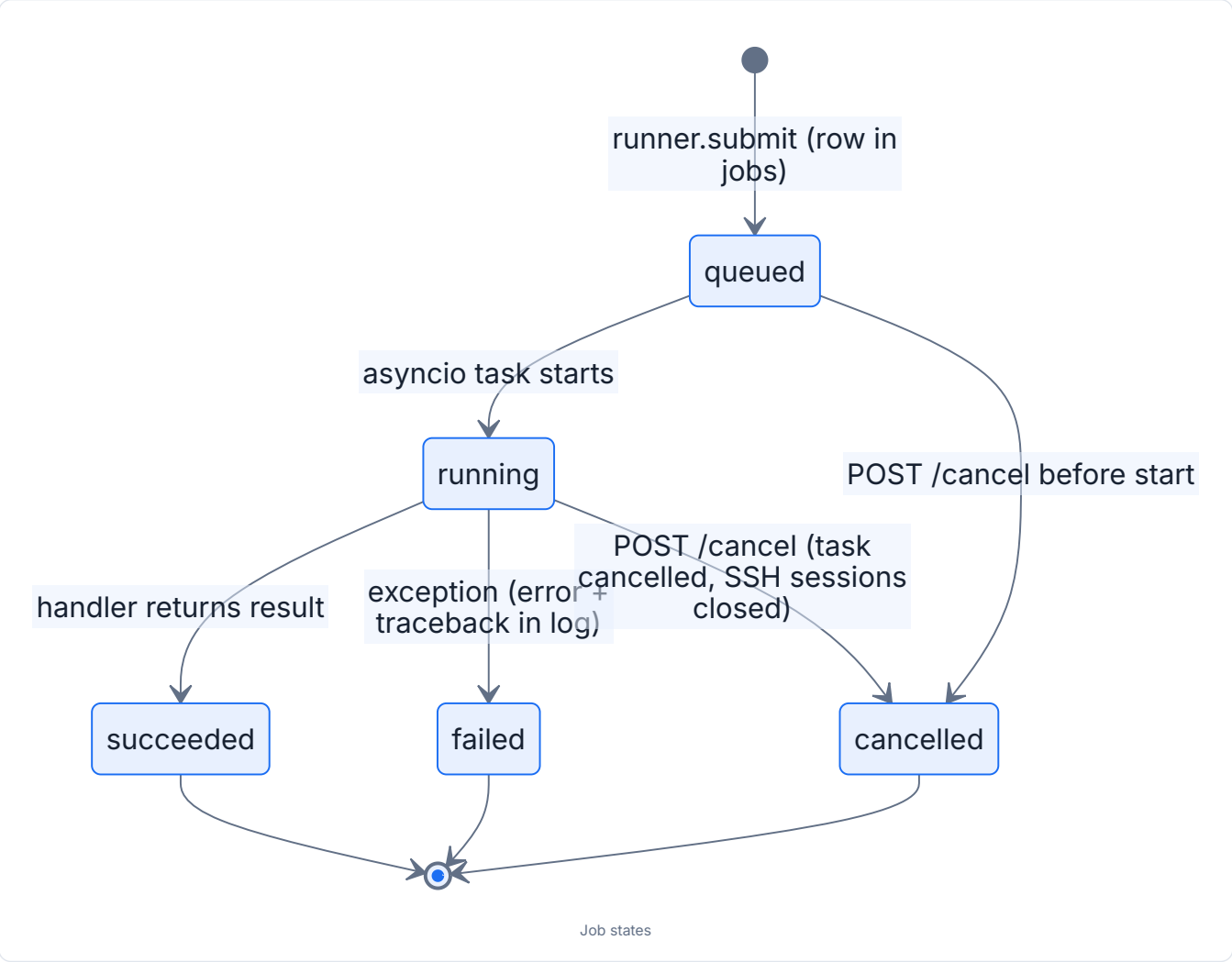
CHAPTER 8

Operations

Jobs and logs, troubleshooting matrix, monitoring, updating, versioning.

How to watch avdlab work, what to do when it does not, how to keep it current, and how releases are numbered.

8.1 Jobs and logs



Job states

WHERE	WHAT YOU GET
Job drawer	opens on every action; live log over the WebSocket; *Kill* while running or queued
Lab → Jobs tab	every job of the lab
`/jobs` (Advanced mode) and `/jobs/<id>`	all jobs; params, result, error, log
`/data/jobs/<id>/log.txt`	the raw log on disk (timestamps `[HH:MM:SS]`, UTC; secrets `***`; capped at 20 MiB; directories pruned after 90 days)
`docker compose logs -f api`	API process log: startup (refused secrets, generated session key, bootstrap password), audit/job housekeeping, update checks, AI calls (`ai design by …`), mail failures, web-terminal errors, uvicorn access log
Admin › Audit log / `GET /api/audit`	who did what: sign-ins, sessions, secrets, hosts, labs, jobs killed (1.1.0)
`/data/update.log`	output of the self-update helper
lab repo `anta/runs/`, `anta/psirt/`, `snapshots/`	validation and PSIRT reports, show-command snapshots, all committed

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Reading a job log: lines are tagged by phase — `job queued: …` while waiting for a slot, `[ssh]` connection attempts and jump hops, `[skip]` / `[apply]` / `[ok]` / `[FAIL]` recipe steps, `[hint]` probe recommendations, `[deploy]`, `[eapi]`, `[ztp]`, `[validate]`, `[push]`, `[cv]`, `[sync]`, `[perms]` (lab tree shared with the agent gid), `[update]`. A failed job ends with `job failed: <error>` and the Python traceback; the job's `error` field (shown in the UI) is the first line. Commands with secrets never show them: every revealed secret is redacted to `***` (plus built-in patterns for `Bearer …`, `password=…`, URLs with credentials, PEM keys), credentials travel on stdin or 0600 files.

Killing a job cancels its task and closes its SSH sessions, so remote commands (a long `containerlab deploy`, an ANTA run) stop with them. Containers already started stay — use *Destroy* or the host monitor to clean up.

After an API restart a job that was `queued` or `running` is not resumed; since 1.0.2 the new process marks such rows `failed` with *interrupted: the avdlab server restarted while this job was running*, so the lab is not blocked. Jobs waiting in the queue (more than `AVDLAB_JOB_MAX_CONCURRENT` running) are lost the same way and must be started again.

8.2 Troubleshooting matrix

Every row comes from a field problem fixed or explained in `CHANGELOG.md` (version in the last column) or from an error message the code produces.

Hosts and SSH

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`TCP connect timed out — host unreachable from the app container …` (after 3 attempts)	host asleep/offline, firewall, or the API container's Docker network lost LAN access	wake/check the host; `docker compose restart api`; from Azure, check VPN/Tailscale or the lab host's NSG	0.5.3
`connection refused — sshd not listening on that port`	wrong port or sshd down (Mac/WSL: sshd not enabled in the VM)	fix the port; `systemctl enable --now ssh` in the VM	0.5.3
`authentication failed (…)`	wrong key/password; public key not in `authorized_keys`; for a jump host, the key is missing on one of the two hops	re-add the public key on the host (and on the jump host)	—
`host key mismatch … remove the host's line from <data>/known_hosts`	the host (or VM) was reinstalled or the address now points elsewhere	delete that line from `/data/known_hosts` in the volume, connect again	—
`sudo needs a password on this host …` / bootstrap fails at the first sudo step	no NOPASSWD and no sudo password on the host	add a sudo password in Edit host, or `echo "$USER ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/avdlab`	0.1.1
`unsupported or undetectable OS` / *macOS (not a lab host)* / *Windows (not a lab host)*	you added the Mac or the PC itself instead of its Linux VM	Address = the VM (`avdlab-vm.orb.local`, or `localhost` on Windows), Jump host = the Mac/PC; probe again	1.0.1
`[FAIL] <step>: apply rc=…` in a bootstrap	the step's apply command failed (no internet to `download.docker.com` / `github.com`, package mirror)	fix connectivity, re-run Bootstrap (steps already done are skipped)	—
bootstrap: `sha256sum: WARNING … FAILED` / `containerlab (native)` fails	the downloaded containerlab package does not match the release `checksums.txt` (interrupted download, proxy tampering)	re-run Bootstrap; if it persists, check the proxy and compare with `https://github.com/srl-labs/containerlab/releases/tag/v0.79.0`	1.1.0
`[FAIL] containerlab (native)` although containerlab works	the installed version is not `AVDLAB_CLAB_VERSION` (the check accepts only the pinned release)	let Bootstrap replace it, or set `AVDLAB_CLAB_VERSION` to the version you want	1.1.0
`jump_host must be [user@]host[:port]` / `jump_host port must be 1-65535` (422)	option-like (`-…`) or shell characters in the field, bad port	enter `user@host` or `user@host:2222`	1.1.0
`403 secret may not be used here` + violations when saving a host	the SSH key or password secret belongs to another user, or has the wrong purpose (`ssh` for keys, `host` for passwords)	pick one of your own secrets (Generate SSH key creates one), or let an admin do the binding	1.1.0
`[perms] WARNING: falling back to world-writable report directories: anta, anta/psirt`	the agent image has no `id` binary or `docker run` failed	harmless (only `anta/` is `a+rwX`); use the default universal image or add `id` to a custom agent image	1.1.0
`user cannot run docker` after bootstrap	group membership applies to new logins	nothing to do: avdlab uses sudo for Docker; re-probe later	—
`host <name> was never probed` / `lacks docker and containerlab` on deploy	deploy checks the stored capabilities	Probe, Bootstrap, Probe	—

Images

SYMPTOM / MESSAGE	CAUSE	FIX	REF
catalogue says arista.com rejected the request	mistyped or expired arista.com token	*Replace token* (Images header) — same secret, cache dropped — then *Retry*	0.7.1
catalogue 409	no `arista_token` secret yet	*arista.com token…* → save	—
`image arista/ceos:<v> not on host <name> — push it from Images`	the image is neither on the host nor in the image store	download it (or upload), then Push or Deploy again	—

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`host is x86_64, image is arm64 …` (or the reverse)	`ceos_arch` does not match the host CPU	set `ceos_arch` (image picker does it), rebuild	0.4.0
upload: `413 upload larger than the limit of N bytes (AVDLAB_UPLOAD_MAX_BYTES)` or a Caddy `413`	the tarball is bigger than the API cap (4 GiB) or Caddy's `request_body max_size`	raise both `AVDLAB_UPLOAD_MAX_BYTES` and the Caddyfile limit together	1.1.0
upload: `409 <file> already exists: send replace=true to overwrite it`	a tarball for that format/version is already in the store	tick *Replace existing file* (form field `replace=true`)	1.1.0
upload: `422` extension	only `.tar.xz`, `.tar.gz`, `.tar` are accepted	rename the file	1.1.0

Deploy and runtime

SYMPTOM / MESSAGE	CAUSE	FIX	REF
deploy after a host crash failed with *Subnet 172.16.1.0/24 already in use by Docker network "clab-mgmt-"*	a lab's management network survived without its containers	fixed: deploy removes empty leftover networks on its subnet automatically	0.10.1
`management subnet … is in use by the running lab '<x>' (n containers) on this host`	two labs with the same `mgmt_subnet` on one host	destroy the other lab, or change this lab's `mgmt_subnet` (e.g. `172.16.2.0/24`) and rebuild	0.10.1
`containerlab deploy failed (rc=…): <last lines>`	containerlab error (image, kernel, sysctl, port names)	read the job log above; check probe recommendations	—
nodes stay `eAPI ✗` / deploy ends with pending nodes	switches still booting, or the timeout is too short for a big fabric	Runtime → raise *timeout*, *Refresh*; check host RAM (2 GB per node)	—
a page refresh or direct link answered `{"detail": "Not Found"}`	the SPA fallback did not cover deep links	fixed: every non-API path that is not a file serves `index.html`	0.10.1
the lab page went blank after a failed CloudVision deploy	a panel assumed a workspace existed and crashed the SPA	fixed: null-safe panel and error boundaries on every page and tab	0.8.0
test-host add/remove failed with an ansible *No route to host* buried under warnings	the lab's switches were not actually running (host rebooted) although the lab said deployed	fixed: a pre-check names the switch that is down and suggests *Redeploy*; remove completes with a warning	0.10.1
test-host / ACT deploy failed with `Could not find or access '/proc/1/fd/pipe:[N]'`	Ansible cannot read `-e @/dev/stdin`	fixed: credentials still on stdin, copied to a 0600 file inside the throw-away agent container	0.7.2
hot-added test host: `interface_visible: false` warning	a hot-added port is not in the `EosIntfMapping.json` the switch booted with	full *Redeploy* (the port is then mapped)	0.7.0
`… is already running for this lab` (409) but nothing runs	stale job row from a process that died; since 1.0.2 such rows are marked failed at the next start	`docker compose restart api`; on an older install: with SQLite `docker compose stop api`, then `docker run --rm -v avdlab_avdlab-data:/data alpine sh -c "apk add -q sqlite && sqlite3 /data/avdlab.db \"UPDATE jobs SET status='failed' WHERE status IN ('queued','running')\""`, then `docker compose start api`	1.0.2
a job stays `queued` with `job queued: 4 of 4 job slots busy`	the concurrency limit (`AVDLAB_JOB_MAX_CONCURRENT`) is reached	wait, kill a job you do not need, or raise the limit	1.1.0
the job log ends with `[log truncated]`	the per-job log cap (`AVDLAB_JOB_LOG_MAX_BYTES`, 20 MiB) was reached; the job itself continued	raise the cap for very chatty jobs; the result/error fields are unaffected	1.1.0
a job directory / `log.txt` is gone but the job row exists	job directory retention (`AVDLAB_JOB_LOG_RETENTION_DAYS`, 90)	`0` keeps directories forever	1.1.0
ACT deploy: `422` on a knob save mentioning `act.password`	the password contains a newline / carriage return / NUL	use a password without control characters	1.1.0
Push config changes shows every device as *not tracked yet*	the lab was deployed before incremental push existed	push them all once; afterwards only changed ones are listed	0.10.1

Validation and PSIRT

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`agent image … lacks ANTA ≥ …`	`AVDLAB_AGENT_IMAGE` points at an image with an old ANTA	use the default universal image or build `agent/` (ANTA 1.10) and set `AVDLAB_AGENT_IMAGE`; re-run Verify	—
`no ANTA JSON report produced (ansible-playbook rc=…)`	the playbook failed before ANTA wrote a report	read the ansible output in the log (inventory, credentials, reachability)	—
advisory scan: every `show running-config` based check errored	`anta psirt` ran without privileged mode	fixed: runs with `--enable` and the dedicated ANTA 1.10 image (`AVDLAB_ANTA_IMAGE`)	0.10.1
advisory scan: image pull slow on first run	the ANTA image is pulled on the lab host on first use	wait, or pre-pull `ghcr.io/aristanetworks/anta:1.10.0`	0.10.1

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CloudVision

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`CloudVision pre-flight failed …: service-account token rejected`	expired / wrong service-account token	generate a new one (CloudVision Settings → Access Control → Service Accounts), save it on the target	0.8.0
`pre-flight failed … could not reach the CloudVision API on 443`	DNS / firewall / proxy from the orchestrator, or `verify_certs` with a private CA	fix reachability or untick verify certificates on the target	0.8.0
Workspace submitted, nothing to push	designed config = running config (the fabric was deployed with the same intended configs), so CloudVision creates no change control	expected; change a knob, build, deploy to CloudVision again to see a change control	0.7.1, 0.8.0
change control stays *pending approval*	*approve & execute the change control* was unchecked	approve and run it in CloudVision, or tick the option	0.10.1
CVaaS region dropdown offered `eu-west-2` and the build failed validation	names AVD does not know	fixed: the list equals AVD 6.4's valid values (`euwest-2`, `us-4`, `apnortheast-1`, …)	0.10.1
onboarding: nodes not streaming, token file missing	the cEOS tmpfs on `/tmp` hid files copied with `docker cp`	fixed: token written inside the container via `docker exec -i`, copy on flash	0.5.2
onboarding: TerminAttr not restarting	single-step restart raced	fixed: two-step restart with a PID check, token verification, newest-log diagnostics	0.5.1
onboarding: TerminAttr state misread	check ran without privilege 15	fixed: privilege-15 checks, strict config parse, `show daemon` + log tail in the job log	0.4.1
switches cannot resolve the CVaaS name	lab DNS points at a private resolver	set `dns_servers` to public resolvers (1.1.1.1, 8.8.8.8), rebuild, push/redeploy	—
`set the cloudvision knob to cvaas or onprem before binding a CloudVision target` (409)	the target is bound while mode is `none`	set the knob first (Settings does it in the right order)	—
a cleared target token was still used	null did not remove the secret	fixed	0.3.0

NetBox

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`Invalid authorization header` with a NetBox 4.5+ token	v2 tokens `nbt_<id>.<secret>` must be sent as `Bearer`	fixed: pynetbox ≥ 7.8 pinned + header override; a pasted `Token` / `Bearer` prefix is stripped; the connection test explains both formats	0.10.1
tokens saved before 0.10.0 fail	stored with a `Token` prefix or whitespace	fixed: normalised on read	0.10.1
devices of two labs collide	the NetBox site is the AVD `dc_name` (else the fabric name)	give each lab its own `dc_name` / fabric name, or its own NetBox	—
`lab is draft: build it before syncing to NetBox`	the sync reads build outputs	Build first	—

Git remotes

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`local repositories (file:// URLs and paths on the API host) are disabled …` (422)	a `file://` URL or a local path as remote	use `https://` / `ssh://`, or set `AVDLAB_GIT_ALLOW_LOCAL=true`	1.1.0
`invalid git branch name '…'` (422 on save, or the push/pull job fails)	the branch violates git ref rules (`..`, `@{`, `//`, leading `-` / `/`, trailing `/`, `.`, `.lock`, or other characters)	rename the branch	1.1.0
`auth_kind token` refused for `http://` / `git://` (422)	a token would travel in clear text	use `https://` (or an SSH key secret)	1.1.0
`403` when choosing an SSH key secret for git	you do not own it, or its purpose is not `git`	create the key with purpose `git` (`POST /api/secrets {"kind":"ssh_key","purpose":"git",…}`) or ask an admin	1.1.0

AI designer, accounts, mail

SYMPTOM / MESSAGE	CAUSE	FIX	REF
AI panel showed a bare `HTTP 502`	the client dropped the server's message	fixed: the provider's (masked) error is shown	0.10.1
`502 …: timeout after N s`	slow model with tool use	larger `AVDLAB_AI_TIMEOUT`, a faster model	—
`409 no AI model linked`	no own link, no shared model allowed	Account → AI assistant, or ask an admin to allow the shared model	0.10.0
`429 AI design rate limit reached`	30 calls per hour per user	wait or raise `AVDLAB_AI_RATE_LIMIT`	0.10.0

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`422 base_url resolves to a private address (…)` when saving or testing an AI link	the model server is on a LAN / loopback / metadata address (SSRF guard)	admin: `PUT /api/admin/settings {"ai_allow_private_urls": true}` or `AVDLAB_AI_ALLOW_PRIVATE_URLS=true`; or use a public endpoint	1.1.0
signed in over `http://…:8080` but bounced back to the login page	the session cookie is `Secure` (public URL is https)	use the https URL	0.8.0
invalid username or password although the password is right	the account is locked (5 wrong passwords) — the form no longer says so	wait 15 minutes, or an admin checks *Locked* on the Users page and *Unlock*s	1.1.0
`429 too many attempts: try again later` on sign-in	more than `AVDLAB_AUTH_RATE_LIMIT` (20) login/MFA attempts per minute from one IP (a shared NAT address counts as one)	wait `Retry-After` seconds; raise the limit for large NATed offices; check that `AVDLAB_TRUSTED_PROXIES` includes your proxy so real IPs are seen	1.1.0
the authenticator code is refused right after it was accepted	TOTP replay guard: a time step is accepted once	wait for the next 30 s code	1.1.0
everyone was signed out after the upgrade	pre-1.1.0 cookies have no session id	sign in again (once)	1.1.0
signed out after 12 hours although active	absolute session lifetime (`AVDLAB_SESSION_ABSOLUTE_MAX_AGE`, 43200 s)	sign in again; raise the setting if your policy allows	1.1.0
`403 mfa_required` on every page / redirected to Account with an amber banner	your role is in `mfa_required_roles` (admins by default) and you have no authenticator	set up two-factor authentication on the Account page; or an admin changes `mfa_required_roles` via `PUT /api/admin/settings`	1.1.0
`403 origin not allowed` (or WebSocket closes with 4403)	the browser sent an `Origin` that is neither the site itself nor a configured CORS origin — typically an extra hostname/port not in `AVDLAB_PUBLIC_URL` / `AVDLAB_CORS_ORIGINS`, or a cross-site form	open the app under its configured URL; add legitimate origins to `AVDLAB_CORS_ORIGINS`	1.1.0
`403 the secret name … is reserved for administrators`	`smtp_password`, `ai_default_api_key`, `github_token` or `system:*` created by an operator	let an admin create it, or choose another name	1.1.0
`403 {message, violations}` when binding a secret (purpose mismatch)	a host needs `ssh` (key) / `host` (passwords), git needs `git`; the secret has another purpose or another owner	create the secret with the right purpose; admins bypass the check; legacy rows (pre-1.1.0) can only be bound by admins	1.1.0
`404` on a secret you can see in the database	it belongs to another user (operators see only their own)	sign in as its owner or an admin	1.1.0
lost phone and recovery codes	MFA	an admin *Reset MFA* (this also signs the user out everywhere)	0.8.0
lost the only admin password	—	stop, `sqlite3 /data/avdlab.db "DELETE FROM users"` (removes all accounts), start: admin re-created from env	0.8.0
no e-mails	SMTP host empty, or the server refuses	Admin settings → *Send test mail* shows the SMTP error; failures are only logged, never block requests	0.9.0
test mail: *SMTP AUTH over a plaintext connection refused: enable STARTTLS, use port 465, or set AVDLAB_SMTP_ALLOW_INSECURE=true*	login configured on port 25/587 without STARTTLS	enable *use STARTTLS* or port 465; `AVDLAB_SMTP_ALLOW_INSECURE=true` only for an isolated relay	1.1.0
`POST /api/register` answered 202 but the status page says *not found*	the username was taken or already pending; the API answers *request received* for everyone and returns a non-existent id	an admin sees the attempt as `registration.duplicate` in the audit log; ask them to approve the pending request	1.1.0
viewer cannot send feedback	`POST /api/feedback` is a write; viewers are read-only	use the *E-mail* button in the feedback form, or give the user operator	—
Save buttons hidden behind the Feedback button	layout	fixed: sticky save bars at the top-left	0.10.1

Installation and updates

SYMPTOM / MESSAGE	CAUSE	FIX	REF
TypeScript build fails after unpacking a tarball (`name 2.ts`)	Finder unpacked next to the folder	extract with `tar -xzf … -C ~`, or delete `* [0-9].*` files (chapter 3)	—
api does not start: `RuntimeError: AVDLAB_SESSION_SECRET is the old default 'change-me-session-secret' …` / `must be at least 32 characters`	`.env` still carries the 1.0.x default or a short secret	set `openssl rand -hex 32`, or leave the variable empty so `/data/session.key` is generated	1.1.0
api does not start: malformed `AVDLAB_SECRET_KEY` / `AVDLAB_SECRET_KEYS`	a key in the list is not a valid Fernet key	fix the list (comma-separated, no spaces); never drop a key that still encrypts rows (check with `check-keys`)	1.1.0
`Permission denied` on `/data/...` after the upgrade	the api now runs as uid 10001 and the volume is owned by root	`docker compose run --rm --user 0 api chown -R 10001:10001 /data`	1.1.0

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

SYMPTOM / MESSAGE	CAUSE	FIX	REF
`Read-only file system` in a job	the container root is read-only; something wrote outside `/data` or the tmpfs	report it (bug); as a workaround set `read_only: false` temporarily	1.1.0
Update panel: `docker socket … is not mounted` / `AVDLAB_DOCKER_HOST=… is not a docker endpoint`	self-update needs Docker access through the proxy (or the legacy socket)	`docker compose --profile update up -d` and `AVDLAB_DOCKER_HOST=tcp://docker-socket-proxy:2375` (`tcp://host:port` form)	1.1.0
source-mode update fails at `docker compose up -d --build` with a 403 from the proxy	`BUILD` / `SESSION` are blocked by the docker-socket-proxy by default	`AVDLAB_PROXY_ALLOW_BUILD=1` and recreate the proxy (`docker compose --profile update up -d`)	1.1.0
Update panel: amber *release image* row — `release image '…' is not digest-pinned (<image>:<tag>@sha256:<64 hex>) …` (`image_error`)	image-mode update, but the manifest / release notes do not carry the digest	publisher: add `@sha256:…` to the manifest `image` (or a `digest` field) or to the release notes; `AVDLAB_UPDATE_REQUIRE_DIGEST=false` only for a trusted private registry	1.1.0
`update source …: manifest URLs must use https://`	`AVDLAB_UPDATE_SOURCE` points at an `http://` manifest	use https	1.1.0
`AVDLAB_UPDATE_COSIGN_IDENTITY is set but the cosign CLI is not installed in the API container`	signature verification configured, binary missing (fail closed)	add `COPY --from=ghcr.io/sigstore/cosign/cosign:v2.4.1 /ko-app/cosign /usr/local/bin/cosign` to `Dockerfile.backend`, or unset the identity	1.1.0
`cosign verification of <image> failed (…)`	the image is not signed by the configured identity/issuer (or was tampered with)	do not update; check the release workflow identity (`^https://github.com/<owner>/<repo>/.github/workflows/release.yml@refs/tags/v`)	1.1.0
`403` on *Check now* / *Update now*	the update routes are admin-only	sign in as an admin	1.1.0
`AVDLAB_DEPLOY_DIR is not set` / `must be an absolute host path`	compose not run from `deploy/`, or `-f` from elsewhere	run compose from `deploy/` or set it explicitly	0.7.0
`409` on Update now	a job is running or queued, no newer release is known, or `image_error`	wait / *Check now*; `force` re-applies a known release	0.7.0
the new version did not answer in time	the helper failed (git conflicts, build error, could not reach `DOCKER_HOST` on `AVDLAB_UPDATE_HELPER_NETWORK`)	read `/data/update.log`	0.7.0
`db` / `netbox` container exits immediately: *… must be set* / empty password	`POSTGRES_PASSWORD`, `NETBOX_SECRET_KEY`, `NETBOX_DB_PASSWORD`, `NETBOX_REDIS_PASSWORD`, `NETBOX_SUPERUSER_PASSWORD` or `NETBOX_API_TOKEN` is empty (no defaults since 1.1.0)	set them in `.env` — with the *old* default values if the database already exists	1.1.0
NetBox not reachable from the LAN	published on `127.0.0.1` only	`NETBOX_BIND_ADDRESS=0.0.0.0`	1.1.0
image build fails: `frontend/package-lock.json is required (npm ci)` / cannot read `backend/requirements.lock`	a tree without the lockfiles	`git pull` / use a release tarball; regenerate the lock as in chapter 9 §9.3	1.1.0
Caddy: no certificate	DNS A record missing/late, port 80 closed, rate limit	create DNS first, open 80/443, use `AVDLAB_ACME_STAGING=true` while testing	0.8.0
Caddy answers `413`	request body above 16 MB (4 GB on the image upload)	raise `request_body max_size` in the Caddyfile (and `AVDLAB_UPLOAD_MAX_BYTES`)	1.1.0
audit / session IPs show the proxy's address	the proxy is not in `AVDLAB_TRUSTED_PROXIES`	add its IP/CIDR (default covers loopback + RFC 1918)	1.1.0

8.3 Monitoring

WHAT	HOW
API health	Docker healthcheck (`curl http://127.0.0.1:8080/api/auth/me` → 200 or 401) every 30 s; `docker compose ps`
Lab hosts	Dashboard *Lab hosts* panel (CPU, memory, disk, containers per host, every 30 s); host Monitor page (charts, processes, containers); 7 days in `host_metrics`
Lab status	Runtime tab (inline inspect + eAPI probe); `lab.status` as a job
Certificates	`docker compose --profile public logs caddy`; Caddy renews ~30 days before expiry

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

WHAT	HOW
Sign-ins, sessions, account, secret, host and lab changes	*Admin › Audit log* (`/admin/audit`) with filters; `GET /api/audit?since=&actor=&event=host.*&username=&limit=&offset=` ; the Users page keeps the last 100 events
Audit export / SIEM	`GET /api/audit/export?since=…` → NDJSON (newest first, ≤ 100 000 rows); schedule it before rows age out (`AVDLAB_AUDIT_RETENTION_DAYS` , 365; `0` keeps forever). Example: `curl -b jar "https://host/api/audit/export?since=2026-01-01T00:00:00Z" > audit-$(date +%F).ndjson`
Active sessions	each user on the Account page; admins revoke indirectly with *Disable*, *Reset password* or *Reset MFA* (all revoke the user's sessions)
Job engine	`job queued` lines when the 4-slot limit is hit; `[log truncated]` when `AVDLAB_JOB_LOG_MAX_BYTES` (20 MiB) is reached; `docker compose logs api` shows `job housekeeping: removed N job directories older than 90 days`
Build facts	`GET /api/meta/build` (authenticated): version, git sha, build date, AVD/ANTA versions, Python, platform
Problems reported by users	`/feedback` (with diagnostics; secrets masked), optional GitHub issues and mail
Updates	version badge pill; Update panel (incl. `image_error`); `GET /api/meta/update`
Supply chain	`docker compose config` to confirm digest pins; the release's `SHA256SUMS` , `avdlab-<ver>.cdx.json` SBOM and cosign signature (8.5)

The monitor samples only hosts with a deployed lab or viewed in the last 10 minutes, one SSH session per host, and never raises out of its loop (a failing host is logged and retried on the next tick). Disable it with `AVDLAB_MONITOR_ENABLED=false` if background SSH is not wanted.

8.4 Updating

Chapter 3 §3.3 has the three paths (tarball, git, in-app). Operational rules:

- Update when no job runs (the in-app update refuses otherwise). Running labs are not touched: containers on lab hosts keep running across an orchestrator update.
- Back up the `avdlab-data` volume first (chapter 3 §3.4).
- The database schema is migrated automatically at startup (new tables, added columns).
- After the update, check *About* (version, git sha, build date) and `GET /api/meta/build` .
- Upgrading from 1.0.x: follow the table in chapter 3 §3.3 (session secret, MFA for admins, `/data` ownership, `update` profile, secrets without defaults, re-bootstrap of lab hosts).

8.5 Versioning and releases

avdlab follows Semantic Versioning and keeps a Keep a Changelog `CHANGELOG.md` . **House rule since 0.10.1: every fix that ships gets its own patch version**, so the sidebar badge and `GET /api/meta` say exactly what is deployed.

The version lives in three files kept in sync by the release script: `VERSION` , `backend/pyproject.toml` , `frontend/package.json` .

`scripts/release.py` :

```
scripts/release.py patch                # 1.0.1 → 1.0.2
scripts/release.py minor                # 1.0.2 → 1.1.0
scripts/release.py major                # → 2.0.0
scripts/release.py 1.2.3                # explicit
scripts/release.py patch --tar <dir>    # also writes <dir>/avdlab-v<version>.tar.gz
scripts/release.py patch --no-commit    # only bump files and changelog
scripts/release.py patch -m "extra line" # extra commit message line
```

STEP	WHAT IT DOES
bump	computes the new version from `VERSION`
write versions	`VERSION` , `version = "…"` in `pyproject.toml` , `"version"` in `package.json`
roll changelog	renames `## [Unreleased]` to `## [<version>] - <today>` ; without an Unreleased section it inserts a stub `### Fixed - (describe the fix)`
commit + tag	`git add -A` , commit `release <version>` , annotated `git tag -a v<version>` . **1.1.0: a tag that already exists is never moved** — the script stops with *tag vX exists — bump again* (re-tagging would silently change what a released version means); the working tree must be clean after the release commit (`--allow-dirty` overrides)
tarball	`avdlab/` tree without `node_modules` , `dist` , `__pycache__` , `.pytest_cache` , `.venv` , `.git` , `*.pyc` , `.work/` , caches; **1.1.0:** `--tar` **also writes** `avdlab-<ver>.cdx.json` (CycloneDX 1.5 source SBOM from `scripts/sbom.py`) **and** `SHA256SUMS` over the assets (the PDF when built with `--pdf`)

Work in progress goes under `## [Unreleased]` in `CHANGELOG.md` (Added / Changed / Fixed / Security); the script renames it.

GitHub workflows (1.1.0). `ci.yml` runs on every push and pull request (not on `v*` tags) with these jobs — the names to require in branch protection on `main` : `lint` (ruff on backend and scripts, eslint + `tsc` on the frontend), `test-backend` (pytest with coverage), `test-frontend` (vitest with coverage + `npm run build`), `sca` (`pip-audit` on `requirements.lock` , `npm audit --omit=dev --audit-level=high`), `secrets` (gitleaks over the full history), `build` (image build, Trivy failing on fixable CRITICAL/HIGH, SARIF to code scanning). `codeql.yml` adds `analyze (python)` and `analyze (javascript-typescript)` on pushes to `main` , pull requests and weekly. Every action is pinned to a commit SHA. `release.yml` runs on a pushed tag

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

`v<version>`: both suites again, `tag == VERSION`, the multi-arch image (amd64 + arm64) pushed as `ghcr.io/<owner>/<repo>:<version>` and `:latest`, a source SBOM and an image SBOM (syft), SLSA provenance attestations, keyless **cosign** signing and attestation, an in-job `cosign verify`, `SHA256SUMS`, and a GitHub Release whose notes are the CHANGELOG section plus a *Supply chain* section with the line `<image>:<version>@sha256:…` — the line the self-update parses to get a digest-pinned image — and the verification commands. A GitHub release is what `AVDLAB_UPDATE_SOURCE=github:<owner>/<repo>` finds. Details: chapter 9 §9.9–9.10.

8.6 Rotating the secret-store key

The secret store encrypts every stored credential and every user's TOTP seed. 1.1.0 supports several keys at once (`AVDLAB_SECRET_KEYS`, comma-separated: the **first** encrypts, **all** decrypt; `AVDLAB_SECRET_KEY` is appended as a decrypt-only key when both are set). Rotation, in order:

1. Generate a key: `python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`.
2. In `.env` put it **first**: `AVDLAB_SECRET_KEYS=<new>,<old>` (keep the old key in the list, or in `AVDLAB_SECRET_KEY`).
3. `docker compose up -d api` (the keys are validated at startup).
4. `docker compose exec api python -m avdlab.cli rotate-secrets --dry-run` — reports how many rows would be re-encrypted; then the same without `--dry-run`. Secrets and TOTP seeds are rewritten with the primary key.
5. `docker compose exec api python -m avdlab.cli check-keys` — which key opens how many rows. Exit status `2` means some rows cannot be decrypted with any configured key: do **not** remove a key until this is `0` rows.
6. Remove the old key from `.env`, restart, run `check-keys` once more. Back up `.env` and the data volume.

The full playbook (cadence, who holds which key, what to do after a suspected key leak, re-enrolling users whose TOTP seeds became unreadable) is in `docs/security/`.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

CHAPTER 9

Development

Repo layout, running locally, tests, templates and knobs, adding jobs and pages, releasing.

This chapter is for people who change avdlab: where things are, how to run and test them, and the recipes for the usual extensions (a job, a page, a template, a knob). The design contracts in `docs/CONTRACT-*.md` are the original per-phase specifications and remain the best explanation of *why* a module looks the way it does.

9.1 Repository layout

```
avdlab/
├── VERSION, CHANGELOG.md, LICENSE, THIRD_PARTY_NOTICES.md   served by /api/meta*
├── SECURITY.md, CONTRIBUTING.md                             1.1.0: disclosure policy, contribution rules
├── ruff.toml, .pre-commit-config.yaml, .gitleaks.toml       1.1.0: lint config for scripts/, pre-commit hooks, gitleaks allowlist
├── backend/                FastAPI app `avdlab` (Python 3.11) + pytest suite
│   ├── pyproject.toml      dependencies (pyavd[ansible]==6.4.0, anta==1.10.0, asyncssh, sqlalchemy, …), [tool.ruff], dev extras
│   ├── requirements.lock   1.1.0: hash-pinned runtime dependencies (uv pip compile); what the image installs
│   ├── avdlab/
│   │   ├── main.py         app factory, middleware, FEATURE_MODULES, SPA mount
│   │   ├── cli.py          1.1.0: `python -m avdlab.cli rotate-secrets [--dry-run]`, `check-keys`
│   │   ├── core/           config (settings), db (all models + light migrations), security (sessions, RoleGuard,
│   │   │                   OriginGuard, MultiFernet secret store, ownership), passwords (policy, argon2id, TOTP),
│   │   │                   audit (recorder, AuditMiddleware, /audit API, retention), ratelimit (token bucket),
│   │   │                   app_settings, mailer
│   │   ├── api/            auth (login, MFA, sessions), jobs, secrets (owner + purpose), users, registration (admin settings)
│   │   ├── jobs/engine.py  @job registry, JobRunner (concurrency limit, housekeeping), JobContext (Redactor, log cap)
│   │   ├── hosts/          api, ssh (asyncssh wrapper, TOFU, jump host), probe, recipes, monitor, devcontainer
│   │   ├── images/         api, service (catalogue, tags, arch), docker (local docker / ardl wrapper)
│   │   ├── labs/           api, service (lab dirs + git), templates (knobs + rendering), build (pyavd), git, archive,
│   │   │                   integrations_api, mss, mss_api, ztp_api, act_api
│   │   ├── topology/       clab (AVD → containerlab), clients, edit (topology ops), ztp, models, api
│   │   ├── executors/      base (interface), clab, act, common, configs, traffic, hot_clients, push, psirt, mss,
│   │   │                   terminal, lab_routes, client_routes
│   │   ├── cloudvision/    api, service (targets, REST, cv_deploy playbook), jobs (onboard, deploy)
│   │   ├── netbox/         api, service (client factory, tokens), sync (plan + reconciler), jobs
│   │   ├── learn/          api, lessons/*.md, glossary.yml, tours.yml, knob_learn.yml, schema/*.json, explain(+_rules)
│   │   ├── ai/             api, config (links, shared default), designer (prompt, tools, dry run), providers
│   │   ├── meta/           api (version, changelog, notices, build), update (check + digest/cosign-verified self-update)
│   │   └── feedback/       api
│   └── tests/              36 test files, pytest-asyncio, fake SSH / docker / HTTP (1.1.0: test_security_11.py, test_platform_11.py)
├── frontend/               Vite + React 18 + TypeScript + Tailwind SPA
│   └── src/
│       ├── App.tsx, main.tsx     routes, MSW start in mock mode
│       ├── api/                  typed fetch clients per area + types.ts (1.1.0: audit.ts, client.ts mfa_required handling)
│       ├── components/           Layout (navigation), ui (SafeLink), KnobForm, TopologyGraph/Editor, JobDrawer, UpdatePanel, ai/, learn/, …
│       ├── pages/ + pages/lab/   one file per page and per lab tab (1.1.0: AuditLog.tsx; Account.tsx sessions)
│       ├── hooks/, lib/          auth, meta, job streams; uiMode, labActions, formatting, markdown (safeHref), domain helpers
│       ├── mocks/                MSW handlers + fixtures (mock mode, tests)
│       └── test/                 20 vitest files + setup.ts (1.1.0: security11.test.tsx)
├── templates/<name>/       AVD example (inventory.yml, group_vars/, playbooks/) + template.yml (knobs)
├── agent/Dockerfile        optional custom agent image (universal image + ANTA 1.10 + eos-downloader + dnsmasq …)
├── deploy/                 docker-compose.yml, Dockerfile.backend, Caddyfile, .env.example, azure/setup-azure-vm.sh
├── scripts/                release.py (tags, SBOM, SHA256SUMS), sbom.py (CycloneDX 1.5), export_schema.py, docs-pdf/, tests/
(test_release.py)
├── docs/                   design notes, CONTRACT-*.md, multi-os-hosts.md, this 1.1/ set, security/ (security documentation set)
├── cli/                    empty placeholder (the CLI lives in backend/avdlab/cli.py)
└── .github/                workflows/ (ci.yml, codeql.yml, release.yml), dependabot.yml, CODEOWNERS, PULL_REQUEST_TEMPLATE.md
```

9.2 Running it locally

Backend

```
cd backend
python3.11 -m venv .venv && . .venv/bin/activate
pip install -e '.[dev]'
AVDLAB_DATA_DIR=$PWD/../data AVDLAB_DATABASE_URL=sqlite+aiosqlite:///$PWD/../data/avdlab.db \
  AVDLAB_ADMIN_PASSWORD=admin uvicorn avdlab.main:app --reload --port 8000
```

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Templates are read from `<repo>/templates` by default (`AVDLAB_TEMPLATES_DIR`). With `frontend/dist` present the API also serves the built SPA. SSH-based features need a real lab host; everything else (templates, build, topology, learn, AI with a reachable provider) works on a laptop.

Frontend

```
cd frontend && npm install
npm run dev            # http://localhost:5173, proxies /api (incl. WebSockets) to :8000
VITE_MOCK=1 npm run dev   # mock mode: MSW intercepts /api in the browser, no backend needed (amber "mock" tag in the sidebar)
npm run build          # tsc -b && vite build → frontend/dist
npm run lint           # eslint (clean since 1.1.0)
npm run typecheck      # tsc -b --noEmit
npm run test:coverage  # vitest with v8 coverage (≈ 87 % lines in 1.1.0)
```

Mock mode uses the handlers and fixtures in `src/mocks/` (`handlers*.ts` , `data.ts` , `phase*.ts`). It is how the intro video was recorded and how the UI can be demoed without a host. A reload resets the in-memory mock database.

Docker image

```
cd deploy && docker compose up -d --build     # same image as production
```

The image build needs `backend/requirements.lock` and `frontend/package-lock.json` (it runs `npm ci` and `pip install --require-hashes --no-deps`); a local change to `pyproject.toml` therefore requires refreshing the lock (9.3).

9.3 Tests

SUITE	COMMAND	SIZE (1.1.0)	NOTES
backend	`cd backend && python3 -m pytest -q -p no:cacheprovider`	**354 passed, 8 skipped** in 36 files (`testpaths = tests` , `asyncio_mode = auto`)	no Docker, no SSH: `SSHClient.connect` is replaced by fakes (`FakeSSH` maps command substrings to `(rc, stdout)`), local Docker by `FakeDocker` , HTTP by `httpx.MockTransport` ; the `client` fixture logs in as the bootstrap admin; `conftest.py` sets `AVDLAB_GIT_ALLOW_LOCAL=true` for the git tests
frontend	`cd frontend && npm test` (`vitest run`)	**176 passed** in 20 files	jsdom, Testing Library, MSW handlers (incl. `/audit` , `/auth/sessions` , `/meta/build` , the `image-unpinned` update scenario)
scripts	`python3 -m pytest scripts/tests -q`	**6 passed**	`release.py` tag / clean-tree / SBOM / SHA256SUMS rules
lint	`ruff check backend scripts` ; `cd frontend && npm run lint && npm run typecheck`	all clean	the `lint` CI job; `ruff format --check` is report-only
coverage	`cd backend && python3 -m pytest --cov=avdlab --cov-report=xml --cov-report=term` ; `cd frontend && npm run test:coverage`		what `test-backend` / `test-frontend` upload as artefacts
audits	`pip-audit -r backend/requirements.lock --no-deps` ; `cd frontend && npm audit --omit=dev --audit-level=high`	0 known vulnerabilities at release	the `sca` CI job (the full `npm audit` incl. dev dependencies is report-only)

Ruff is configured in `backend/pyproject.toml` (`[tool.ruff]` : E, F, W plus the security-relevant B023, B904, S113, S324, S501, S602, S604, S605, S608; E501 ignored; `tests/**` may have E402) and extended for `scripts/` by the root `ruff.toml` . `pre-commit install` runs ruff, eslint/tsc and gitleaks before each commit (`.pre-commit-config.yaml`).

Notable test guarantees: the pyavd build of the default `single-dc-l3ls` template is byte-identical to Arista's shipped `intended/configs` ; the generated containerlab topology equals the shipped example link for link; every template builds (`test_templates_all.py`); `test_security_11.py` covers sessions, rate limit, MFA policy, TOTP replay, key rotation, secret scoping, audit API, Origin guard, CSP, SSRF guard and the uniform login/registration answers; `test_platform_11.py` covers digest/cosign update checks, `DOCKER_HOST` , job queueing / log cap / retention, redaction patterns, upload limits, ACT secret knob, git and branch validation, SMTP refusal, the signed bootstrap recipe and the lab-tree permissions.

Run time: the backend suite takes several minutes (pyavd builds).

Refreshing the lockfile

`backend/requirements.lock` pins every runtime dependency with SHA-256 hashes for Python 3.11 / Linux (the `python:3.11-slim` image, valid for `linux/amd64` and `linux/arm64` ; not for macOS dev machines). After editing `backend/pyproject.toml` , or for a Dependabot bump, from the repository root:

```
uv pip compile --python-version 3.11 --generate-hashes --no-strip-extras \
    --output-file backend/requirements.lock backend/pyproject.toml
# pip-tools equivalent (much slower: it downloads every distribution to hash it):
python3.11 -m piptools compile --generate-hashes --output-file backend/requirements.lock backend/pyproject.toml
```

Then `pip-audit -r backend/requirements.lock --no-deps` and the backend suite. `pyavd` and `anta` stay pinned in `pyproject.toml` (`==6.4.0` , `==1.10.0`) and are ignored by Dependabot — bump them deliberately. The frontend lock is the ordinary `npm install` / `npm update` + `npm audit fix` (non-breaking); the image stage runs `npm ci` only.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

9.4 How templates and knobs are defined

A template is a directory `templates/<name>/` with the AVD example files and a descriptor `template.yml`:

```
name: single-dc-l3ls
title: Single DC L3LS (EVPN/VXLAN, 2 spines, 2 MLAG leaf pairs, 2 L2 leafs)
description: |
  eBGP underlay + eBGP EVPN overlay …
tags: [l3ls, evpn, vxlan, mlag, ebgp]
notes: ...                # optional caveats shown with the template
learn: |                  # what it teaches + AVD doc URL
  Teaches the reference AVD design end to end: …
fabric_name: FABRIC
generator: l3ls           # l3ls | static | none
base_mgmt_subnet: 172.16.1.0/24   # static: the example's mgmt subnet (re-based onto knob mgmt_subnet)
l3ls: {…}                 # l3ls generator only: group names, mgmt offsets, L2 leaf MLAG pool
knobs:
  - key: leaf_pairs
    type: int             # int | string | cidr | ipv4 | pool | secret | object | yaml | bool
    default: 2
    min: 1
    max: 8
    label: L3 leaf MLAG pairs
    help: Spine downlinks are extended beyond Ethernet1-8 automatically …
    learn: {summary, avd_keys, renders, pitfalls, doc_url}
  - key: tenants
    type: yaml
    default_from: "NETWORK_SERVICES:tenants"   # default read from the example's group_vars
  - key: something
    type: string
    path: "FABRIC:some.dotted.key"            # static generator: where the value is written
```

Knob model fields (`labs/templates.py` `Knob`): `key`, `type`, `label`, `help`, `default`, `default_from` (`GROUP:dotted.path` into the base group_vars), `min`, `max`, `path` (static generator target), `choices`, `value_type` (type every value of an object knob must satisfy, e.g. `pool`). `coerce_knobs()` merges user values over the defaults, coerces types and checks ranges (raising `KnobError` → 422); `render_lab()` produces the complete file set:

- generator `l3ls` builds inventory and node-type group_vars from counts and pools;
- generator `static` loads the example and applies targeted edits (mgmt re-basing, password, serials, platform, knobs with a `path`);
- both end with a common pass (passwords, serial numbers, platform, eAPI);
- `apply_topology_overrides` re-applies topology-editor overrides afterwards.

Cross-cutting knobs (`cloudvision`, `dns_servers`, `device_tags`, `provisioning`, `segmentation`, `client_image`, `ceos_arch`, `act`) are handled by name in the renderer, the build (`labs/build.py`) or the topology generator. Every knob should carry a `learn` block; `python3 -m avdlab.learn.knobs --write [templates_dir]` inserts missing blocks from `learn/knob_learn.yml` without disturbing comments or key order. `GET /api/labs/templates` serves the descriptors with resolved defaults and `learn` blocks; the frontend `KnobForm` renders them by type.

Adding a template

1. Copy the AVD example into `templates/<name>/` (`inventory.yml`, `group_vars/`, `playbooks/`).
2. Write `template.yml`: `generator: static` (start here), `fabric_name` (the inventory group the build and ANTA use), `base_mgmt_subnet`, tags, `learn`, and the standard knobs (copy them from an existing static template: `mgmt_subnet`, `mgmt_gateway`, `ceos_version`, `ceos_arch`, `device_password`, `serial_scheme`, `dns_servers`, `cloudvision`, `client_image`, `device_tags`, `provisioning`, `segmentation`, `act`).
3. Replace hardware platforms cEOS cannot emulate with cEOSLab where needed (see `campus-fabric`), and make passwords literal (see the `cv-pathfinder` notes in the changelog).
4. Run `python -m pytest -q tests/test_templates_all.py` — it renders and builds every template.
5. Build it once in the UI or `build_lab_dir()` and check the containerlab topology (`clab/topology.clab.yml`).
6. The AI designer and the from-a-sentence catalogue pick it up automatically (descriptor + knobs).

9.5 How to add a job

1. **Write the job** in the feature's module and register it with the decorator: ```python from ..jobs.engine import JobContext, job

@job("lab.something") async def lab_something(ctx: JobContext) -> dict: lab = await ctx.get_lab() ctx.log("[something] starting") async with ctx.session() as s: async with SSHClient.connect(host, s, log=ctx.log, ctx=ctx) as ssh: # ctx= → Kill closes it res = await ssh.run("...", sudo=True, timeout=600, log=ctx.log, input=secret_on_stdin) ctx.check_cancelled() return {"ok": True} # stored as the job result `Raise an exception (e.g. `ExecutorError("clear, actionable message")`) to fail the job; the first line becomes the job's `error`. Put CPU-bound work in `asyncio.to_thread`. 2. **Start it from a route** that answers 202:` python @router.post("/{lab_id}/something", status_code=202) async def start(lab_id: str, body: SomethingIn | None = None, session=Depends(get_session), user=Depends(current_user)): lab = await _lab(session, lab_id) await _busy(session, lab, ("lab.deploy", "lab.something")) # 409 while a conflicting job runs return {"job_id": await runner.submit("lab.something", lab_id=lab.id, host_id=lab.host_id, params=(body or SomethingIn()).model_dump())} ``` `3. **Register the module**` in main.FEATURE_MODULES `(append a line; importing the module registers its jobs and router). 4. **Secrets:** never put them on a command line or in the log. Use` input= `(stdin), a 0600 env-file (` executors.common.env_file_content `, which refuses control characters)`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

`removed in` finally `, or` executors.common.ansible_stdin_cmd() `for Ansible extra vars (never` -e @/dev/stdin `). **Call` ctx.redact(value) `at the point where the secret is revealed** (` secret_store.reveal(...) `) — every later log line masks it with` ** *`; this is mandatory since 1.1.0 and part of the` ExecContext `protocol. Validate anything that reaches a shell (` check_mgmt_ip `for addresses,` shlex.quote `). 4b. **Audit:** a route that creates, changes or deletes something records an event — either with` audit(session, ".", target, request, detail) `in the handler, or declaratively by adding` (method, path regex, event) `to` core/audit.ROUTE_EVENTS `(recorded on 2xx with` METHOD path `+ path params as detail, never the body). Name events` . `so the` area. `filter works. 5. **Tests:** fake` SSHClient.connect `with a scripted runner, submit through the API with the` client `fixture,` await runner.wait(job_id) `, assert on the job row and` read_log(job_id) `. 6. **Frontend:** call the endpoint via an` api/*.ts `client and wrap it with` drawer.track(...) `from` useJobDrawer() `so the job drawer follows the log. 7. **Changelog:** add an entry under` ## [Unreleased]`.

Role checks come for free: a `POST` is operator-only through RoleGuard. Use `require_role("admin")` for admin-only routes (and keep them under `/api/admin` or `/api/users` so RoleGuard agrees).

9.6 How to add a page or a lab tab

STEP	PAGE	LAB TAB
component	`frontend/src/pages/<Name>.tsx`	`frontend/src/pages/lab/<Name>Tab.tsx` taking `{ lab }`
route / registration	`<Route path="…">` inside the layout route in `App.tsx`	add `{ id, label }` to `TABS` in `LabDetail.tsx` and the render switch; add the id to `ADVANCED_TABS` to hide it in Beginner mode
navigation	add a `NavItem` to `NAV_GROUPS` in `components/Layout.tsx` (`advanced: true` hides it in Beginner mode, `adminOnly: true` for admins); add a breadcrumb label to `SECTION`	—
data	an `api/*.ts` client with types in `api/types.ts`; TanStack Query hooks in the component	same
mocks	handlers in `src/mocks/handlers*.ts` so mock mode and tests work	same
help	a `<Hint>` for Beginner mode; optional tour steps (`learn/tours.yml`, `data-tour` attributes)	same
tests	a vitest file in `src/test/` rendering with the MSW handlers	same
Getting started	add a card to the menu list in `pages/GettingStarted.tsx`	—

Styling uses the `--cv-*` CSS variables mapped into Tailwind (`cv.*`) and the shared classes in `components/ui.tsx` (`btn-primary`, `card`, `input`, `table`, `badge`, `Tabs`, `Field`, `ErrorBox`, `ConfirmButton`, …). Each routed page and each lab tab already sits inside an error boundary.

9.7 Conventions that keep the codebase coherent

CONVENTION	WHY
AVD inputs are the only source of truth; new features derive from build outputs	one model, no drift (chapter 1)
every slow action is a job with a clear log and an actionable error	the UI is a job runner with forms on top
secrets: write-only APIs, Fernet at rest (MultiFernet, rotatable), stdin / 0600 files in transit, **never on argv**, `ctx.redact()` at every reveal	chapter 2 §(f)
every secret row has an owner and a `purpose`; bindings go through `require_binding()` (ownership + purpose, admins bypass)	operators cannot use each other's credentials
every mutation leaves an audit event with an actor (`audit()` or `ROUTE_EVENTS`)	the Audit log page tells the truth
admin-only routes live under `/api/users`, `/api/admin`, `/api/audit` **and** use `require_role("admin")`; everything else is operator-writable, viewer-readable	RoleGuard and the router agree
external URLs from settings or users are validated (`https://` for manifests, no private addresses for AI base URLs unless allowed, no local git remotes unless allowed, no `http://` / `git://` tokens)	SSRF and credential leaks
hrefs built from server or admin data go through `safeHref()` / `<SafeLink>`	no `javascript:` / `data:` links from data
third-party images and GitHub actions are pinned by digest / commit SHA; Python deps by hash	reproducible, auditable builds
SSH, Docker and HTTP go through one seam each (`SSHClient.connect`, `images.docker.backend`, `http_client()`)	tests fake them
new columns go into `core/db._ADDED_COLUMNS`	`create_all` never alters existing tables
each workstream owns its router module, appended to `FEATURE_MODULES`	parallel work without merge conflicts (contracts)
settings are `AVDLAB_*` fields in `core/config.py`; admin-editable ones also in `core/app_settings.py`	one place for defaults
every user-visible change gets a CHANGELOG line; every shipped fix a patch version	the version badge tells the truth

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

9.8 Design contracts

CONTRACT	VERSION	SCOPE
`CONTRACT.md`	0.1.0	phase 0 + module 1 + module 2a: repo, job engine, settings, hosts and bootstrap, templates, build, topology, images, first UI
`CONTRACT-2b.md`	0.2.0	containerlab executor: deploy, destroy, status, validate; remote layout; Runtime and Validation tabs
`CONTRACT-2c.md`	0.3.0	CloudVision: targets, onboarding token, TerminAttr, cv_deploy, tags, `cloudvision` knob
`CONTRACT-3.md`	0.4.0	clients and traffic, git coupling, running configs and snapshots, custom labs, host monitoring, cEOS arm, web terminal, VS Code
`CONTRACT-3b.md`	0.5.0	per-device tags, kill job, kill container
`CONTRACT-4.md`	0.5.0	ZTP (module 3), NetBox (module 6), MSS foundation (module 4), ACT executor
`CONTRACT-5.md`	0.6.0	editable topology, changelog, feedback, About/copyright, version badge, logo
`CONTRACT-6.md`	0.7.0	updates, all AVD examples, in-app learning, interactive test hosts
`CONTRACT-7.md`	0.8.0	multi-user auth with MFA, public TLS deployment, CloudVision robustness, error boundaries
`CONTRACT-8.md`	0.9.0	CloudVision-inspired UI, beginner/advanced mode, lean dashboard, account requests, admin settings, mail, intro video
`docs/_changes/A.md`, `B.md`, `C.md`	1.1.0	implementation notes of the three hardening streams: security (sessions, MFA policy, secrets, audit, headers), platform / supply chain (compose, Dockerfile, self-update, job engine, bootstrap), secure SDLC (CI, CodeQL, SBOM, frontend)

The architecture note `docs/10-app-architecture.md` (design v0.1) explains the original decisions; where it differs from the code (it planned arq/Redis and Postgres as default; the code uses an in-process engine and SQLite by default), the code and this documentation win. `docs/multi-os-hosts.md` is the research behind the Mac/Windows beta hosts.

9.9 Releasing

1. Put the changes under `## [Unreleased]` in `CHANGELOG.md` (Added / Changed / Fixed / Security).
2. Run the suites and the gates: `ruff check backend scripts`, `npm run lint && npm run typecheck`, `pytest -q`, `npm test`, `npm run build`, `pip-audit -r backend/requirements.lock --no-deps`, `npm audit --omit=dev`.
3. `scripts/release.py patch` (or `minor` / `major` / `X.Y.Z`; `--tar <dir>` for a tarball, `--pdf` for the documentation PDF): bumps `VERSION`, `backend/pyproject.toml`, `frontend/package.json`, rolls the changelog section, commits `release <version>` and creates the **annotated** tag `v<version>`. **Tag rules (1.1.0):** an existing tag is never moved — the script stops with *tag vX exists — bump again*; the tree must be clean after the release commit (`--allow-dirty` to override). With `--tar` it also writes `avdlab-<ver>.cdx.json` (CycloneDX 1.5 SBOM: pypi purls with SHA-256 hashes from `requirements.lock`, npm purls with SRI hashes from `package-lock.json`, dev packages `scope: excluded`, root component with git SHA and licence) and `SHA256SUMS`.
4. `git push && git push --tags` → `release.yml` re-runs both suites, asserts `tag == VERSION`, builds the multi-arch image to `ghcr.io/<owner>/<repo>:<version>` + `:latest`, produces the source SBOM (`scripts/sbom.py`) and an image SBOM (syft), attests SLSA provenance for the image and the assets, signs keylessly with **cosign** (`cosign sign`, `cosign attest --type cyclonedx`) and verifies in-job, writes `SHA256SUMS`, and creates the GitHub Release (`gh release create --verify-tag`). The release notes are the changelog section plus a **Supply chain** section containing the digest line `<image>:<version>@sha256:…` — the self-update parses it (chapter 3 §3.3) — and the verification commands.
5. Installs with `AVDLAB_UPDATE_SOURCE=github:<owner>/<repo>` see the new version within a day (or on *Check now*) and can apply it in image mode because the release is digest-pinned; set `AVDLAB_UPDATE_COSIGN_IDENTITY` to `^https://github.com/<owner>/<repo>/.github/workflows/release.yml@refs/tags/v` to verify the signature.

Details of the script and the workflow names: chapter 8 §8.5.

9.10 CI, code scanning and dependency updates (1.1.0)

WORKFLOW / FILE	WHAT IT DOES
`.github/workflows/ci.yml`	on every push and pull request: `lint` (ruff backend + scripts, eslint, tsc), `test-backend` (pytest, coverage xml artefact + step summary), `test-frontend` (vitest, lcov artefact, `npm run build`), `sca` (`pip-audit` on the lock, `npm audit --omit=dev --audit-level=high`; the full audit report-only), `secrets` (gitleaks over the full history with `.gitleaks.toml`; an organisation repository needs the `GITLEAKS_LICENSE` secret), `build` (image build, Trivy failing on fixable CRITICAL/HIGH, SARIF with MEDIUM+ uploaded to code scanning except for fork PRs). Minimal `permissions` per job, superseded runs cancelled, every action pinned to a commit SHA with the version in a trailing comment
`.github/workflows/codeql.yml`	CodeQL `security-extended` for `python` and `javascript-typescript` on pushes to `main`, pull requests and weekly
`.github/workflows/release.yml`	see 9.9; permissions `contents: write, packages: write, id-token: write, attestations: write`
`.github/dependabot.yml`	weekly updates for pip, npm, github-actions and docker (`/deploy`, `/agent`); minor + patch grouped; `pyavd` / `anta` ignored
`.github/CODEOWNERS`, `PULL_REQUEST_TEMPLATE.md`	review ownership; a security checklist in every PR
`SECURITY.md`, `CONTRIBUTING.md`	how to report a vulnerability; how to contribute (lockfile refresh, lint, tests)
`.pre-commit-config.yaml`, `.gitleaks.toml`, `ruff.toml`	local hooks; secret-scanning allowlist; ruff for `scripts/`

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks

Branch protection on `main`: require the checks `lint`, `test-backend`, `test-frontend`, `sca`, `secrets`, `build` (from `ci.yml`) and `analyze (python)`, `analyze (javascript-typescript)` (from `codeql.yml`); also *Require signed commits*, *Require linear history*, *Require review from Code Owners* and *Dismiss stale approvals*.

Known follow-ups at 1.1.0: `ruff format --check` would reformat many files (report-only until a one-shot formatting PR); 13 `npm audit` advisories remain in dev-only dependencies (vite 5 / vitest 2 / tailwind 3 transitive) whose fixes are major upgrades; the AI designer still receives the decrypted ACT password of ACT labs (chapter 6 §6.5).

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks



avdlab 1.1.0 — documentation generated 22 October 2026 from `docs/1.1/` and the application source.

© 2026 Dietrich De Bruyne — Systems Engineer, Arista Networks. Arista, EOS, CloudVision and AVD are trademarks of Arista Networks. containerlab, ANTA and NetBox belong to their respective projects.